In this course, we will move from being new to Codex to using it productively. We will develop six practical abilities that help us direct Codex clearly, preserve useful context, create reusable workflows, and control how Codex operates.

What we will learn

We will begin by learning to describe the outcome we want instead of supplying only a sequence of steps. This approach helps Codex understand the goal and produce more precise, usable results.

We will then build confidence with the essential setup and operation of Codex. This includes installing and configuring Codex correctly, reading its terminal interface, and understanding how data privacy affects our work.

The course also covers how to manage and resume sessions strategically. By preserving valuable context, we can continue longer tasks without rebuilding work that has already been completed.

To improve the quality of our results, we will learn how to write effective prompts using:

- Relevant context
- Clear constraints
- Concrete examples
- Plan mode

We will create an AGENTS.md file that captures project guidance and turns recurring mistakes into permanent rules. This allows Codex to follow lessons learned across future sessions.

Finally, we will save reusable workflows as skills and use the config.toml file to control Codex permissions and sandbox behavior.

Course prerequisites

Before starting the hands-on work, we need three essentials in place:

- **Basic terminal familiarity:** We should be comfortable navigating directories and running commands from the command line.
- **Node.js version 18 or higher:** Codex is installed through npm, so a recent version of Node.js is required.
- **An OpenAI account:** We need access through the OpenAI API or an eligible ChatGPT subscription to use Codex.

With these prerequisites ready, we can begin the first module and start building productive Codex workflows.

© 2026 Zenva Pty Ltd. All rights reserved.

AI coding changes our role in software development. Instead of manually implementing every line, we can guide an AI agent toward a clear result, establish the boundaries it must respect, and review the work it produces. This director mindset helps us use tools such as Codex more effectively.

Moving from typist to director

Traditional development often places us in the role of a typist. We imagine a feature, plan its implementation, and write the code line by line. Much of our time is spent on manual execution.

With Codex, we can move into the role of a director. We describe the outcome we want and guide an AI agent as it works toward that objective. Our attention shifts toward strategy, clear direction, constraints, and quality control. We still lead the process, but we do not need to perform every implementation step ourselves.

Giving Codex clear direction

Codex can work across a project to complete complex tasks. For example, it can write functions, run tests, fix bugs, and clean up files. However, its work depends on the instructions we provide. It follows our direction closely rather than independently guessing what we intended.

Ambiguous instructions can produce ambiguous or unsatisfactory results. Precise guidance gives Codex a clearer target and makes useful results more likely. In practice, the quality and clarity of our input strongly influence the quality of the output.

Understanding agentic work

An agentic AI tool does more than answer questions. It actively performs tasks inside a project through a continuing work loop:

- We provide a prompt with an objective.
- Codex acts by reading files, writing code, or running commands.
- It checks the results of those actions.
- It decides what to do next and reports back.

This prompt, act, check, and report loop turns the interaction into practical delegation. We are assigning work to an agent that can execute and verify tasks, not simply holding a conversation with a chatbot.

Describing outcomes instead of steps

A common mistake is to treat Codex like a typist by dictating every implementation detail. For example, we might specify a function name, its parameter, the exact method it should call, and what it should log. This keeps the implementation burden on us and limits Codex’s ability to choose an effective solution.

Outcome-based prompting focuses on the desired behavior or user experience instead. We could ask for a form submission to validate its inputs and, when they are valid, add a new task to the list without refreshing the page. This defines what should happen while allowing Codex to make intelligent implementation choices.

Our three duties as director

Using Codex as an agent gives us three core responsibilities:

1. **Define the outcome:** State clearly what should exist or work after the task is complete,

© 2026 Zenva Pty Ltd. All rights reserved.

from the user’s or system’s perspective.
2. **Set constraints:** Specify the boundaries Codex must follow, such as the technology stack, naming conventions, or files it must not modify.
3. **Review and decide:** Evaluate the proposed solution and decide whether it meets the required standard.

Codex can propose and implement solutions, but we remain the final quality gate. Our expertise and review determine whether the work is sound and ready to accept.

Adopting the new mental model

The key shift is from manually producing every line of code to directing an agent toward clear goals. Agentic tools can read, write, run, check, and report within a project, while outcome-focused prompts let them use those capabilities effectively. By defining outcomes, setting limits, and carefully reviewing the result, we can guide Codex while retaining control over the final product.

© 2026 Zenva Pty Ltd. All rights reserved.

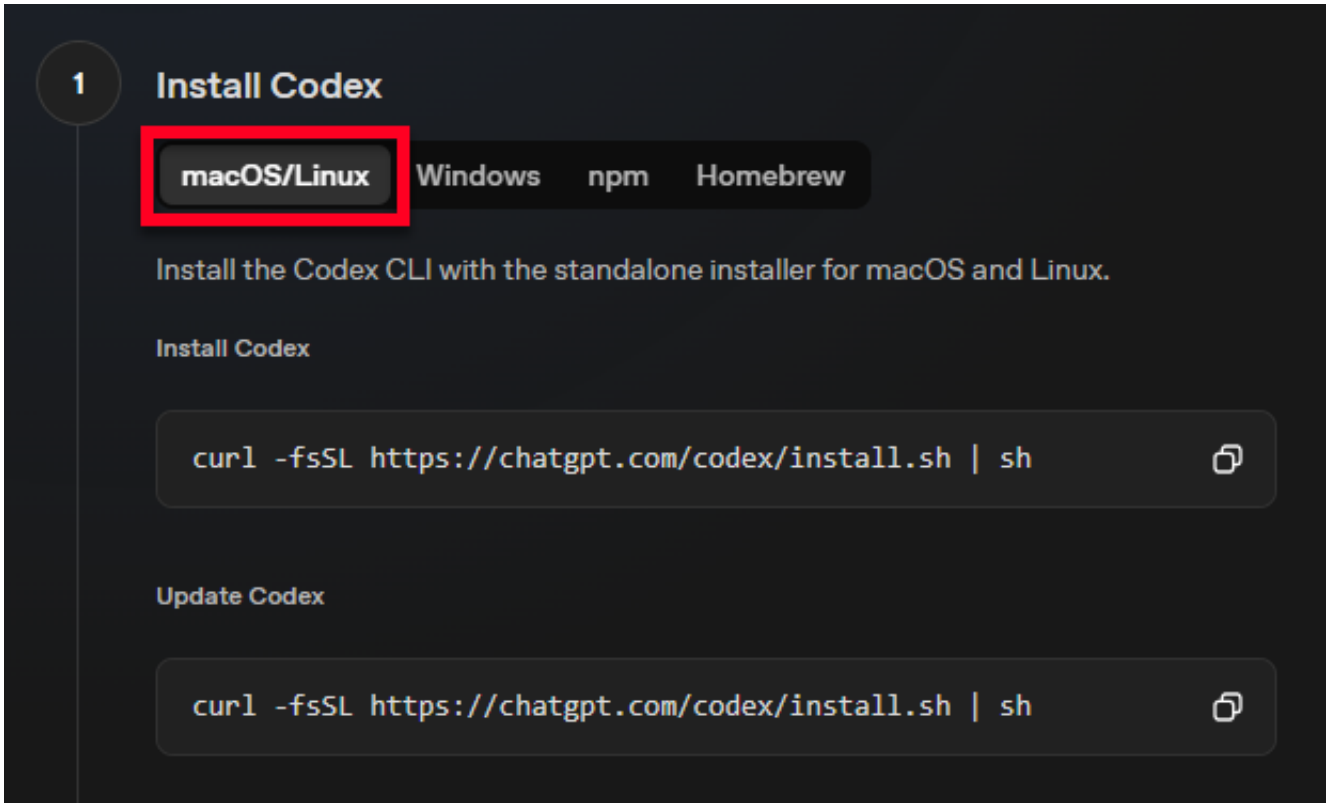
Installing Codex

Before we start using Codex inside real projects, we need to make sure our computer has the right tools installed.

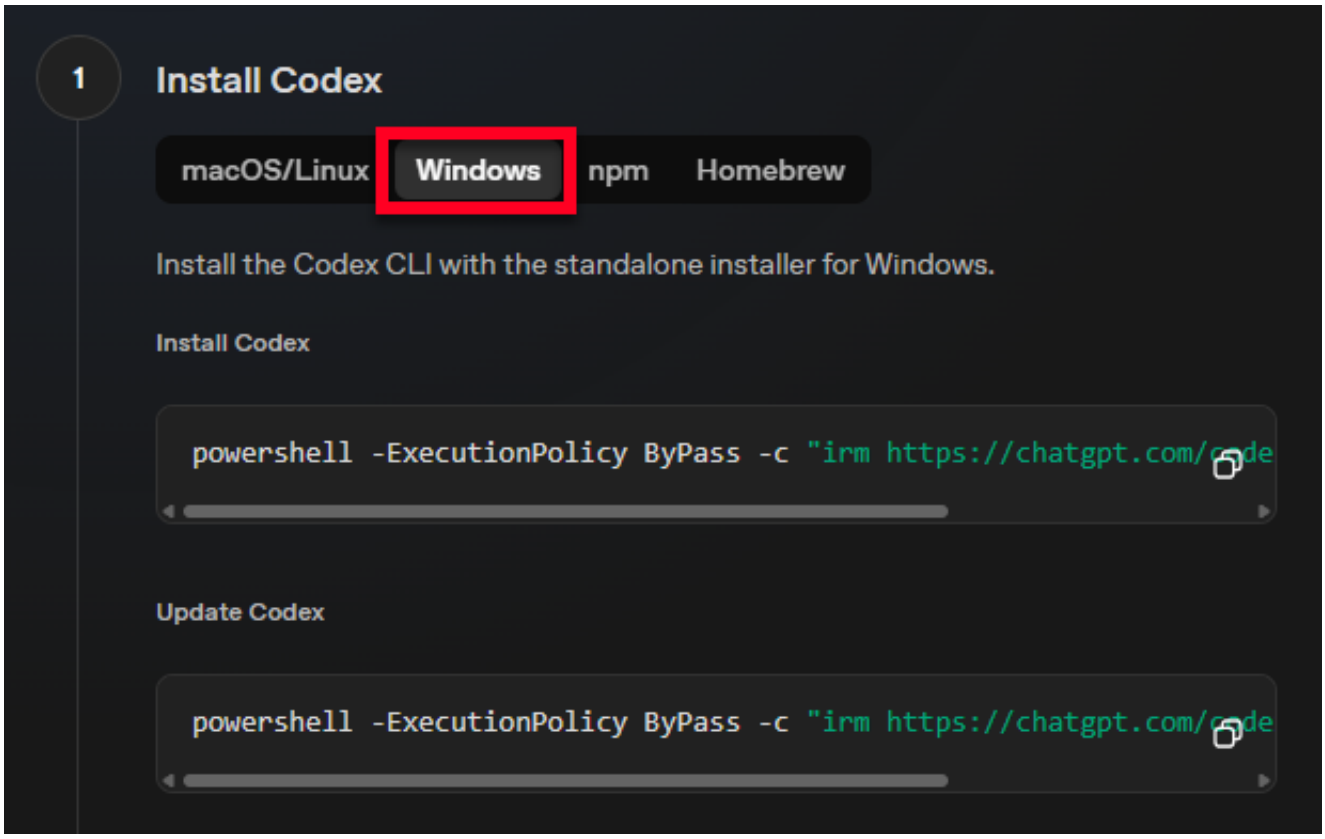
- **Key update.** Node.js is not needed for the recommended standalone Codex installer, but it is needed to run the React project files later in the course.

Recommended Installation

For most learners, the recommended path is the standalone installer. The latest official instructions are available in the Codex CLI documentation at https://learn.chatgpt.com/docs/codex/cli. Open a terminal and run the command for your operating system.



© 2026 Zenva Pty Ltd. All rights reserved.

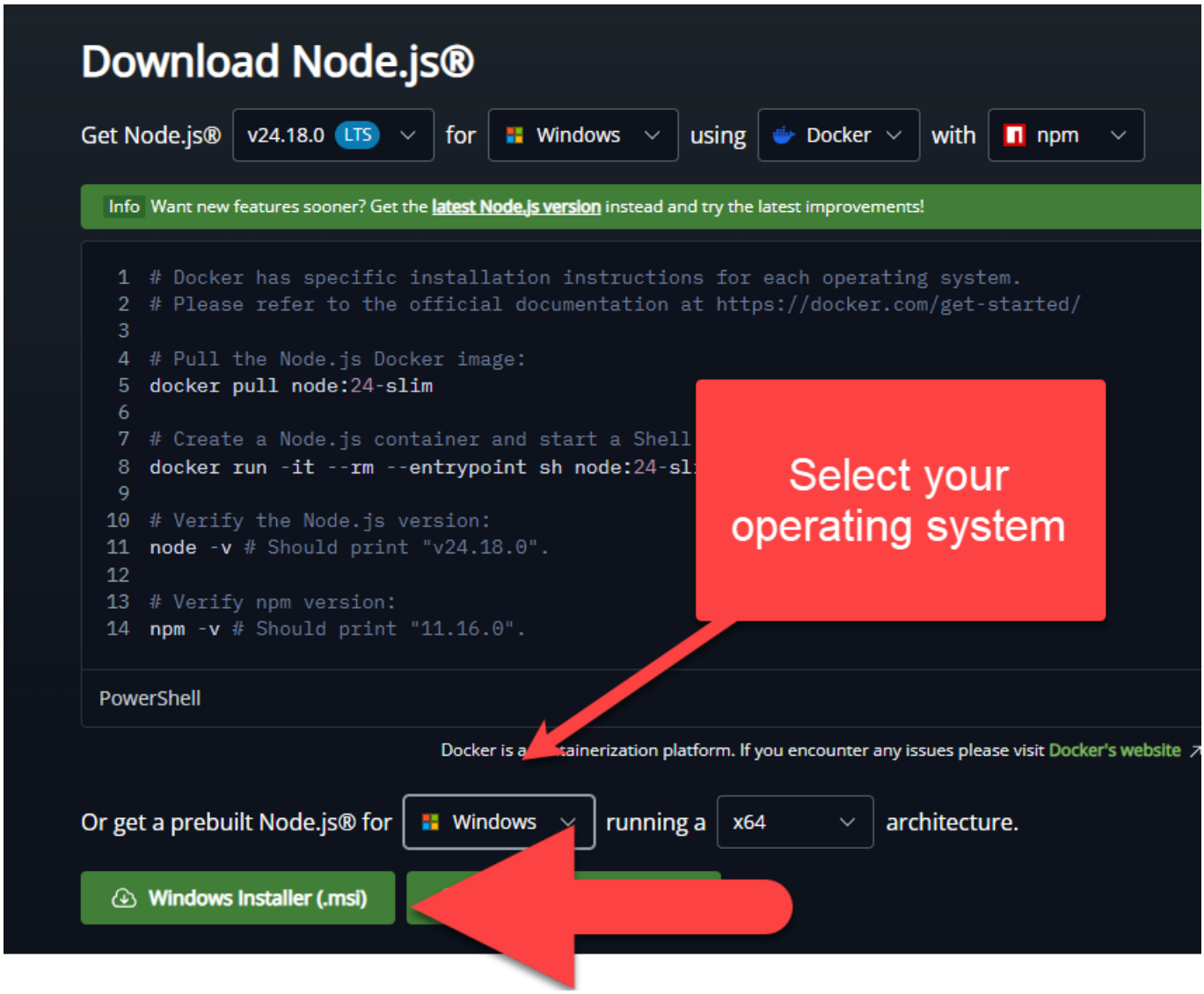


- **macOS, Linux, WSL** Run curl -fsSL https://chatgpt.com/codex/install.sh | sh in your terminal.
- **Windows PowerShell** Run powershell -ExecutionPolicy ByPass -c "irm https://chatgpt.com/codex/install.ps1 | iex" in PowerShell.
- **Other installers** You may also see installation options for Homebrew or npm. These are alternative paths, but the standalone installer is the simplest option to start with.

Install Node.js

Node.js lets your computer run the React project files used in this course. Go to https://nodejs.org/download and select your operating system. Then choose the prebuilt installer for your platform.

© 2026 Zenva Pty Ltd. All rights reserved.



Alternative Codex Installation

This is NOT needed if you already installed Codex using the standalone installer presented earlier. If you do choose the npm installation method for Codex, you can run this command:

```
npm install -g @openai/codex
```

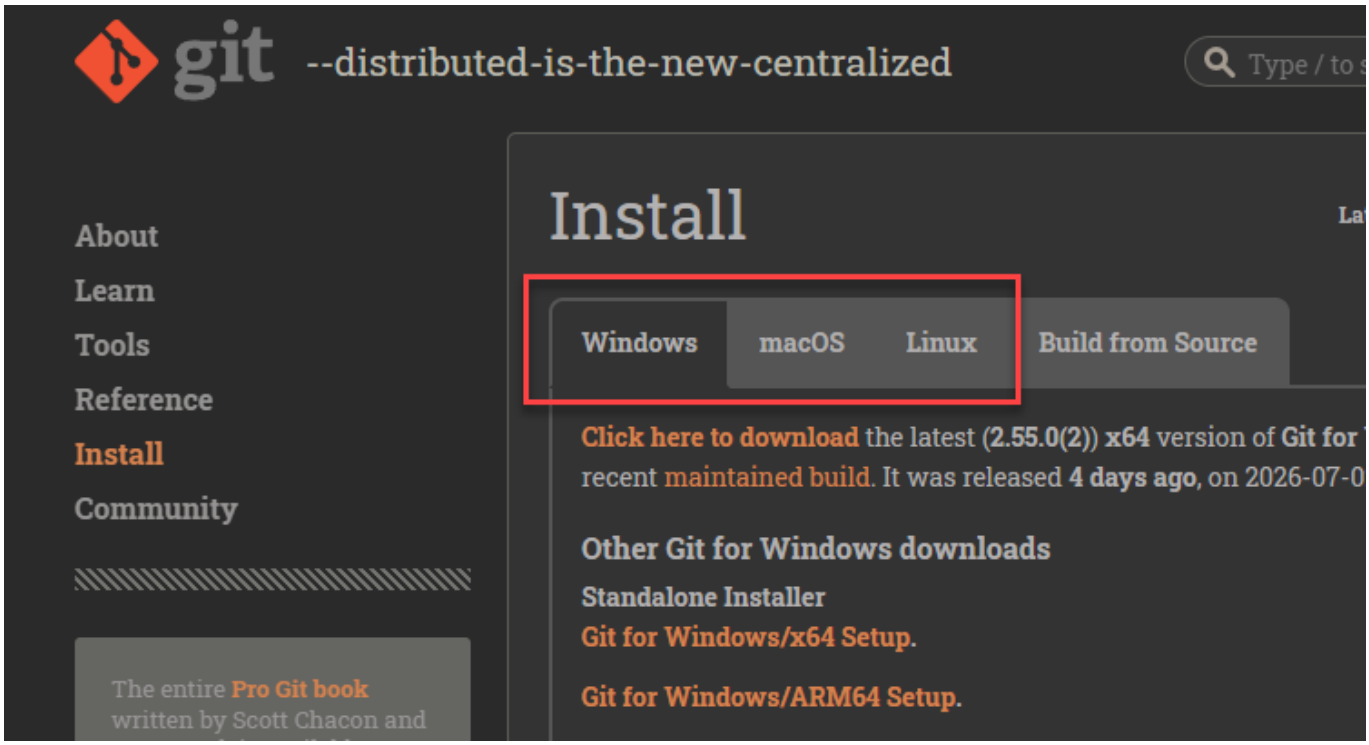
- Use the standalone Codex installer if you want the simplest setup path.
- Install Node.js so you can run the course project files locally.

Install Git

- **What is Git?** Git is a version control tool that tracks changes in a project over time.
- **Why is it important?** Git gives us a safety net when working with code. We can review what changed, save good versions of our work, and roll back changes if needed.

Go to https://git-scm.com/downloads and choose your operating system. The site will show the installer or installation instructions that match your computer.

© 2026 Zenva Pty Ltd. All rights reserved.



Once Git is installed, your computer will be ready to work with repositories and version-controlled projects.

Ready to continue

In the next lesson, we will continue setting up Codex and cover the important setup details that help you use it effectively inside real projects.

© 2026 Zenva Pty Ltd. All rights reserved.

Using the To Do List App Starter Project

Before asking Codex to change the To Do List app, make sure the practice project is on your computer and running locally.

- **Use the starter project for the lessons.** The complete project is there as a reference if you want to compare your work later.

Step 1: Download the project files

Open the **Course Files** tab for this course and download:

- **Starter Projects – Codex Intro** Use it to follow the course exercises.
- **Completed Projects – Codex Intro** Use it only as a reference of the complete work.

After downloading the starter ZIP, unzip it on your computer.

Step 2: Open the Todo project

For the first real task, open this folder in Visual Studio Code:

```
? Starter Projects - Codex Intro
  ? To Do List App
    ? react_project ? open this folder
```

Make sure you open the react_project folder itself. This is the folder that contains package.json.

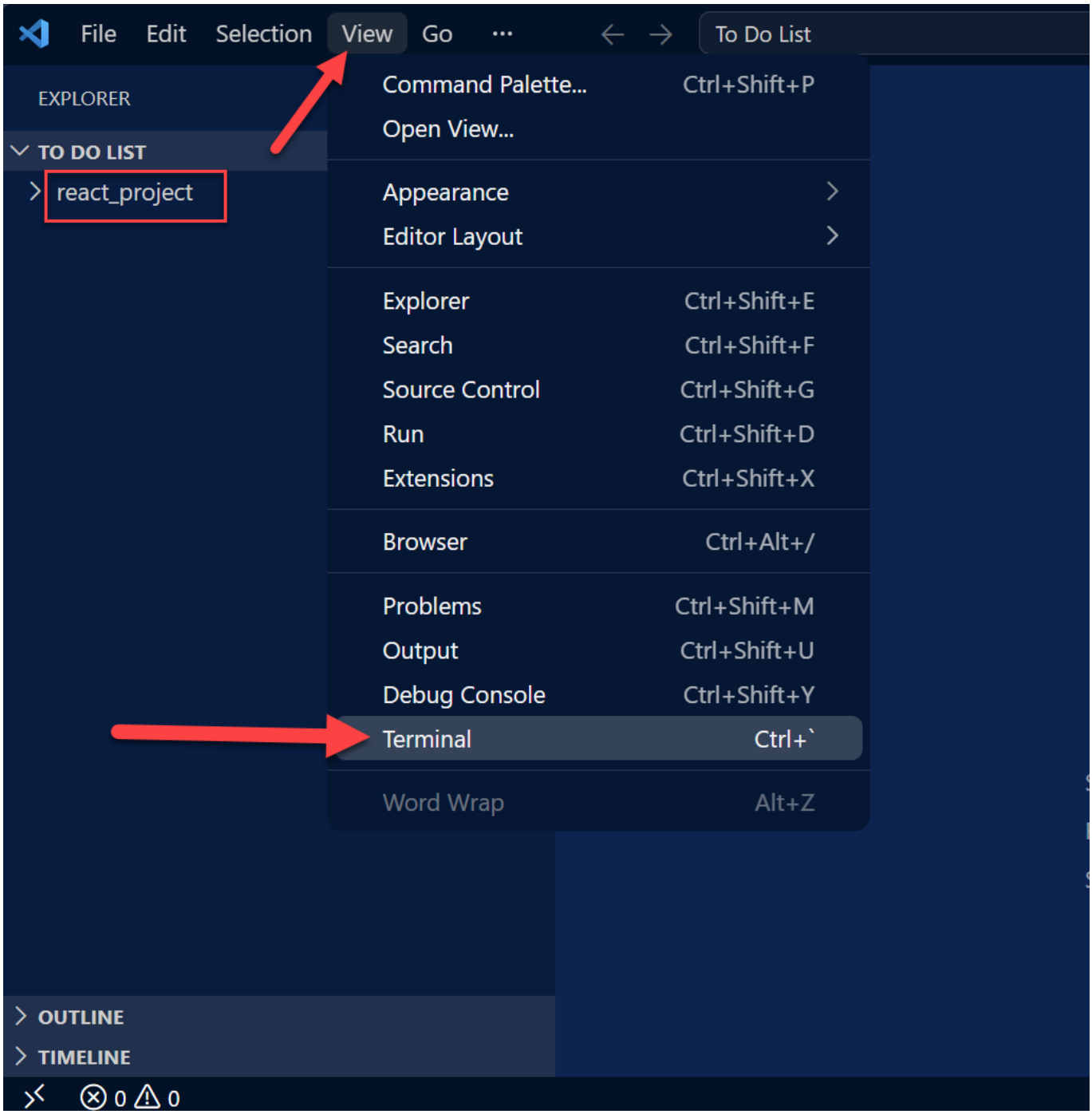
- **What is package.json?** It is a small settings file for the project. It tells Node.js which tools the app needs and which commands you can run, such as npm run dev.

Step 3: Run the app locally

- **Node.js note.** Node.js is optional for installing Codex with the standalone installer, but it is needed to run this React project locally.

Open the VS Code terminal while you are inside the react_project folder:

© 2026 Zenva Pty Ltd. All rights reserved.



Then, in the **Terminal**, we will run npm like so:

```
npm install
npm run dev
```

- **What is npm?** npm is the tool that downloads the project tools listed in package.json and runs commands for the app. npm is included in the official Node.js installer.

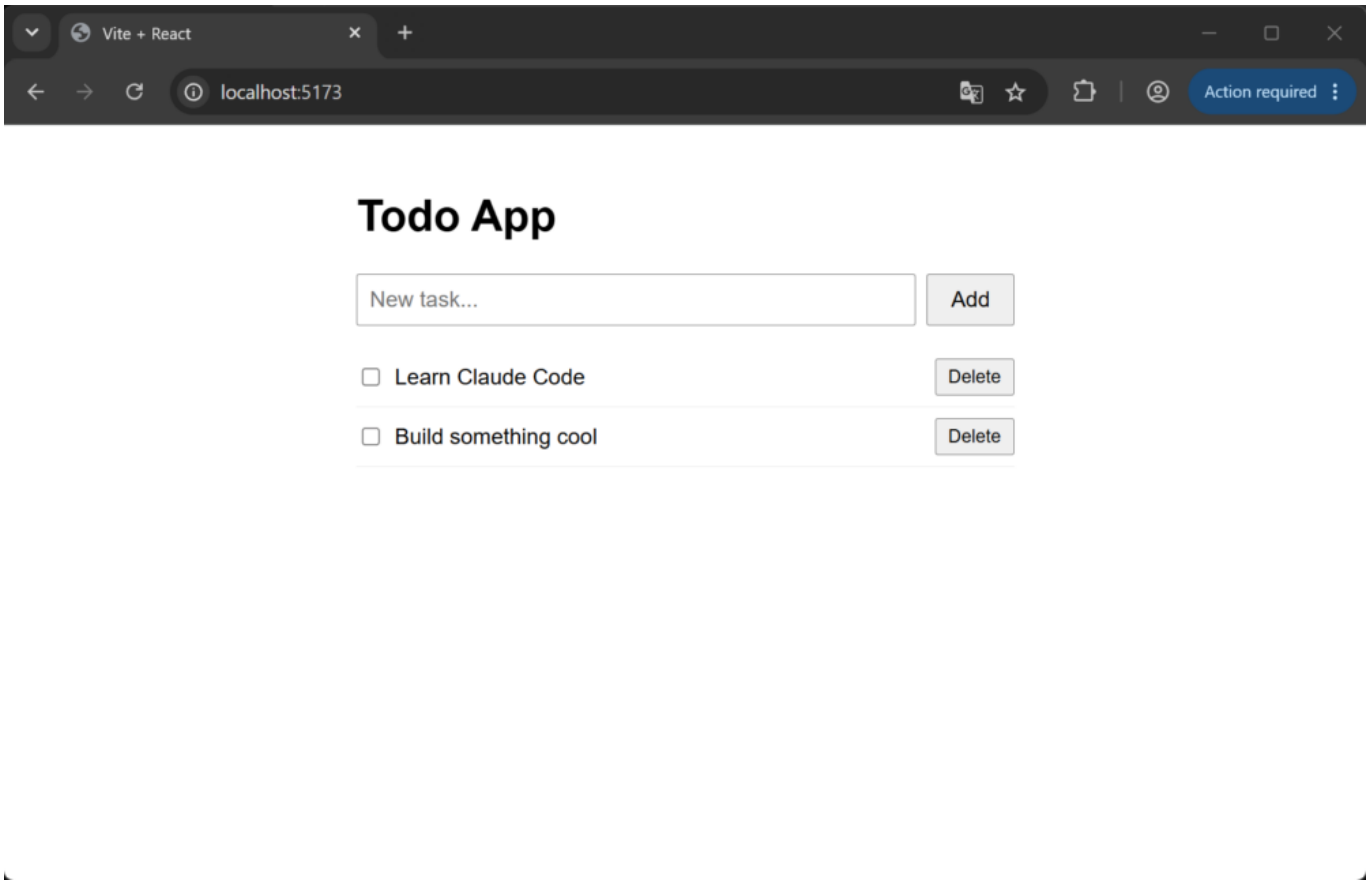
The terminal will show a local URL, usually:

```
http://localhost:5173
```

© 2026 Zenva Pty Ltd. All rights reserved.

Open that URL in your browser. Keep the terminal running while you work on the app.

You should see the To Do List app in your browser.



Step 4: Start Codex in the same folder

In the project terminal, start Codex:

```
codex
```

Then ask for one small change at a time. For example:

```
Add a simple filter so I can view all tasks, active tasks, or completed tasks. Keep t
he current app simple and explain the files you changed.
```

Step 5: Save a safe starting point

Before Codex changes the project, create a Git checkpoint:

```
git init
git add .
git commit -m "Initial starter project"
```

© 2026 Zenva Pty Ltd. All rights reserved.

After Codex makes changes, you can review them with:

```
git diff
```

If something does not work

- If npm is not found, install Node.js from https://nodejs.org/download and try again.
- If npm install fails, check that your terminal is inside the react_project folder.
- If the browser does not show the app, use the exact local URL printed in your terminal.
- If Codex cannot find the files, start Codex from the same react_project folder.

Once the app is running and Codex is open in the project folder, continue to the next lesson.

© 2026 Zenva Pty Ltd. All rights reserved.

In this lesson, we will prepare Codex for practical work, establish a safe Git workflow, and complete a first low-risk interaction with an existing project. We will also create an initial AGENTS.md file so Codex has useful project context from the beginning.

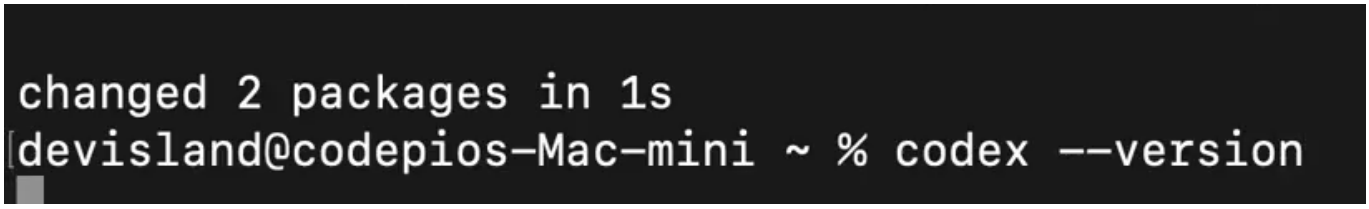
Check the prerequisites

Before installing Codex, make sure the following tools and access are ready:

- Node.js version 18 or later, which provides the runtime used by the Codex command-line tool
- An OpenAI account with either API access or an eligible ChatGPT subscription
- Git, which will track every change made during the course

Install and verify Codex

Install Codex globally by running npm install -g @openai/codex in a terminal. Once the package installation finishes, run codex --version. A displayed version number confirms that the CLI is available. The first time Codex starts, it may also prompt us to log in and connect the session to our OpenAI account.



Create a clean Git baseline

Before asking an agent to work on a project, we should create a traceable starting point. From the project folder, run git init to create the hidden .git directory. Next, use git add . to stage the current files, then create an initial commit. This commit becomes a clean rollback point if later changes do not work as expected.

Four Git commands are especially useful in daily agent-assisted work:

- git add . stages the current changes.
- git commit saves the staged work as a checkpoint. Create a clean commit before each Codex session.
- git diff displays line-by-line changes so we can review what Codex or we have modified.
- git restore discards unwanted local changes and returns files to their last committed state.

Understand the bypass flag

The command codex --dangerously-bypass-approvals-and-sandbox "Your instruction here" removes approval prompts and sandbox protections. This allows uninterrupted execution, but it also removes important safety checks. A misunderstood instruction could modify or delete critical files or compromise the system.

Use this flag only in a disposable, fully isolated environment, such as a temporary cloud test machine or a Docker container that contains no private information. Never use it on a primary development computer or in a project connected to production credentials, customer data, live databases, or active systems. For real projects, keep the normal approval and sandbox protections enabled.

Prepare the Todo project

© 2026 Zenva Pty Ltd. All rights reserved.

The course uses a simple Todo app to demonstrate Codex. The app can add a task, toggle its completed state with a checkbox, and delete it. Test these controls by entering Learn Codex, adding the task, marking it complete, and deleting it. Download the provided source files, initialize a Git repository in the project, and create the clean initial commit before continuing.

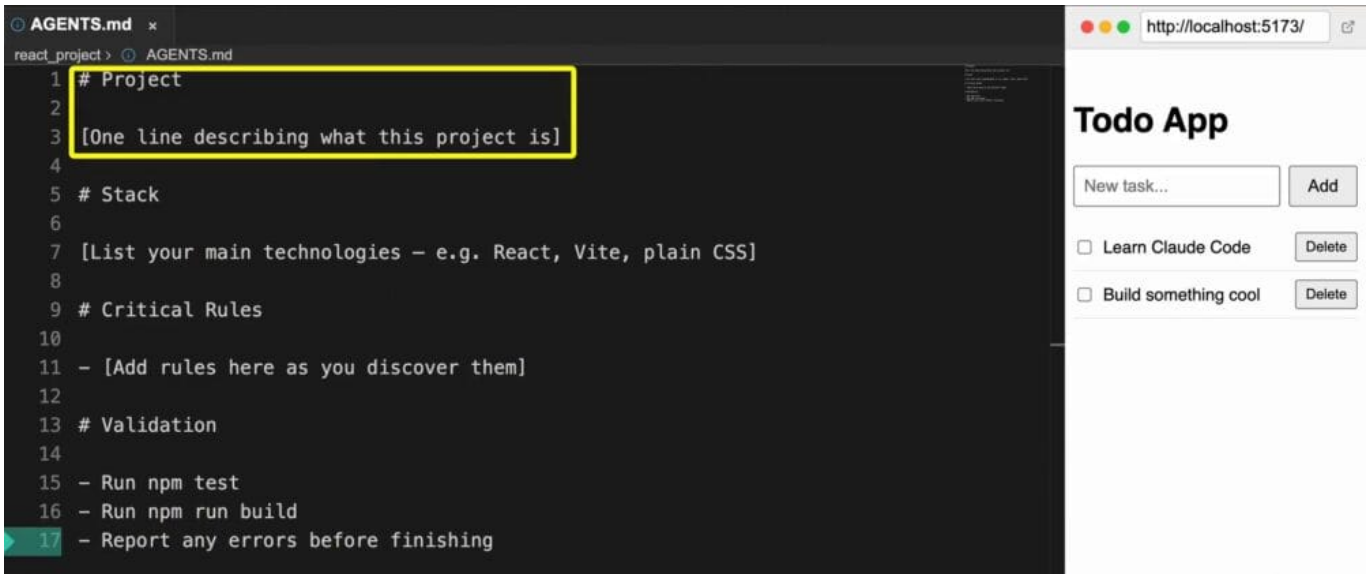


Create the first AGENTS.md file

Create AGENTS.md in the project root. This file gives Codex immediate context about the repository and provides durable instructions that can evolve with the project. Begin with these sections:

- **Project:** a short description of what the application does
- **Stack:** the main technologies, such as React, Vite, and plain CSS
- **Critical Rules:** important constraints that Codex must follow
- **Validation:** commands such as npm test and npm run build, plus an instruction to report errors before finishing

Commit AGENTS.md to Git so its history remains visible as the instructions become more detailed in later lessons.

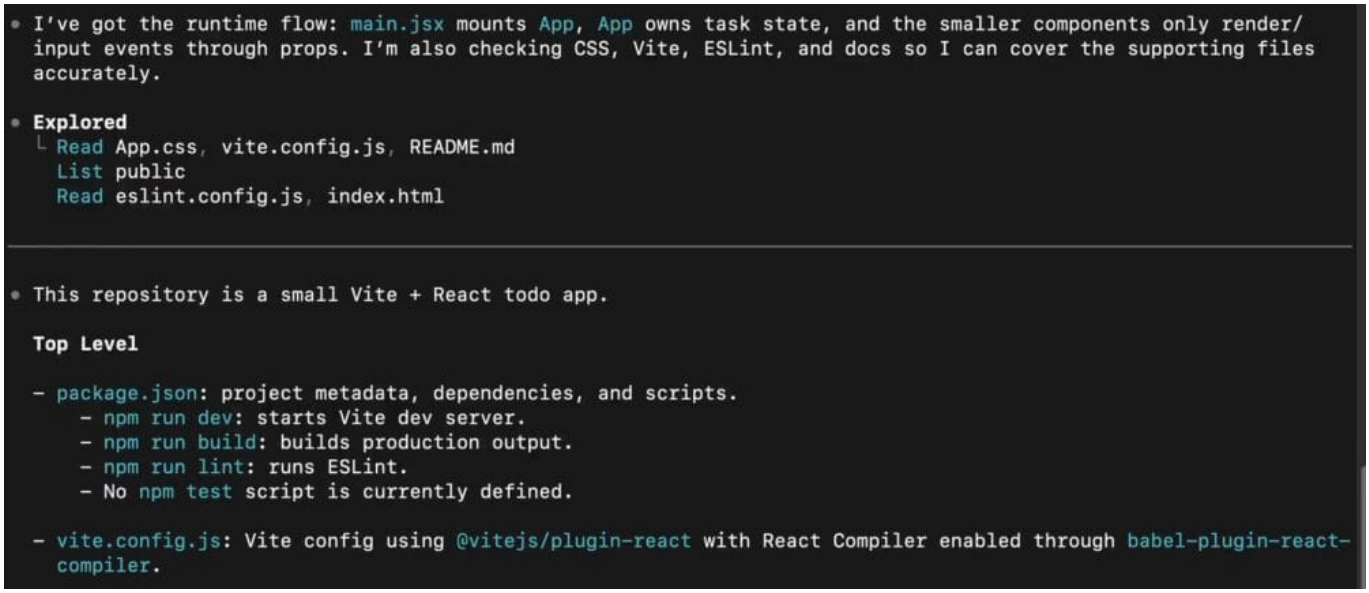


© 2026 Zenva Pty Ltd. All rights reserved.

Run the first Codex session

Open a terminal in the project root, not in a subfolder, because Codex treats the current directory as its workspace. Run codex, complete the login flow if prompted, and submit the simple request Explain the structure of this repository.

Codex first plans how to inspect the project, then lists relevant files while avoiding folders that do not need analysis. It examines the React entry point, application component, supporting components, styles, package configuration, and Vite configuration. Its response explains how main.jsx mounts the app, how the main application component owns task state, and how data and callbacks move through the input, list, and item components. It also summarizes available package commands and identifies current project limitations.



This first interaction asks Codex only to read and explain the repository. Starting with a small, low-risk request helps us confirm that the setup works and understand how the agent approaches the codebase before assigning larger implementation tasks.

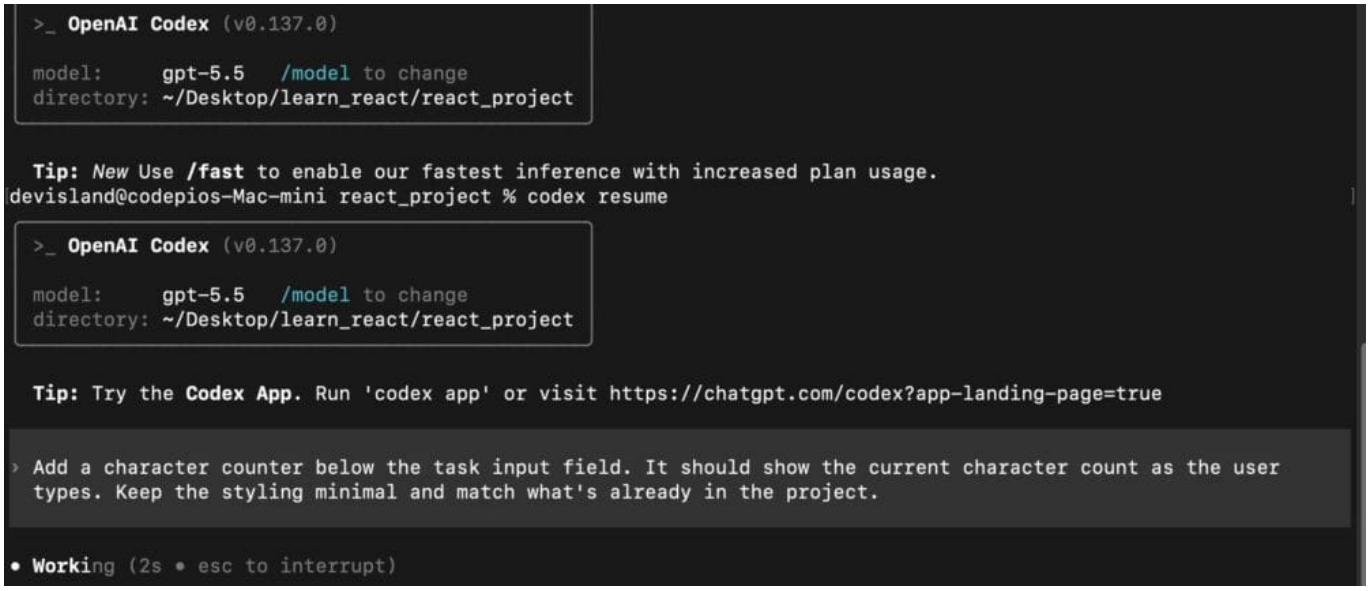
© 2026 Zenva Pty Ltd. All rights reserved.

Codex communicates continuously through its terminal interface. In this lesson, we will learn how to read its main signals, monitor its actions, choose an appropriate permission level, stop unwanted work safely, and diagnose setup problems.

Starting from a clean state

Before beginning a new task, we should commit the current project state. A clean commit gives us a reliable point to return to if an experiment goes wrong. We can then launch Codex from the project root.

If we need to continue earlier work, we can leave a newly opened session with Ctrl+C, run codex resume, and select the most recent session. This restores the previous conversation context so we can continue without restating everything.

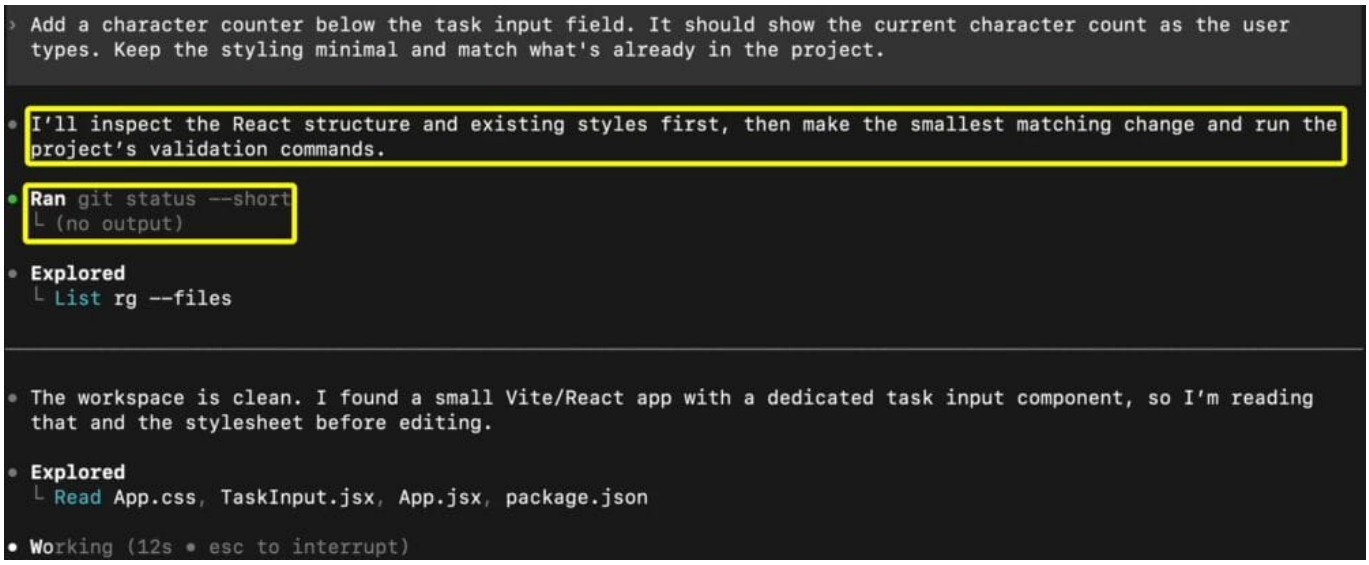


Reading the working signals

When we submit a task, Codex displays a working spinner. This means it is reading the prompt, loading project guidance such as AGENTS.md, and deciding what to do. At this initial stage, the spinner does not mean that files have already changed.

Codex then reports its plan, the files it explores, and the commands it runs. These tool-use notifications are valuable checkpoints. Reading them as they appear helps us confirm that Codex is inspecting the correct project areas and lets us catch an unwanted direction early. The terminal also reminds us that Escape can interrupt active work.

© 2026 Zenva Pty Ltd. All rights reserved.



Interpreting changes and validation

For the first task, we ask Codex to add a character counter beneath the to-do input. Codex inspects the React project, explains the intended change, and presents a colored diff. Removed lines appear separately from added lines, allowing us to review the exact modification.

Codex next attempts to validate the work. When npm test is unavailable because the project has no test script, it does not treat that as a successful test. Instead, it reports the limitation, runs npm run build, and confirms that the production build succeeds. Its final summary distinguishes what changed, which check failed to run, and which check passed.

A second request extends the same feature with a 100-character maximum and a counter that turns red at the limit. Codex follows the same pattern: it plans the work, updates the project, tries the available checks, and summarizes the outcome.

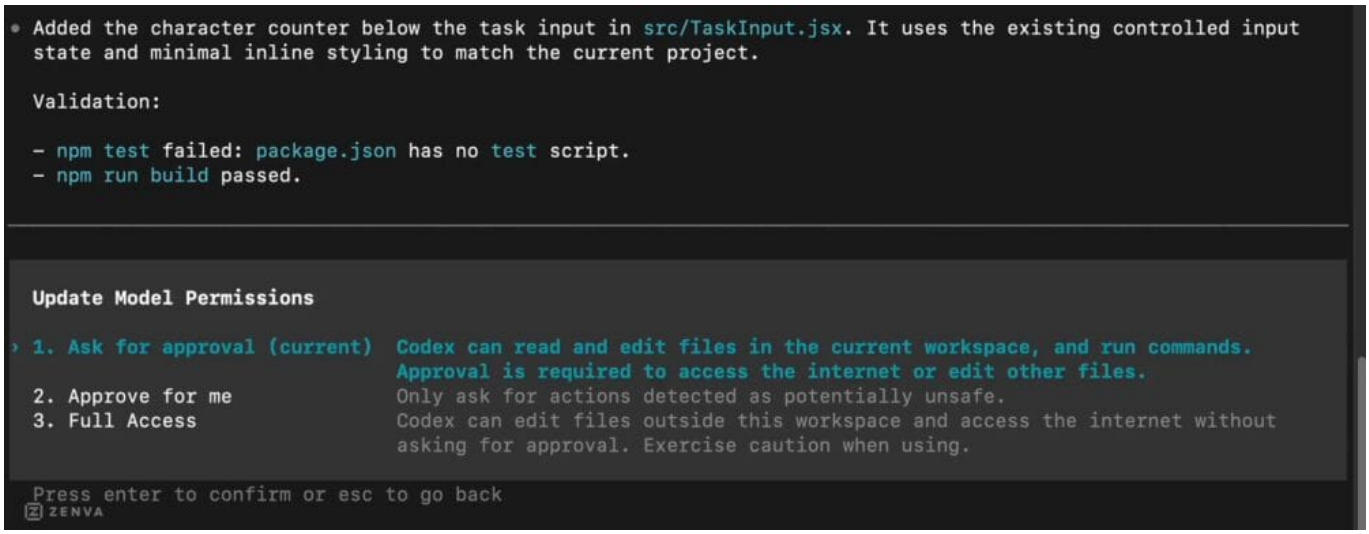
Choosing permissions

Typing a forward slash opens Codex's command list. The /permissions command controls how much Codex can do without stopping for confirmation:

- **Ask for approval** is the safest starting point. Codex can work inside the project but asks before operations such as internet access or work outside the current workspace.
- **Approve for me** reduces routine prompts and allows more commands to run immediately.
- **Full Access** gives Codex the broadest system and network access, so it also carries the greatest risk.

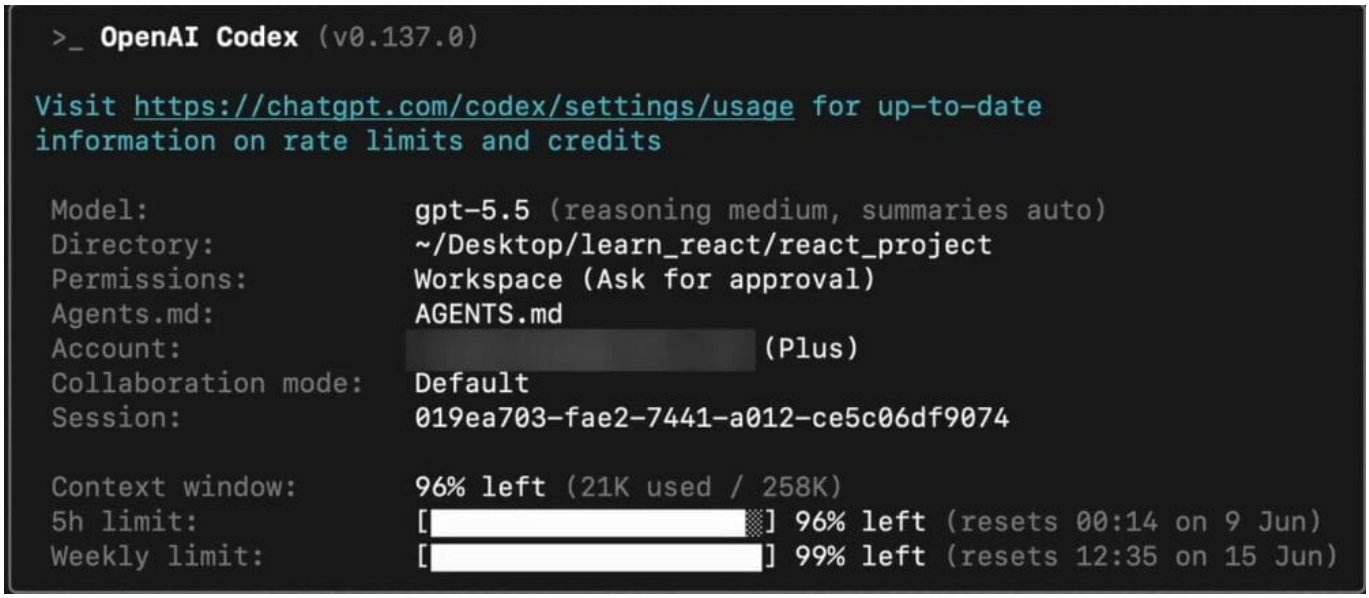
We should select the least permissive mode that still supports the task and review approval prompts carefully.

© 2026 Zenva Pty Ltd. All rights reserved.



Checking session status

The /status command provides an overview of the active session. It identifies the current model, project directory, and permission mode. It also reports remaining context capacity and subscription usage for the five-hour and weekly limits. Checking these values helps us understand the session's configuration and monitor resource use during longer tasks.



Understanding approval prompts

When we ask Codex to commit the completed character-counter changes, it first checks the diff and repository status. It confirms that only the intended file changed before staging it. Writing to Git's index crosses the active sandbox boundary, so Codex pauses and asks for approval before running the staging and commit commands.

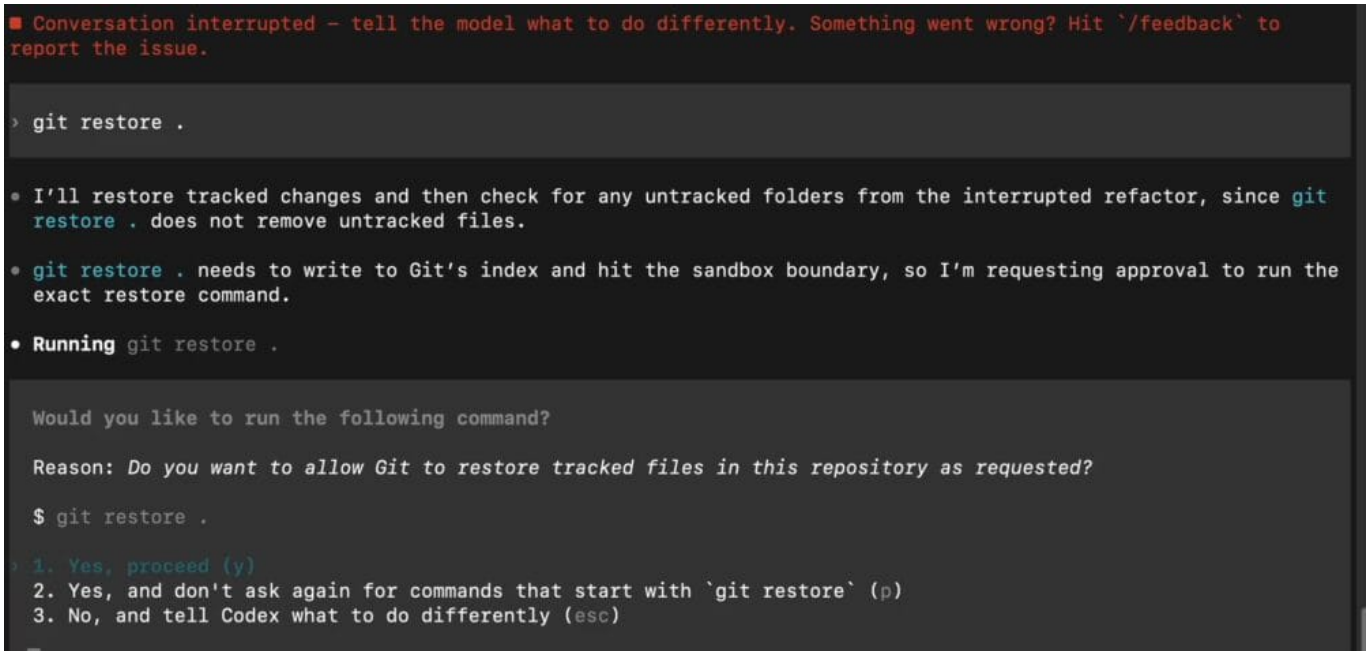
An approval request should be treated as a decision point, not a routine obstacle. We should verify the exact command, its stated purpose, and its possible effect before proceeding. After approval, Codex can stage the file, create the requested commit, and confirm that the working tree is clean.

Interrupting and recovering from unwanted work

© 2026 Zenva Pty Ltd. All rights reserved.

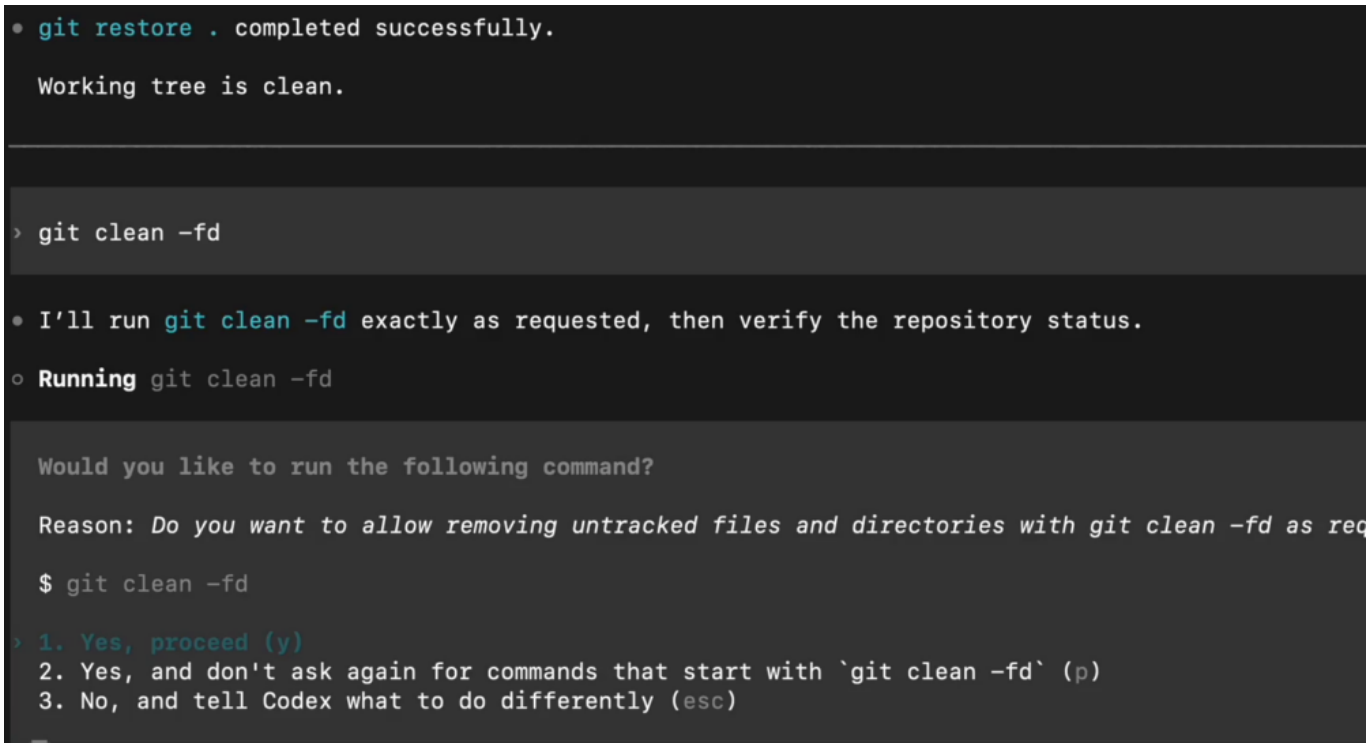
To practice staying in control, we give Codex an overly broad request to refactor the entire app. Codex maps the components, proposes a structural plan, and begins creating directories. Once we realize the task is not what we intended, pressing Escape stops the current operation while keeping the session and its history available.

Because we created a clean commit beforehand, we can request git restore . to revert tracked files. Codex explains that the command affects tracked changes and requests approval where required.



The restore command does not remove untracked files or folders. To return the workspace to a completely clean state, the lesson uses git clean -fd. This command permanently removes untracked files and directories, so Codex presents an explicit approval prompt before running it. We must inspect destructive commands with particular care and confirm only when the targets and outcome are correct.

© 2026 Zenva Pty Ltd. All rights reserved.



Diagnosing Codex

When Codex is not behaving as expected, we can exit the active session with /exit or Ctrl+C and run codex doctor from the regular shell. This health check examines the CLI version, authentication, and environment settings. It is a useful first troubleshooting step because it can reveal setup problems before they block further work.

By reading the spinner, plans, tool notifications, diffs, validation results, approval prompts, status information, and diagnostic output, we can understand what Codex is doing and intervene before an unintended action grows into a larger problem.

© 2026 Zenva Pty Ltd. All rights reserved.

Intro to Agentic Coding with Codex - Direct Codex with Control

1. You want Codex to add filtering, but implementation details can vary. What should your prompt define first?

a. The desired behavior and boundaries
b. Every line of the implementation
c. Only the function and variable names
d. No constraints, so Codex stays flexible

2. Codex begins a broad refactor you did not want. What should you do first?

a. Press Escape to interrupt it
b. Let it finish before reviewing
c. Switch the session to Full Access
d. Start a second unrelated request

© 2026 Zenva Pty Ltd. All rights reserved.

Codex combines cloud-based model assistance with local security controls. To use it safely, we need to understand which information can leave our machine, how privacy settings affect model training, and how network and sandbox restrictions limit Codex's actions.

Understand the data flow

Codex sends our prompt and the context needed to answer it to OpenAI's models. That context can include project files that Codex must read, the current session history, and the contents of our AGENTS.md file. These inputs help the model understand both the task and any project-specific instructions.

The rest of the project stays local. Codex does not automatically scan every file in the project. Unreferenced files, environment variables, and credentials remain on our machine unless we explicitly provide or reference them. This distinction gives us control, but it also means we must pay attention to which files we ask Codex to read.

Treat .gitignore correctly

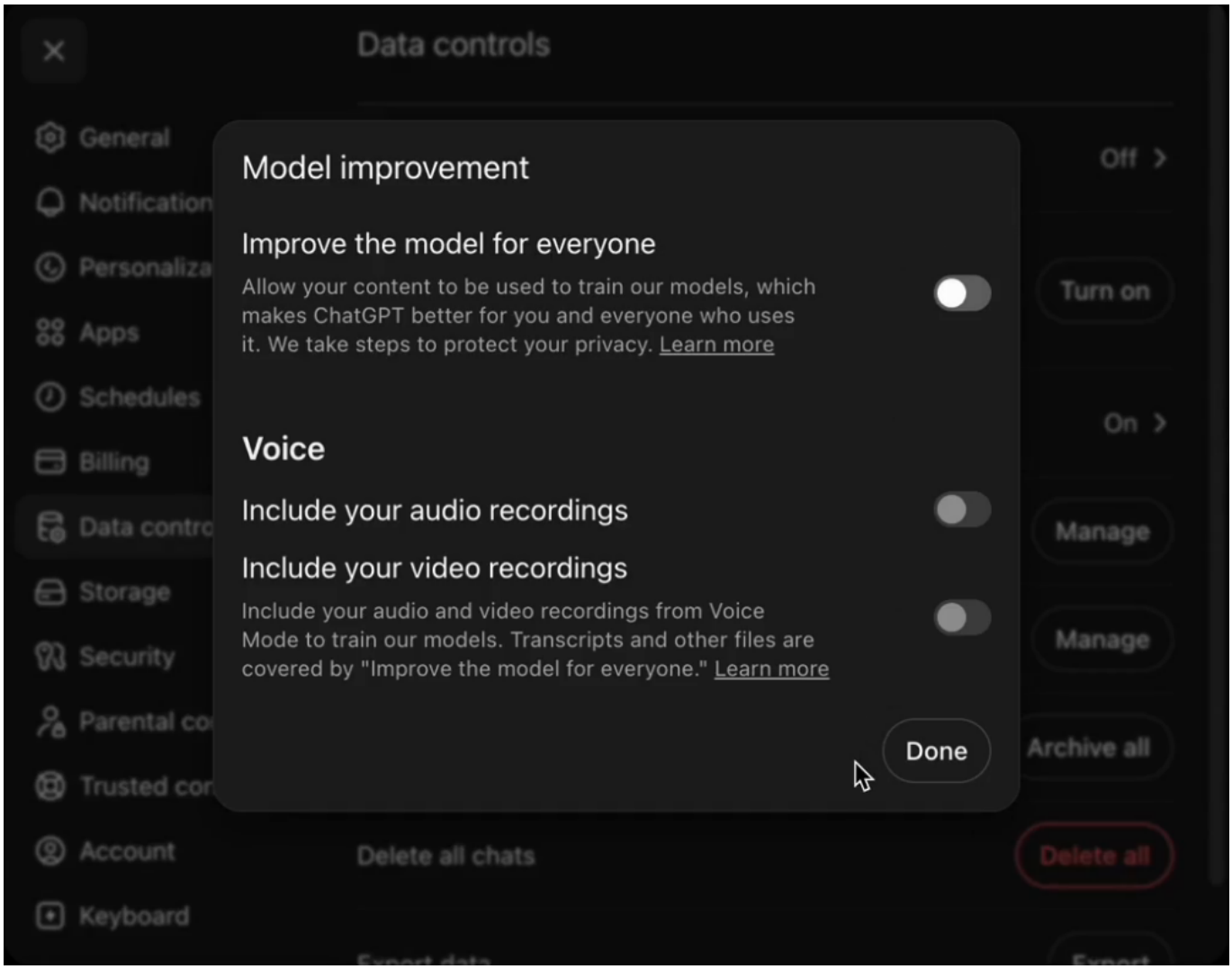
A .gitignore file tells Git which files it should not track. It is not a security boundary for Codex or other AI tools. If we ask Codex to read a file, it can read that file even when Git ignores it.

We should avoid referencing files that contain credentials, API keys, or other secrets in prompts. Sensitive values should be provided through environment variables at runtime instead of being hard-coded in project files. A file's Git status never makes it safe to expose to an AI tool.

Control model improvement settings

Before using Codex with sensitive work, we should review whether our interactions can be used to improve OpenAI's models. ChatGPT Pro or Max users can open Data Controls in ChatGPT and adjust the Model improvement settings. These controls can include options for using content and separate choices for audio or video recordings.

© 2026 Zenva Pty Ltd. All rights reserved.



OpenAI API users can visit platform.openai.com, open their organization settings, and look for Data Controls or Sharing settings. Turning off the relevant model-improvement options helps protect sensitive project information.

Prepare sensitive projects

Privacy settings are only one part of the decision. Before running Codex on a sensitive project, we should confirm our opt-out status and check our organization's rules for using AI tools. A company may impose additional requirements that go beyond the tool's own settings.

When we are uncertain, the safer starting point is to opt out first. We can enable a feature later after confirming that it is appropriate, but we cannot undo an accidental disclosure.

Limit outbound network access

Codex keeps outbound network access off by default. This helps keep operations local and reduces the risk of accidentally sending information to an external service or making unintended changes to a remote system.

We should treat network access as an exception and enable it only for a specific task, such as installing a package or retrieving remote documentation. Even when network access is enabled, interactive approval prompts give us control over when Codex can perform network actions.

© 2026 Zenva Pty Ltd. All rights reserved.

Use the sandbox model

The sandbox controls how Codex interacts with our computer. We can configure how much filesystem access it has and which local commands it can run. By default, Codex uses the workspace-write sandbox mode.

This mode restricts file reads and writes to the current project directory. Keeping Codex inside the workspace protects the rest of the operating system from unintended changes. We should change this default only when a task has a clear need for broader access.

Run a pre-flight checklist

Before beginning sensitive work, we can use a short checklist to confirm that our setup is safe:

1. Confirm that we have opted out of model training where required.
2. Make sure prompts do not reference credentials, API keys, secrets, or other sensitive files.
3. Use environment variables for sensitive values, and remember that .gitignore does not prevent Codex from reading a requested file.
4. Verify that the default workspace-write sandbox mode is active unless there is a specific reason to change it.

These checks combine data privacy, organizational policy, network restrictions, and local sandboxing. Together, they help us use Codex confidently while keeping sensitive information and the wider system protected.

© 2026 Zenva Pty Ltd. All rights reserved.

Intro to Agentic Coding with Codex - Protect Project Data

1. A secrets file is listed in `.gitignore`. What does that guarantee?

a. Git ignores it, but Codex can read it if asked
b. Codex cannot access it
c. OpenAI never receives its contents
d. The sandbox encrypts the file

2. Codex needs internet access to install one package. What is the safest approach?

a. Enable it for that task and review the approval
b. Give Full Access for the whole project
c. Use the bypass flag on your main computer
d. Paste your credentials into the prompt

© 2026 Zenva Pty Ltd. All rights reserved.

Codex sessions let us pause work and return later without losing the conversation and context that helped shape it. By understanding what a session preserves and how the available session commands differ, we can move between tasks efficiently while keeping useful context available.

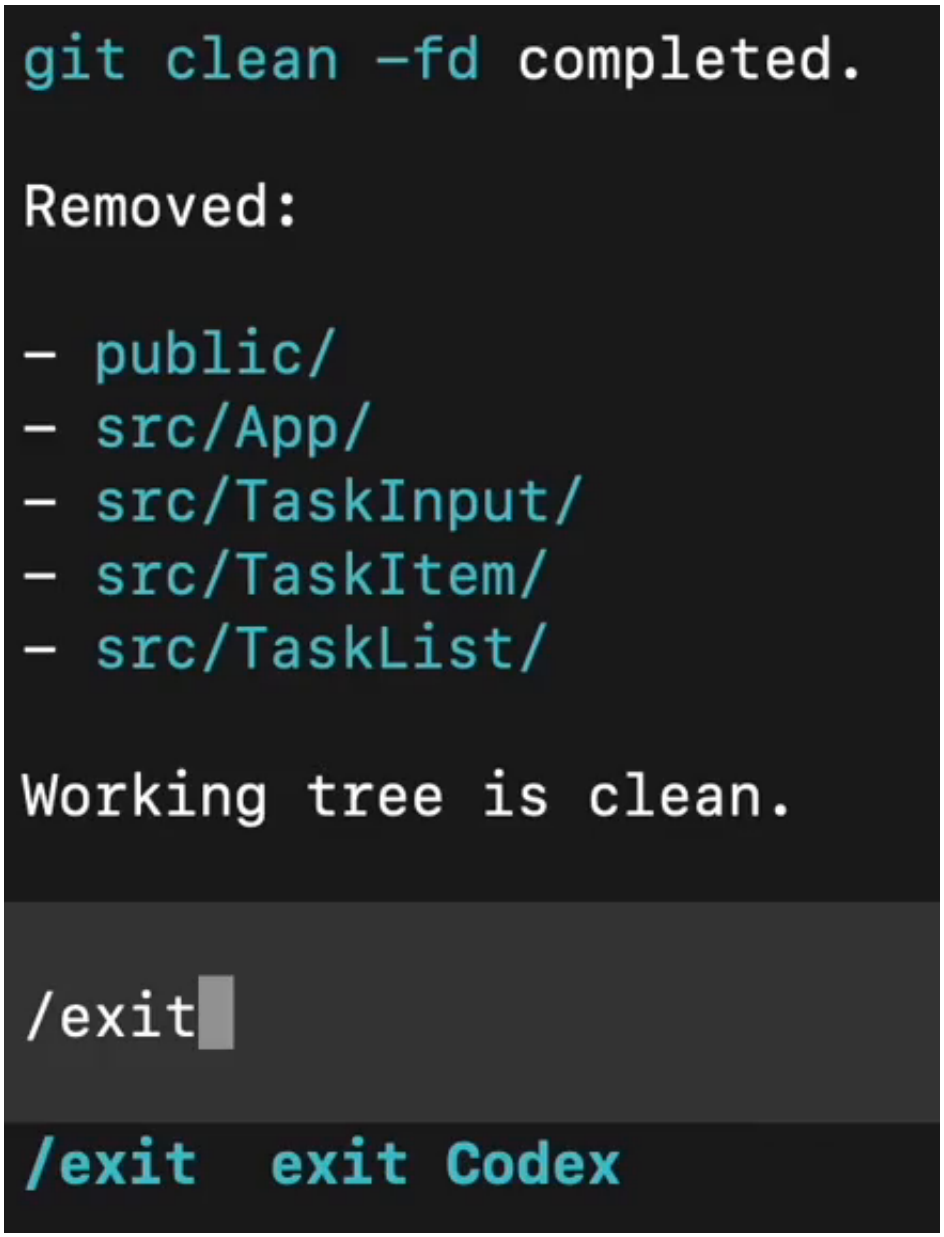
How Codex sessions preserve work

Codex saves three important parts of a session locally:

- **Conversation history:** The instructions we provided and the replies Codex produced remain available.
- **Context:** Codex retains the context it built while understanding the project and reading relevant files.
- **Current work state:** Incomplete tasks and changes waiting for approval can still be present when we return.

This persistence means we can close a session cleanly and later continue from the same point. To finish the current session and return to the terminal, enter /exit. Codex saves the session before closing, so we can resume it later.

© 2026 Zenva Pty Ltd. All rights reserved.



Resuming a saved session

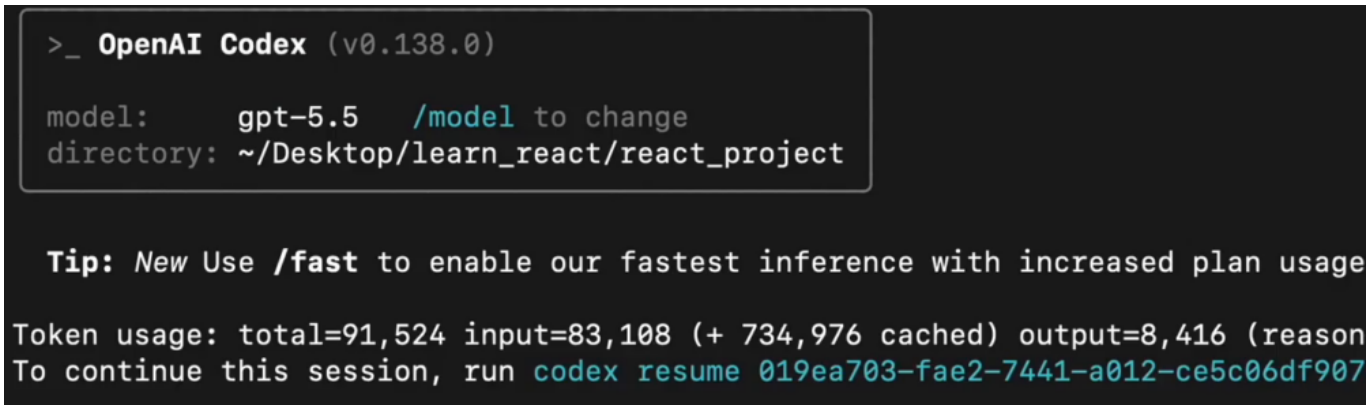
From the project root, run codex resume to open the interactive session picker. The picker lists saved sessions for the project and lets us browse with the arrow keys. After choosing a session, press Enter to restore it.

© 2026 Zenva Pty Ltd. All rights reserved.



Resuming restores the conversation history, the context Codex had built, and information about the files involved. If we already know that we want the most recent session, codex resume --last skips the picker and restores it immediately.

Codex also displays a session ID when a session ends or resets. We can resume one specific session by passing that identifier to codex resume. This is useful when several saved sessions exist and we want to reopen an exact thread.



Understanding /new and /exit

The /new and /exit commands both preserve the existing session, but they serve different purposes:

- /new starts a fresh conversation while keeping us inside Codex. The previous conversation context is cleared from the active thread, but the old session remains saved.
- /exit saves the current session, closes Codex, and returns us to the terminal.

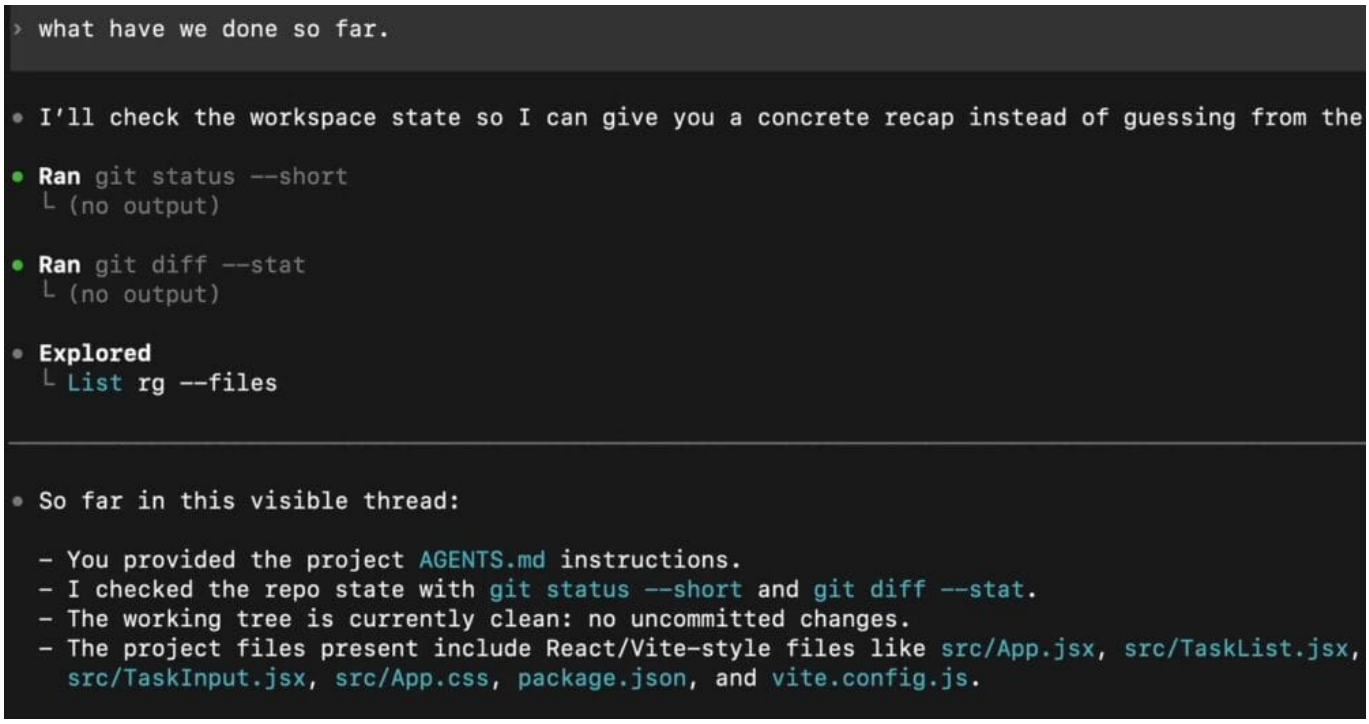
Use /new after finishing one task when the next task is unrelated and no longer needs the current conversation. Use /exit when we are done working in Codex or want to return to the terminal before moving to another project.

© 2026 Zenva Pty Ltd. All rights reserved.

Project context and conversation context

Starting a new conversation does not prevent Codex from understanding the project. In a fresh thread, it can read project guidance such as AGENTS.md, inspect files, and run commands such as git status and git diff to establish the current workspace state.

What a fresh conversation does lose is the prior thread's accumulated conversation history: earlier decisions, files discussed, instructions, and reasoning. A resumed session retains that history, while a new one must rebuild the relevant understanding from the repository and the new prompt.



Choosing whether to resume or start fresh

Before opening Codex for a task, ask whether the previous session's context is still relevant:

- If the task continues earlier work or depends on decisions already made, resume with codex resume, codex resume --last, or a specific session ID.
- If the task follows a completely different direction, run codex to begin a new session, or use /new when Codex is already open.

A resume-first habit helps us preserve useful decisions and avoid rebuilding context unnecessarily. When the old context remains valuable, resuming saves time and keeps the workflow moving smoothly. When it no longer applies, starting fresh gives the new task a clean conversation.

© 2026 Zenva Pty Ltd. All rights reserved.

Codex works best when we manage its context deliberately. In this lesson, we will explore what the context window is, what fills it, how to monitor it, and how to keep each session focused. These habits improve accuracy, make sessions faster, and help control costs.

Understanding the context window

The context window is Codex's fixed-size workspace for the current session. We can picture it as a physical desk with limited space. Everything Codex needs for the task goes onto this desk, and every item competes for the model's attention.

A tidy context helps Codex focus on the current goal. If the workspace becomes crowded with unnecessary or unrelated information, accuracy can decline and costs can increase. When the context grows too large, older details may be summarized or lost to make room.

What consumes context

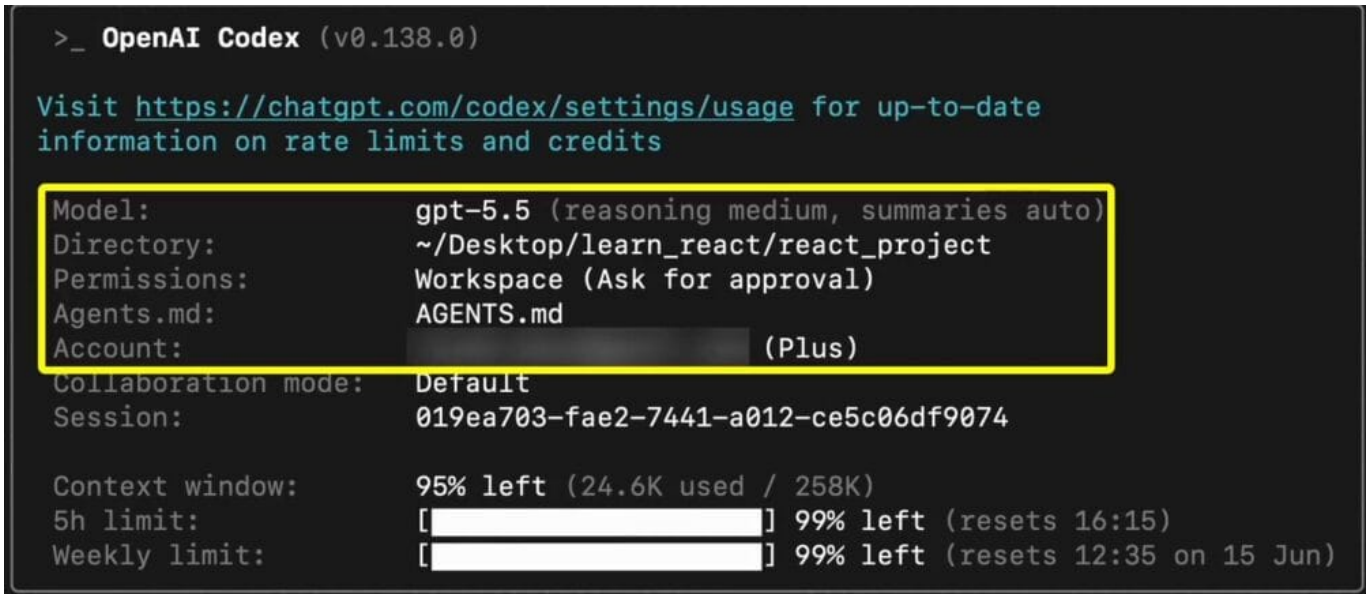
Context includes much more than the messages visible in the conversation. The main sources are:

- **Prompts and responses:** Everything we type and every answer Codex generates becomes part of the conversation history.
- **AGENTS.md:** Project rules and guidelines from this file load automatically when a session begins.
- **Files:** When Codex reads a file to understand or change the project, that file's contents enter the context.
- **Command output:** Build logs, test results, lint reports, and other terminal output also use context space.

Reading many files or producing long command outputs can therefore fill the context even when the conversation itself seems short.

Checking session status

We can inspect an active Codex session by entering /status. This command displays a snapshot that includes the selected model, working directory, permission settings, loaded AGENTS.md file, account details, and session information.



© 2026 Zenva Pty Ltd. All rights reserved.

For context management, the most important line shows how much of the context window has been used and how much remains. The status panel also shows five-hour and weekly rate-limit usage with reset times. It is useful to check /status after several exchanges, during long sessions, or after Codex has read many files.

Continuing a task with /compact

When the context is filling up but we still want to continue the same task, we can enter /compact. Codex summarizes the earlier conversation, preserving the key information while shortening the history. This creates more room without discarding the goal or forcing us to start over.

Natural pauses are good times to compact, especially during a long task. Afterward, we can run /status again to confirm that the context has been optimized.

Choosing between /compact and /new

The correct command depends on whether we are continuing the current concern or starting a different one:

- Use /compact to continue the same task. It condenses older interactions while retaining the session's main idea and focus.
- Use /new when switching to a completely different task. It provides a fresh context window and removes prior conversation history so unrelated material cannot distract Codex.

A new session is also useful when the current conversation has gone off track and a clean reset would improve the next attempt.

Avoiding kitchen sink sessions

A common mistake is mixing unrelated concerns in one long session, such as asking Codex to add a feature, fix a separate bug, and update a README together. This quickly fills the context with noise and makes it harder for Codex to focus, which can reduce the quality of its answers.

Instead, we should treat each session as one focused work block. Define one task, complete it, and reset before beginning an unrelated item. This principle can be summarized as **one concern, one context**.

Following a focused workflow

We can turn this principle into a repeatable development habit:

1. **Commit the current code.** This saves the existing work and creates a clean baseline that we can return to.
2. **Define one task clearly.** Decide precisely what we want to accomplish before opening a new session.
3. **Open a focused Codex session.** Use /new or start a fresh Codex command if the task is unrelated to the previous one.
4. **Work on that single task.** Keep the conversation and project exploration centered on the defined goal.
5. **Review the proposed changes.** Check that the diff matches the goal before accepting it, and revert changes that are not correct.
6. **Commit the completed work.** Record the verified result in version control to create a clear history.
7. **Reset for the next concern.** Use /new before moving to a completely different task.

© 2026 Zenva Pty Ltd. All rights reserved.

Using automatic compaction as a safety net

Codex can compact automatically when the context becomes too full. This safety net keeps a session working even if we forget to enter /compact. However, automatic compaction is reactive because it starts only after space has become tight.

Manual compaction is proactive and gives us more control. We should aim to compact at natural breaks before context pressure affects the session, while trusting automatic compaction as a fallback.

Key habits for reliable sessions

Effective context management begins with recognizing that prompts, responses, project instructions, files, and command output all share one limited workspace. We can use /status to monitor that workspace, /compact to continue a long task efficiently, and /new to begin unrelated work with a clean state.

By keeping one concern in each context and following the cycle of commit, prompt, review the diff, then accept or revert, we create more consistent Codex sessions. These habits prepare us to turn a vague feature request into a focused, working result in the next lesson.

© 2026 Zenva Pty Ltd. All rights reserved.

Intro to Agentic Coding with Codex - Keep Sessions Focused

1. You are continuing a feature discussed yesterday. Which choice preserves those decisions?

a. Resume the saved session
b. Start fresh and restate what you remember
c. Use `/new` inside the current session
d. Exit and reopen without resuming

2. The context window is filling, but the task is unchanged. What should you do?

a. Use `/compact` and continue
b. Use `/new` and discard the thread
c. Ask Codex to read more files
d. Add the next unrelated task

© 2026 Zenva Pty Ltd. All rights reserved.

In this lesson, we will complete a real feature task with Codex from start to finish. We will begin with a safe restore point, turn a vague idea into a precise prompt, let Codex implement and validate the feature, review every change with Git, and then choose whether to accept or revert the work.

Create a restore point before starting

Every Codex session should begin from a clean commit. This commit records the project's current state before Codex changes anything, giving us a reliable point to return to if the result does not meet our expectations.

We first stage and commit the current project with a command such as git add . && git commit -m "end of module 1 - character counter and limit added". The exact message should describe the state we are saving. With this baseline in place, we can experiment confidently because the original state is preserved.

Turn a vague idea into a specific prompt

A request such as "Add filtering to the Todo app" leaves important decisions unclear. Codex would have to guess which filters are needed, how filtering should behave, what visual feedback to provide, and whether it can install additional packages.

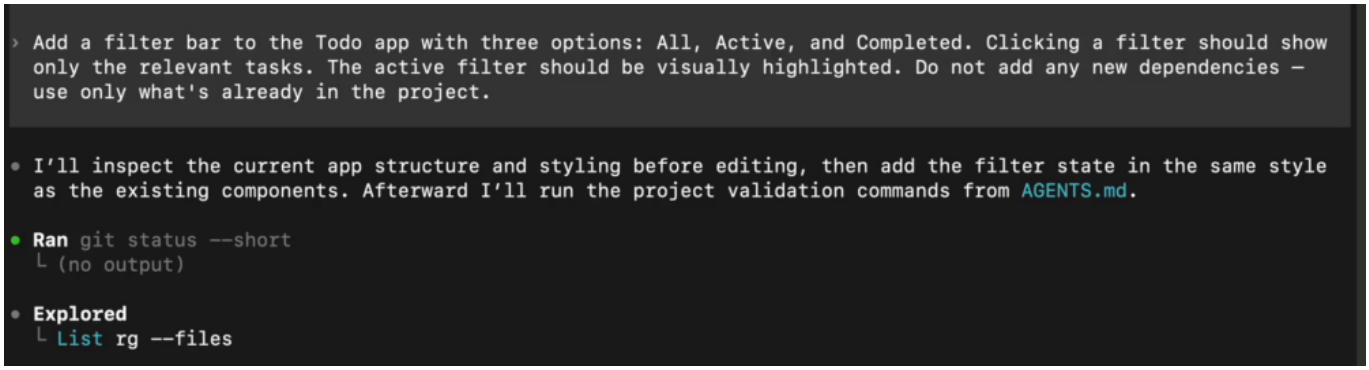
A stronger prompt defines the desired outcome and its constraints. For this task, we ask Codex to add a filter bar with All, Active, and Completed options. Selecting a filter should display only the relevant tasks, the selected option should be visually highlighted, and the implementation should use only the project's existing dependencies.

This level of detail gives Codex a clear target while keeping us in control of the scope. It also reduces the chance of unexpected changes.

Run the task with Codex

After we submit the prompt, Codex starts its MCP server automatically. This gives it the tools needed to read project files, run commands, and write changes throughout the session.

Before editing, Codex inspects the repository. It checks the current Git state, uses rg --files to discover the project structure, reads the relevant React files and project instructions, and then plans how to add the filter bar and connect it to the app.

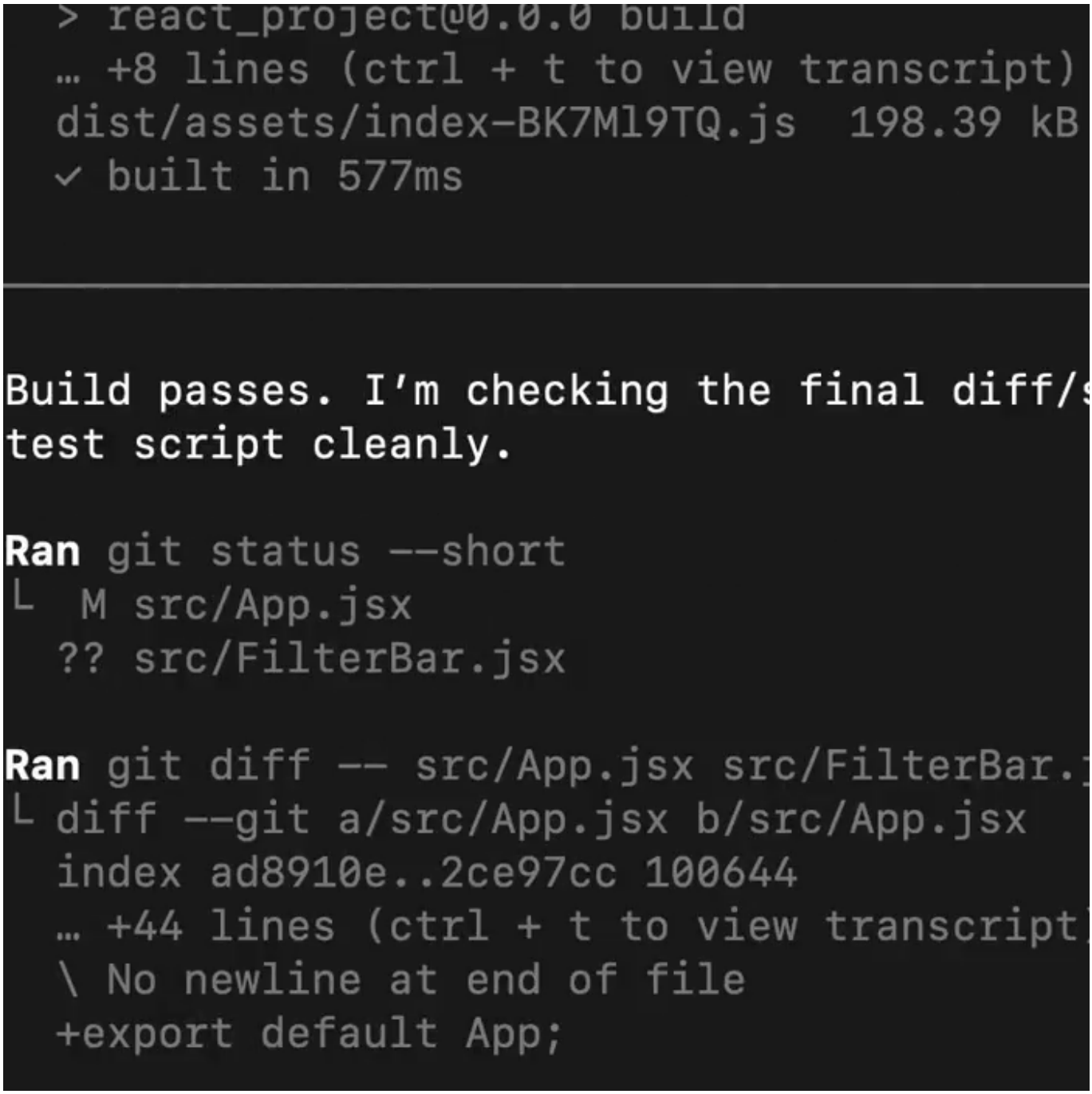


Codex creates the new filter bar component and updates the app so it can store the selected filter and show the corresponding tasks. It then runs the available validation commands, including the project's build process. A successful build confirms that the implementation compiles correctly.

Finally, Codex checks git status and git diff, then summarizes the feature and the validation results.

© 2026 Zenva Pty Ltd. All rights reserved.

We can also run the app ourselves to confirm that the filter bar behaves as requested.



Answer clarifying questions precisely

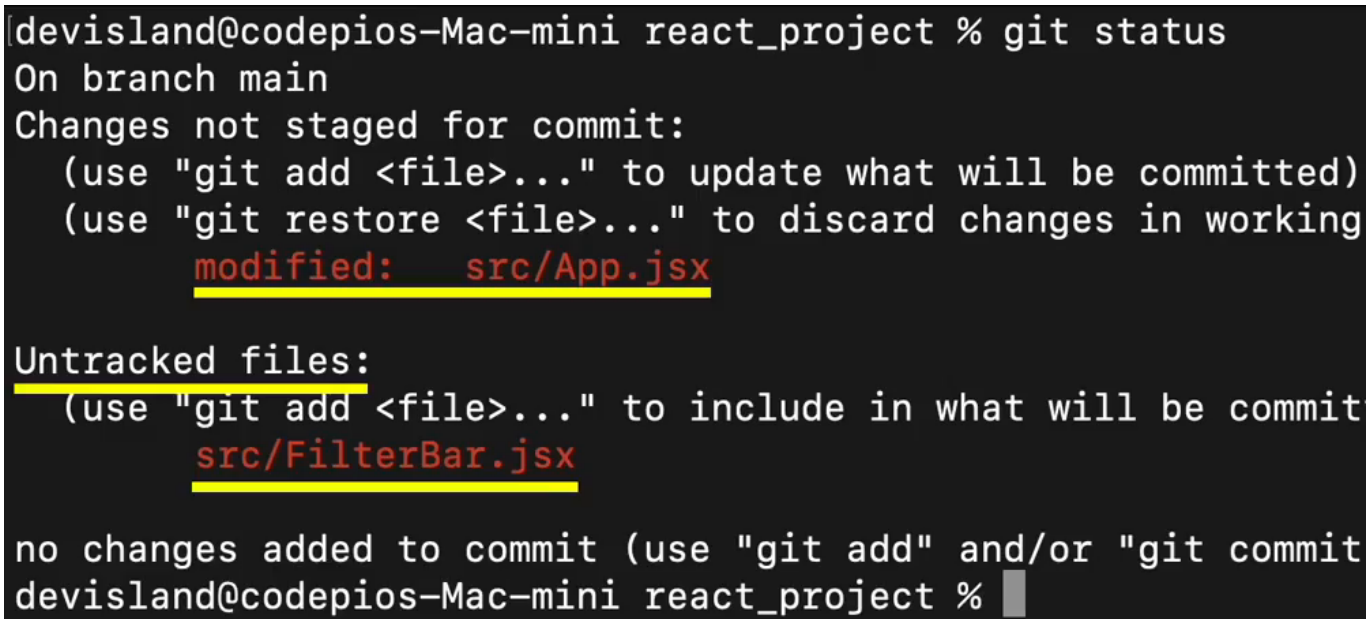
Codex usually begins work immediately when the request is clear, but it may ask a focused question when part of the behavior is still ambiguous. For example, it may ask whether the selected filter should persist after a page reload or whether an in-memory state is sufficient.

A short answer such as “In-memory is fine” resolves the ambiguity without adding unnecessary detail. Codex can then confirm the decision and continue with a clear implementation path. These questions are useful because they expose assumptions before they become unwanted code.

Review every changed and new file

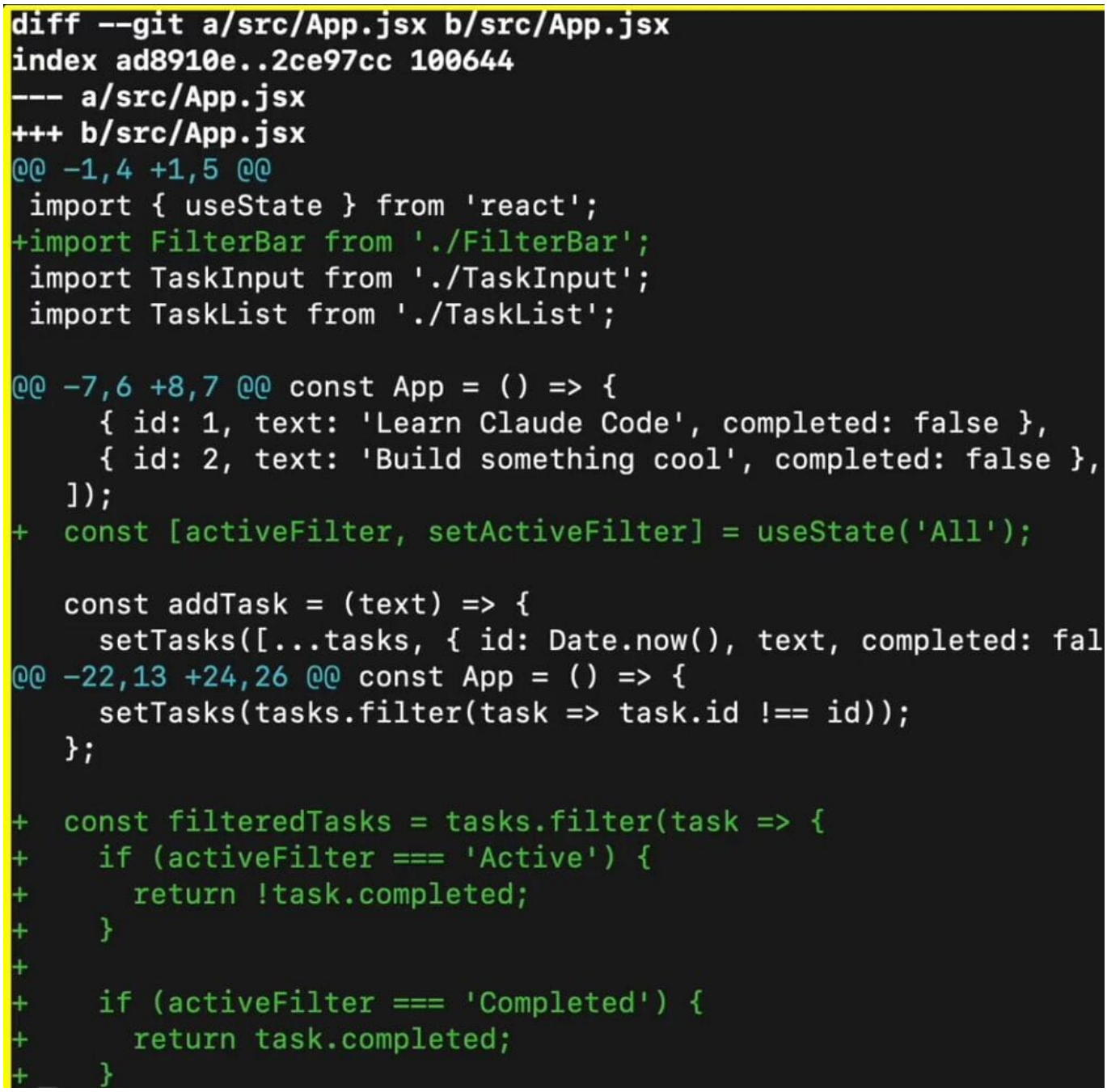
© 2026 Zenva Pty Ltd. All rights reserved.

Once Codex finishes, we begin our own review with git status. This command gives us a complete file-level overview, including modified files and newly created untracked files. In this task, it shows that src/App.jsx was modified and src/FilterBar.jsx was created.



Next, we use git diff to inspect the exact edits to tracked files. As we read the diff, we should ask whether Codex changed the correct files, whether it added anything unexpected, and whether the logic makes sense. Here, the diff shows the new filter bar import, the active filter state, and the logic that selects active or completed tasks.

© 2026 Zenva Pty Ltd. All rights reserved.



```
diff --git a/src/App.jsx b/src/App.jsx
index ad8910e..2ce97cc 100644
--- a/src/App.jsx
+++ b/src/App.jsx
@@ -1,4 +1,5 @@
 import { useState } from 'react';
+import FilterBar from './FilterBar';
 import TaskInput from './TaskInput';
 import TaskList from './TaskList';

@@ -7,6 +8,7 @@ const App = () => {
     { id: 1, text: 'Learn Claude Code', completed: false },
     { id: 2, text: 'Build something cool', completed: false },
   ]);
+  const [activeFilter, setActiveFilter] = useState('All');

   const addTask = (text) => {
     setTasks([...tasks, { id: Date.now(), text, completed: fal
@@ -22,13 +24,26 @@ const App = () => {
     setTasks(tasks.filter(task => task.id !== id));
   };

+  const filteredTasks = tasks.filter(task => {
+    if (activeFilter === 'Active') {
+      return !task.completed;
+    }
+
+    if (activeFilter === 'Completed') {
+      return task.completed;
+    }
```

Because an untracked file does not appear in a standard git diff, we must open and review src/FilterBar.jsx directly. Only after checking both the tracked edits and every new file do we have a complete view of Codex's work.

Accept or revert the result

If the implementation and validation results look correct, we accept the work by staging and committing it. For this feature, we can use git add . && git commit -m "add filter bar - All, Active, Completed". The commit locks in the approved changes and creates the next safe point in the project's history.

If the review reveals a problem, we can return to the clean baseline with git restore . && git clean -fd. This restores tracked files and removes the untracked files created during the task, allowing us to revise the prompt and try again from the saved state.

© 2026 Zenva Pty Ltd. All rights reserved.

Build the workflow into every session

The essential workflow is a repeatable loop:

1. **Commit:** Create a clean restore point before Codex begins.
2. **Prompt:** Describe a specific outcome and include the important constraints.
3. **Review:** Use git status for the full file list, git diff for tracked edits, and direct inspection for new files.
4. **Decide:** Commit work that passes review, or revert everything and try again.

By practicing this commit, prompt, review, accept-or-revert cycle, we can use Codex efficiently without giving up control of the project. Clear prompts guide the work, validation checks confirm the project still builds, and Git gives us the confidence to inspect, approve, or undo every change.

© 2026 Zenva Pty Ltd. All rights reserved.

Intro to Agentic Coding with Codex - Review a Real Codex Task

1. Before Codex implements a feature, what gives you the safest restore point?

a. A clean Git commit
b. A screenshot of the current app
c. A longer prompt with no checkpoint
d. An uncommitted copy of one file

2. `git diff` looks correct, but `git status` shows a new untracked file. What next?

a. Open and review the new file directly
b. Commit because the tracked diff passed
c. Ignore it because Git did not track it
d. Run the same prompt again

© 2026 Zenva Pty Ltd. All rights reserved.

An AGENTS.md file gives Codex lasting project context, helping every session begin with the same important rules, constraints, and technical knowledge. In this lesson, we will examine what belongs in this file, how multiple AGENTS.md files work together, and how Codex uses these instructions during a real project task.

What we will learn

Our main goals are to understand the structure of an AGENTS.md file, learn the three levels at which these files can apply, and see how to generate a useful first draft with the /init command. Most importantly, we will build the habit of updating the file whenever a repeated mistake reveals a missing rule.

Codex does not rely on information from a previous session. It works from the explicit details available in the current project and conversation. An AGENTS.md file therefore acts as project memory: each new session can begin with the context Codex needs to make relevant, consistent decisions.

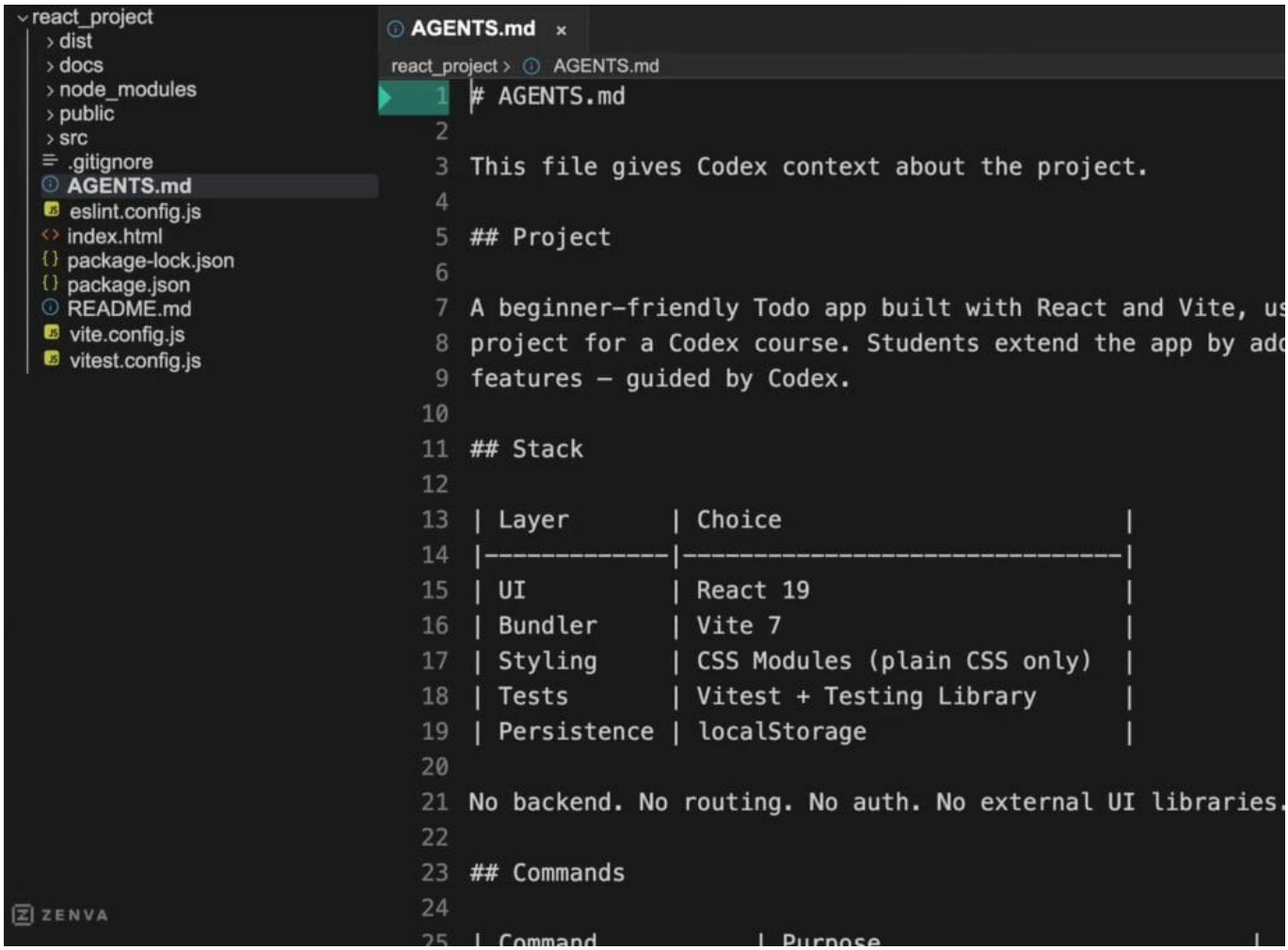
The main parts of AGENTS.md

A strong AGENTS.md file captures the information that should guide work across the project. We should include four main types of context:

- **Project stack:** List the main technologies and tools, along with the commands used to start the application, run tests, and complete other common tasks.
- **Conventions:** Record coding style, naming rules, component organization, and preferred libraries so changes remain consistent with the existing codebase.
- **Critical rules:** State absolute restrictions, outdated approaches to avoid, and fixes for mistakes that have happened repeatedly.
- **Architecture decisions:** Explain important design choices such as data flow, authentication, persistence, and external API usage.

These details help Codex understand both how the project is built and how we expect future work to fit into it.

© 2026 Zenva Pty Ltd. All rights reserved.

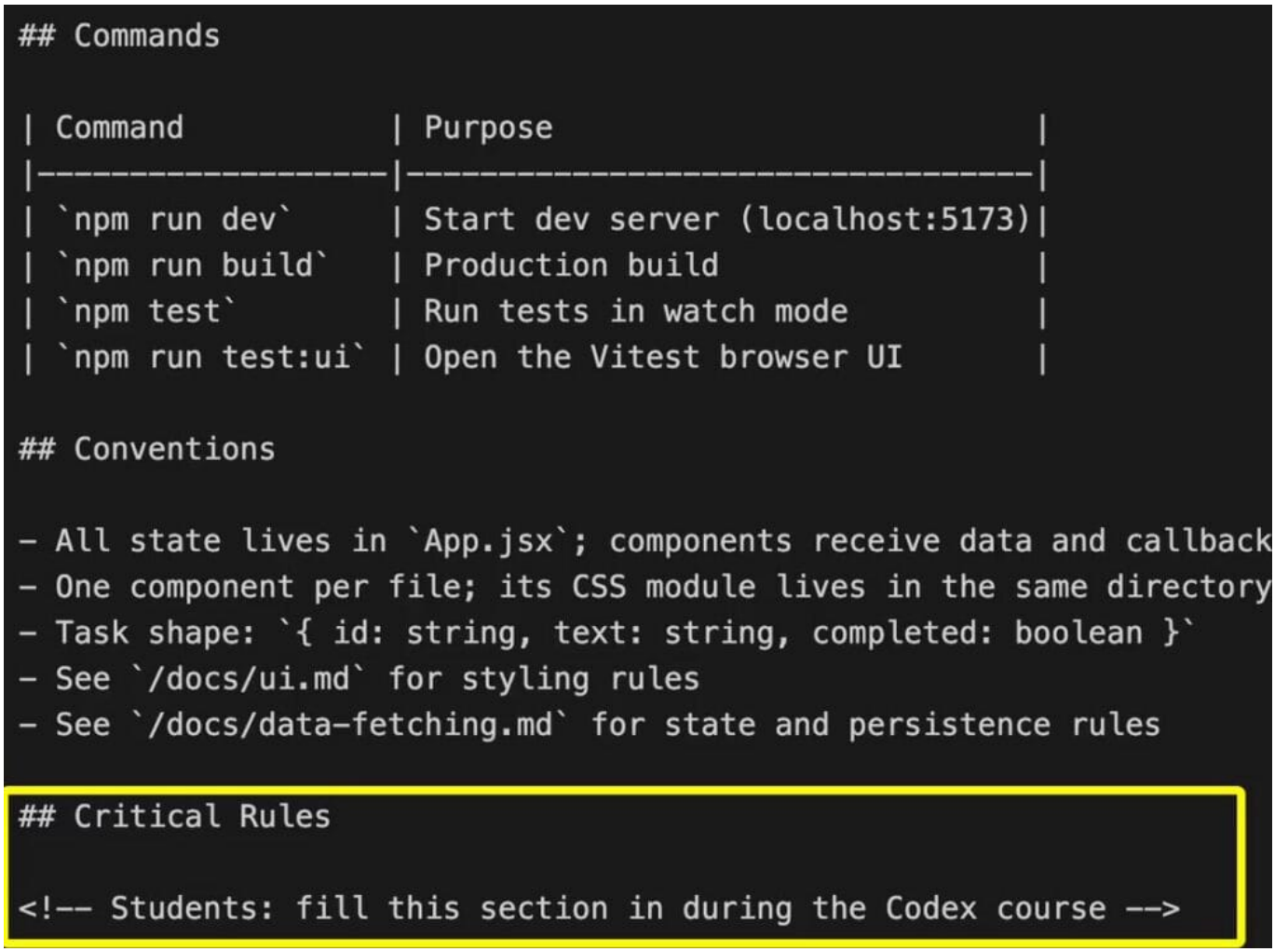


Starting with a generated draft

We can use /init to create an initial AGENTS.md draft. Codex examines the repository to identify the project type, technology stack, common commands, and existing conventions. This provides a practical starting point that we can refine instead of writing every section from scratch.

In the example React project, the generated file identifies the application as a React and Vite project, finds the commands for running the app and tests, and records the main conventions. The Critical Rules section is intentionally empty at first. We will add knowledge there over time as the project reveals recurring problems that should be prevented in future sessions.

© 2026 Zenva Pty Ltd. All rights reserved.

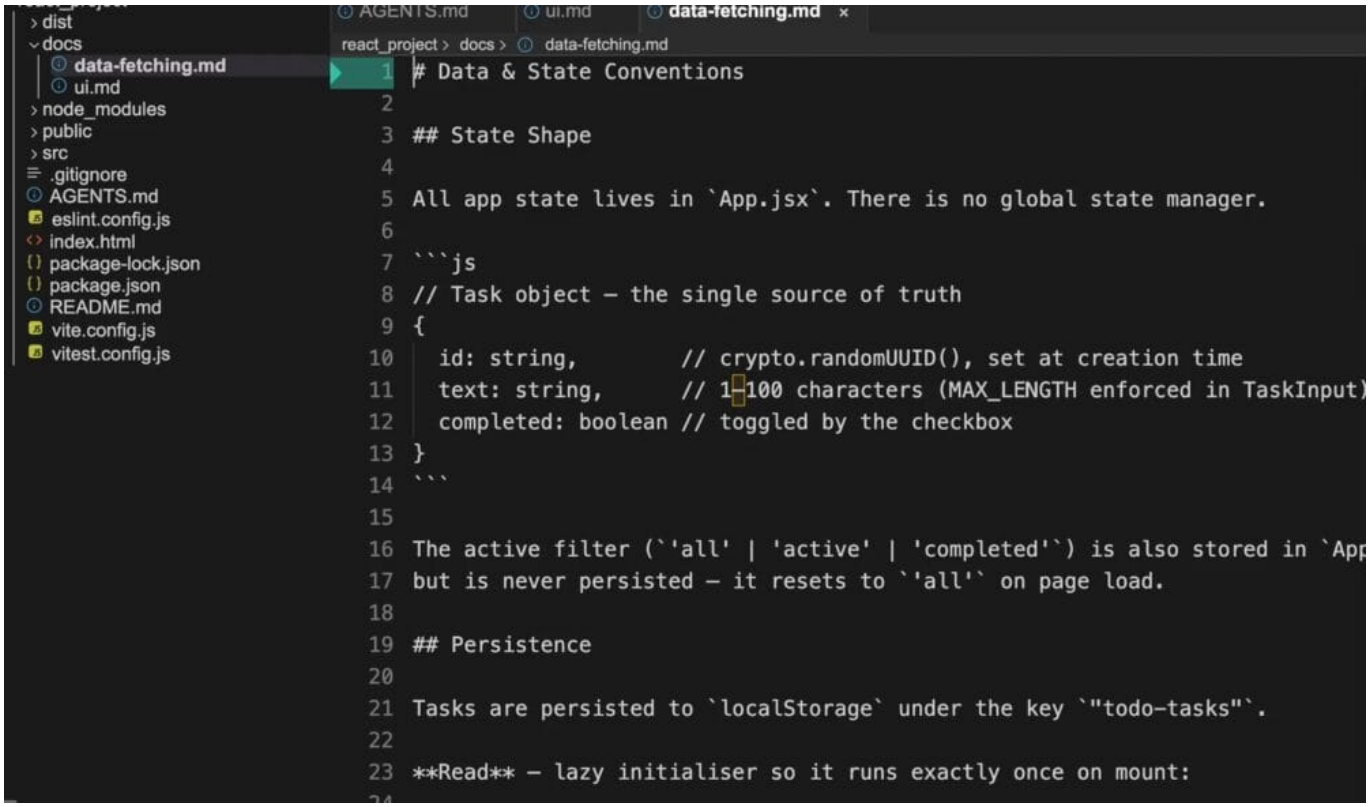


Supporting project documentation

The root instructions can direct Codex to more focused documents. In this project, ui.md defines interface rules such as layout, colors, border styles, transition timing, the component structure, and the process for adding components.

The data-fetching.md document explains how task objects are structured and how application state is managed. Tasks are loaded from local storage with a lazy state initializer, then kept synchronized through an effect. The document also covers the main task actions, their callback-based use, filtering behavior, and work that remains pending, such as API calls or optimistic updates.

© 2026 Zenva Pty Ltd. All rights reserved.



Separating detailed guidance into focused documents keeps the main instructions concise while still giving Codex access to precise information when a task touches that area.

The three-level hierarchy

Codex can combine instructions from three levels, allowing rules to range from broad personal preferences to requirements for one part of a repository:

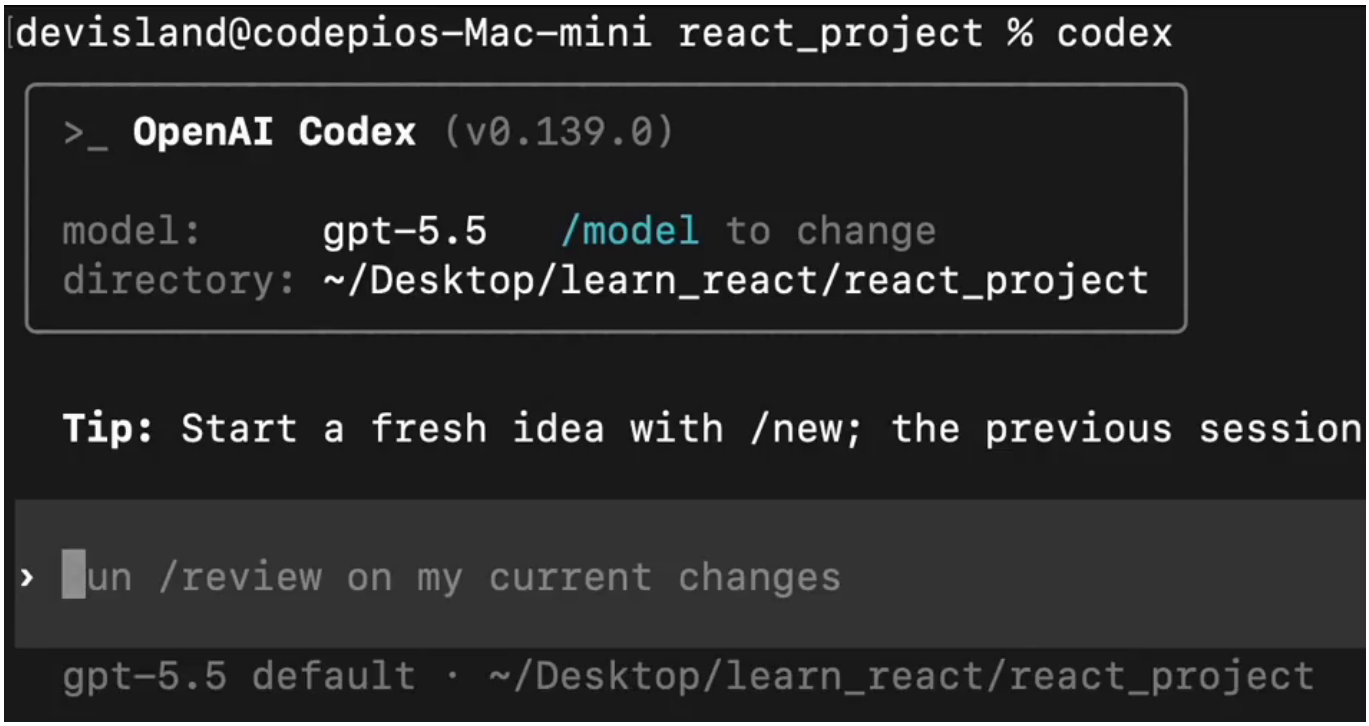
1. **Global instructions:** A personal AGENTS.md file in the home configuration folder stores preferences and general coding rules that should apply to every project.
2. **Project-root instructions:** The AGENTS.md file at the repository root describes the current codebase, including its stack, conventions, architecture, and project-wide critical rules.
3. **Subfolder instructions:** Additional AGENTS.md files inside specific directories provide local rules for those areas, such as separate frontend and backend conventions in a monorepo.

The most specific applicable instructions take priority. This layering gives us precise control without forcing every rule to apply across the entire project.

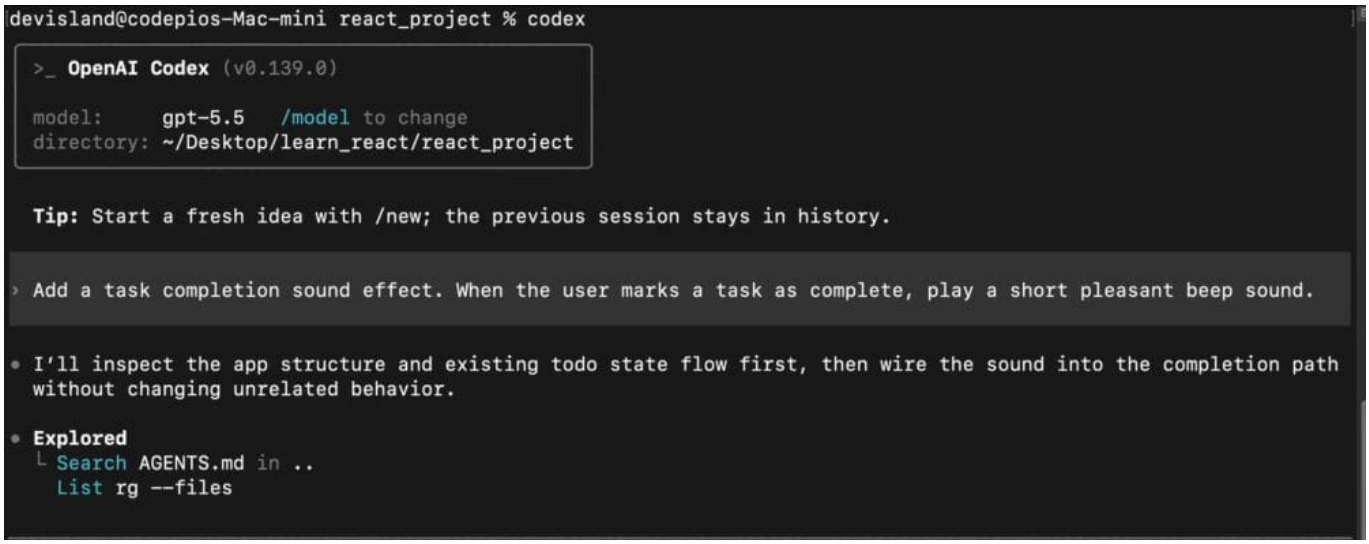
Applying the instructions to a real task

Before asking Codex to change the project, we initialize Git and create an initial commit. We then launch Codex from the project directory so it can inspect the repository and its instructions.

© 2026 Zenva Pty Ltd. All rights reserved.



Our request is to add a short, pleasant beep when a user marks a task as complete. Codex begins by reading the relevant files and tracing the task state flow. It recognizes the Vite application, the centralized to-do state, and the project documentation before deciding how to integrate the feature.



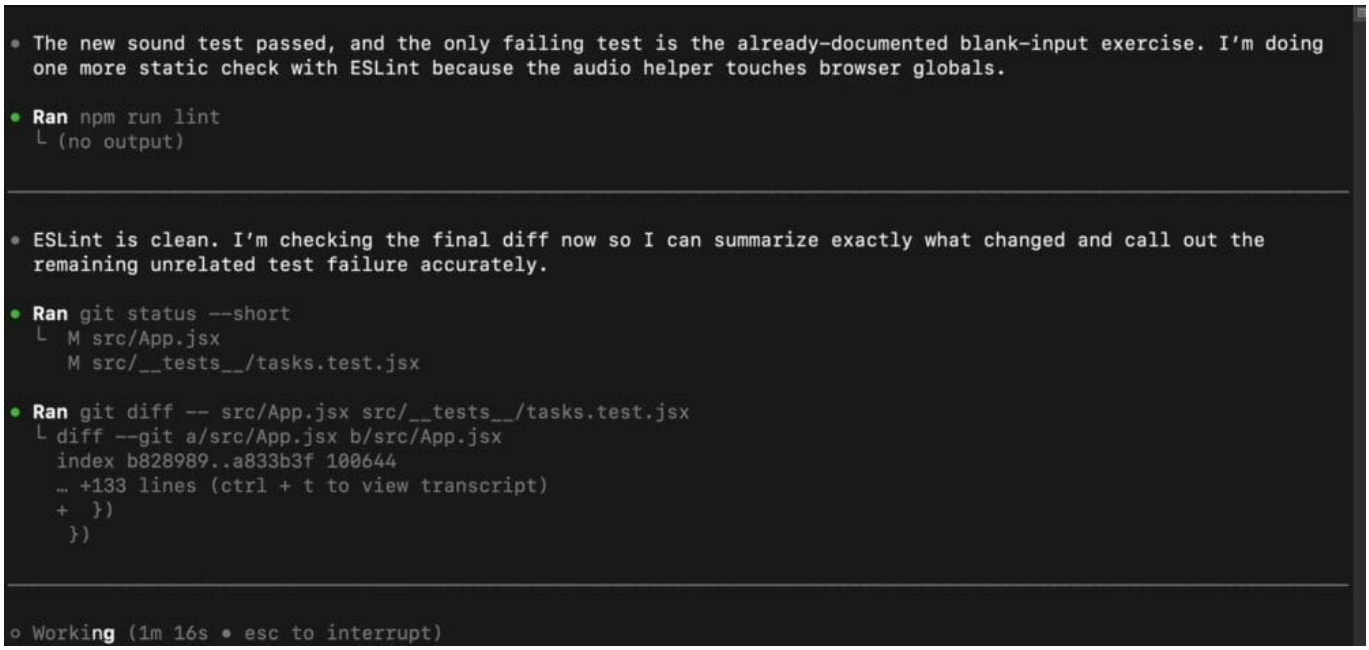
Codex plans to use the Web Audio API in App.jsx. It creates a helper for playing the completion sound and connects it to the task-toggle behavior so the sound runs only when a task changes from active to complete. The implementation also handles audio playback errors, which helps it behave safely in unsupported browsers or test environments.

Testing and reviewing the result

Codex adds a focused test for the sound behavior. The test confirms that audio starts and stops when a task becomes complete, but does not play again when the task returns to its active state. While checking the project, Codex also notices an existing failure related to blank task input. Because that failure is unrelated to the requested feature, it leaves the issue unchanged and reports it clearly.

© 2026 Zenva Pty Ltd. All rights reserved.

After the new test passes, Codex runs the full test command, performs a static check with ESLint, and reviews the final Git diff. This verification catches syntax or behavior problems and confirms that browser globals used by the audio helper do not introduce linting issues.



Keeping project memory useful

An AGENTS.md file becomes more valuable as we maintain it. When Codex repeats a mistake, we should identify the missing constraint and add a clear rule. Over time, this turns project experience into reusable guidance, helping future sessions avoid known problems and begin with a stronger understanding of the codebase.

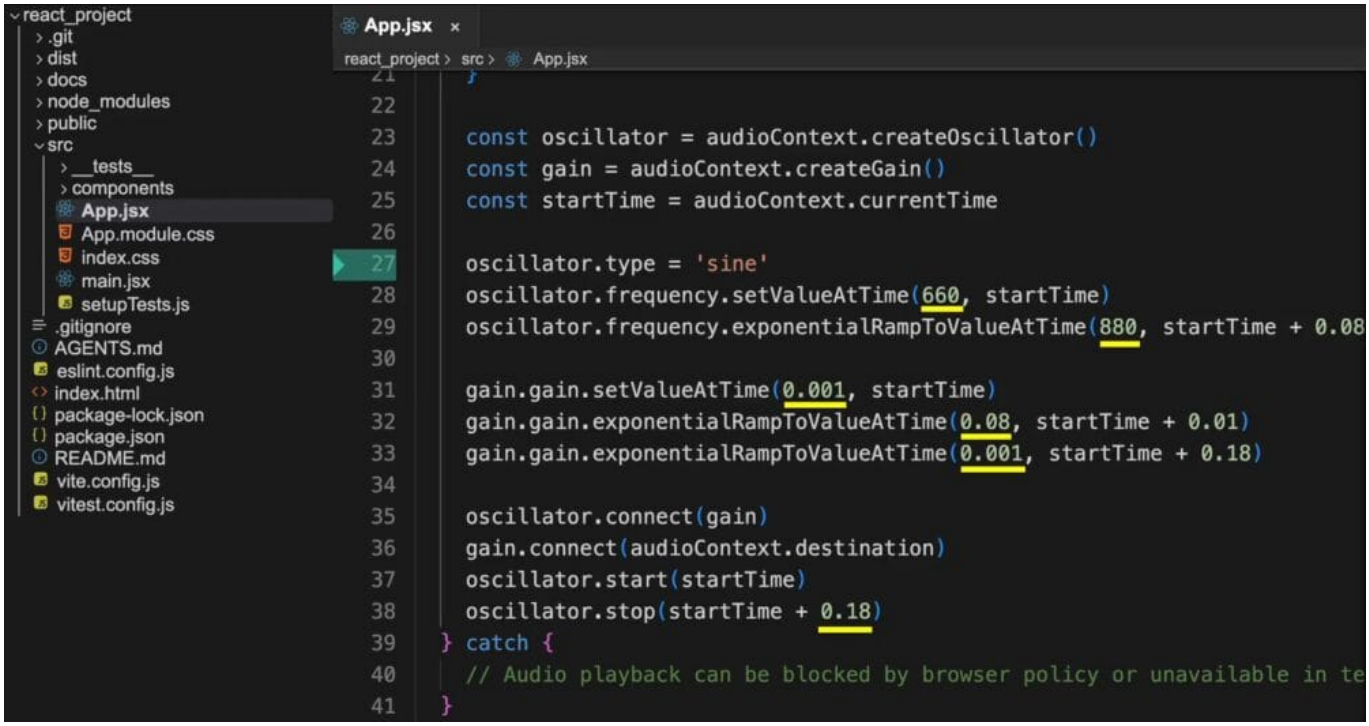
© 2026 Zenva Pty Ltd. All rights reserved.

In this lesson, we will continue improving our Todo project while seeing how AGENTS.md can capture a useful lesson for every future Codex session. We will refactor a completed sound feature, verify the result, turn the underlying issue into a lasting project rule, and commit the related changes.

Reviewing the completed sound feature

Codex has successfully added a task-completion sound with the Web Audio API. The implementation includes appropriate checks and a new sound test. Linting and the sound test pass, while the older blank-input exercise still has a known failing test that is unrelated to this work.

When we inspect App.jsx, however, we can see that several audio values are hardcoded inside the component. These values control details such as frequency, volume, and timing. Even though the feature works, keeping configuration values directly inside a component makes the code harder to reuse and maintain.

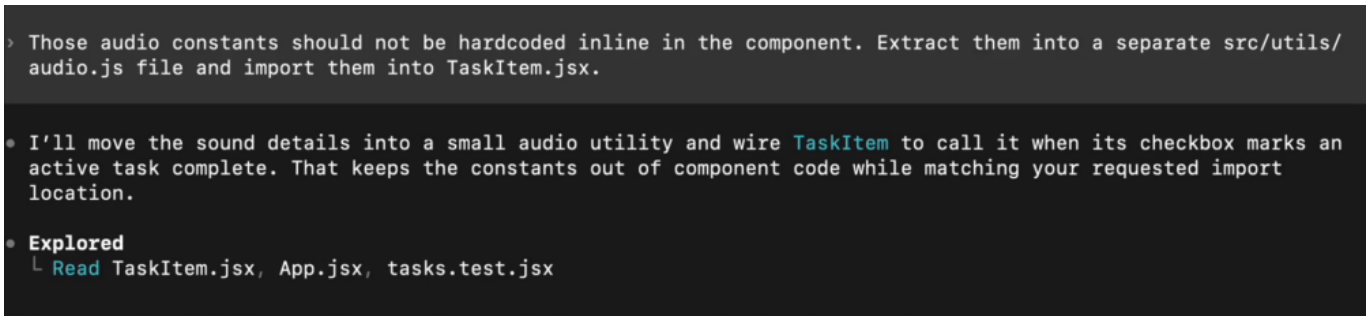


We therefore ask Codex to extract the audio constants and playback logic into a dedicated utility file, then import the helper wherever it is needed. This request addresses the immediate code-quality issue without changing the behavior of the feature.

Refactoring the audio logic

Codex begins by exploring the relevant component, application, and test files. It plans to move the sound implementation into src/utils/audio.js and update TaskItem.jsx so the checkbox interaction triggers the helper when a task is marked as complete.

© 2026 Zenva Pty Ltd. All rights reserved.



After applying the refactor, the responsibilities are clearer:

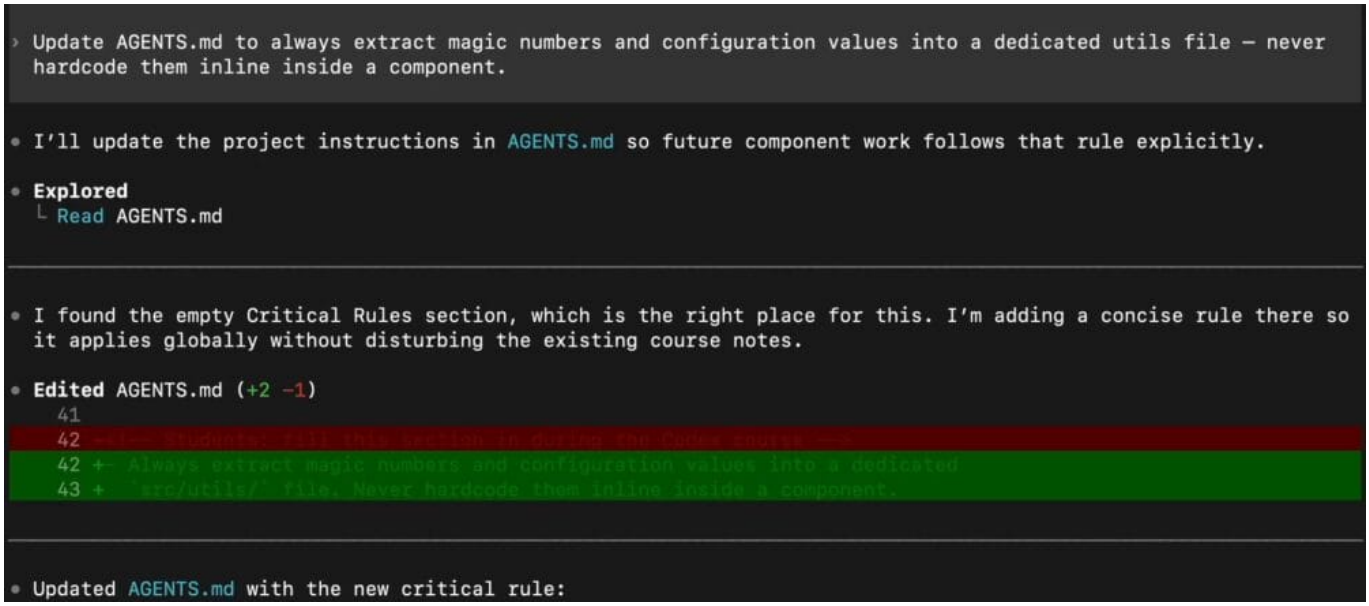
- src/utils/audio.js contains the audio constants and Web Audio playback logic.
- TaskItem.jsx imports and calls the completion-sound helper at the interaction point.
- App.jsx remains focused on task state changes.

Codex immediately runs linting and the relevant tests. The refactored sound test continues to pass, confirming that moving the logic did not break the feature. Codex also clearly reports the pre-existing blank-input test failure instead of presenting it as a new regression. A final diff and status check confirm which files changed and that the refactor is clean.

Turning the fix into a project rule

Fixing the current implementation is useful, but the same mistake could appear again in another component. We can prevent that repetition by recording the lesson in AGENTS.md. We ask Codex to add a critical rule requiring magic numbers and configuration values to be placed in a dedicated src/utils/ file rather than hardcoded inside a component.

Codex reads the existing file, finds the empty Critical Rules section, and replaces its placeholder with the new instruction. This location makes the rule apply across the project while preserving the existing stack, conventions, and architecture notes.



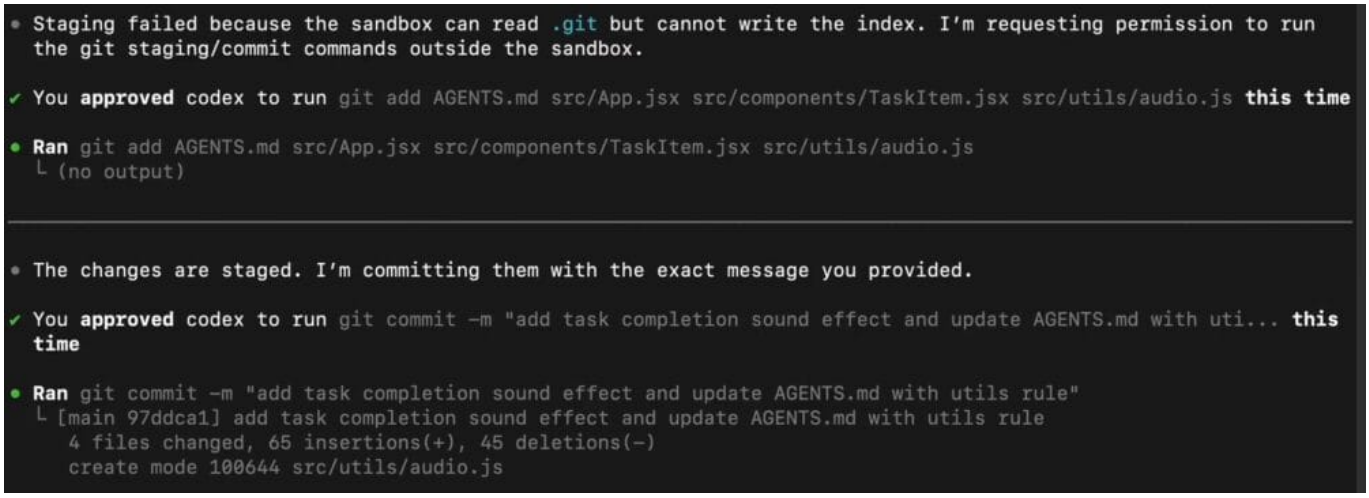
We then run a targeted diff for AGENTS.md. The diff confirms that only the intended placeholder was removed and the new rule was added. This manual review gives us a chance to verify the wording before it becomes part of the project's permanent guidance.

© 2026 Zenva Pty Ltd. All rights reserved.

Committing the completed work

The next step is to record the sound feature, utility refactor, and AGENTS.md rule in one descriptive commit. When Codex first tries to stage the files, the sandbox can read the repository metadata but cannot write to the Git index. Codex does not bypass this restriction. Instead, it explains the problem and asks for permission to run the staging command with the required access.

After we approve the staging command, Codex asks for separate approval before creating the commit. Once approved, the commit succeeds and includes the updated application files, the new audio utility, and the project rule.



A final status check confirms that the working tree is clean. We now have both a working refactor and a lasting instruction that will guide future changes.

Treating AGENTS.md as living infrastructure

The first AGENTS.md draft created by /init is only a starting point. As we encounter repeated mistakes, improve workflows, and discover project-specific practices, we can add the most important lessons to the file. Over time, it becomes a precise record of how the project should be developed.

This cumulative knowledge travels with the codebase. Any developer who uses Codex on the project can benefit from the decisions and corrections made in earlier sessions. In this way, AGENTS.md becomes more than documentation or a static rulebook. It acts as project infrastructure that shapes every interaction with an AI coding assistant.

Reviewing the key ideas

We can now summarize the main ideas covered across our exploration of AGENTS.md:

- **Core content:** record the project stack, coding conventions, critical rules, and architecture decisions that Codex needs to understand.
- **Instruction levels:** distinguish between global, project-root, and local AGENTS.md files, and understand how their order provides increasingly specific guidance.
- **Fast initialization:** use /init to generate a useful first draft from the existing codebase.
- **Continuous improvement:** turn recurring mistakes and important fixes into durable project instructions so future sessions start with better context.

By following this habit, each correction can improve more than the current task. It can make every later Codex session more consistent, efficient, and aligned with the project.

© 2026 Zenva Pty Ltd. All rights reserved.

Intro to Agentic Coding with Codex - Turn Lessons into Project Rules

1. Codex keeps hardcoding configuration values inside components. What makes the correction last?

a. Add a clear critical rule to `AGENTS.md`
b. Explain it once in the current prompt
c. Leave a comment only in one component
d. Rely on the current session history

2. The frontend folder needs rules that differ from the project root. Where should they go?

a. An `AGENTS.md` inside the frontend folder
b. The global home-directory instructions only
c. Every future prompt for the project
d. The latest Git commit message

© 2026 Zenva Pty Ltd. All rights reserved.

A strong prompt gives Codex enough information to understand our project, achieve a specific outcome, respect important limits, and return the result in a useful form. In this lesson, we will build prompts from four essential ingredients, examine common prompting mistakes, use existing files as live examples, add negative constraints, and challenge Codex to improve its first solution.

The four ingredients of a great prompt

Every effective prompt should include four parts:

1. **Context:** Set the scene by describing the current project, relevant files, existing behavior, and any patterns Codex needs to understand. Clear context gives Codex the full picture.
2. **Task:** Define the specific outcome we want. We should describe what needs to exist or work when the task is complete, instead of prescribing every implementation step.
3. **Constraints:** State the rules Codex must follow. These limits can identify files it must not modify, tools it must not use, and project behavior it must preserve.
4. **Output format:** Explain how we want the result delivered. We might request a single function, a complete component, changes to specific files, an explanation, or a plan.

Together, these ingredients reduce guesswork and make the response easier to use.

Turning a weak prompt into a strong prompt

Consider the request, “Add a due date to tasks.” The goal may seem obvious to us, but Codex still has to guess how tasks are stored, where they are created and displayed, which dependencies are allowed, and which files should change. Those guesses can produce code that does not fit the project.

A stronger version would provide the Todo app’s task structure and name the files that create and render tasks. It would define the outcome, such as adding an optional due date field and displaying it only when present. It would also add constraints, such as using a plain HTML date input, avoiding new libraries, and preserving the existing data structure. Finally, it would limit the output to the files that should change.

The goal is the same, but the stronger prompt gives Codex the information required to work precisely.

Avoiding the context assumption

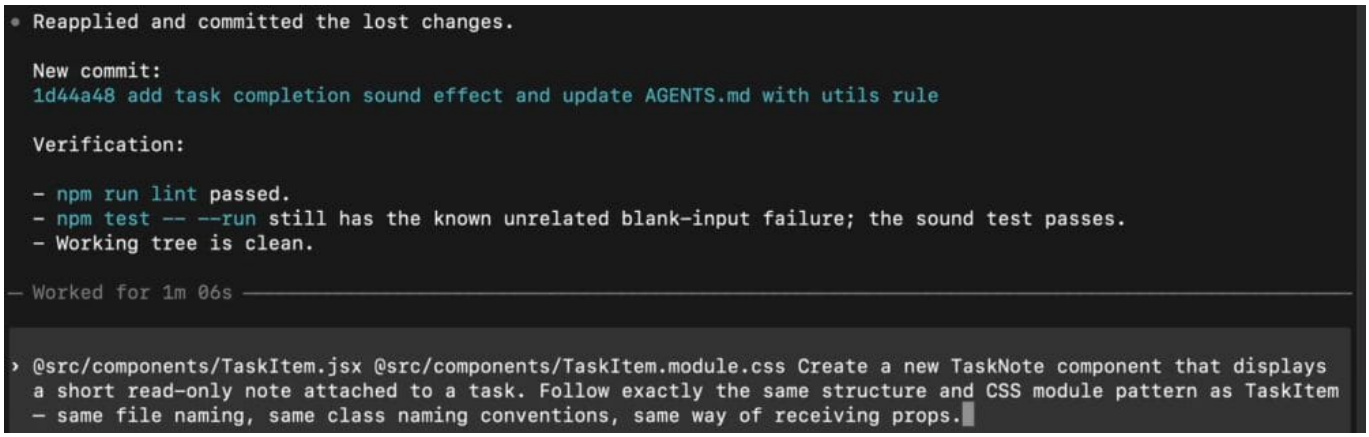
A common mistake is assuming Codex already knows everything about a project. Even a detailed AGENTS.md file cannot always provide the local details needed for work that touches several components, existing state, or established patterns.

Before writing a prompt, we can ask: “What would a new developer need to know to complete this correctly?” The answer identifies the context we should provide. For many tasks, one well-chosen example is more useful than a long set of abstract instructions.

Using existing files as live examples

To create a new TaskNote component, we can link the existing TaskItem.jsx file and its CSS module directly in the prompt. We then ask Codex to follow the same structure, file naming, class naming, prop handling, and CSS module pattern.

© 2026 Zenva Pty Ltd. All rights reserved.



Codex reads the referenced files before making changes. From those examples, it can identify details such as the default export pattern, destructured props, the matching CSS module, and the component's visual conventions. This is a practical form of showing rather than telling: the existing project becomes the specification.

The resulting TaskNote files follow the established JSX and CSS module patterns. The component uses the task's note value and renders nothing when a note is absent.

© 2026 Zenva Pty Ltd. All rights reserved.



Verifying and saving the result

Generation is only one part of the workflow. Codex should also run the project's checks and review the resulting changes. In this example, it runs the lint command to detect style or import problems. No lint output indicates that the check passed without new errors.

Codex then checks the working tree and requests a targeted diff for the new component files. A normal Git diff can show no output for files that are still untracked, so we must not mistake an empty diff for proof that nothing changed. We should inspect new files directly and confirm that their JSX, props, class names, CSS module structure, and empty-state behavior match the request.

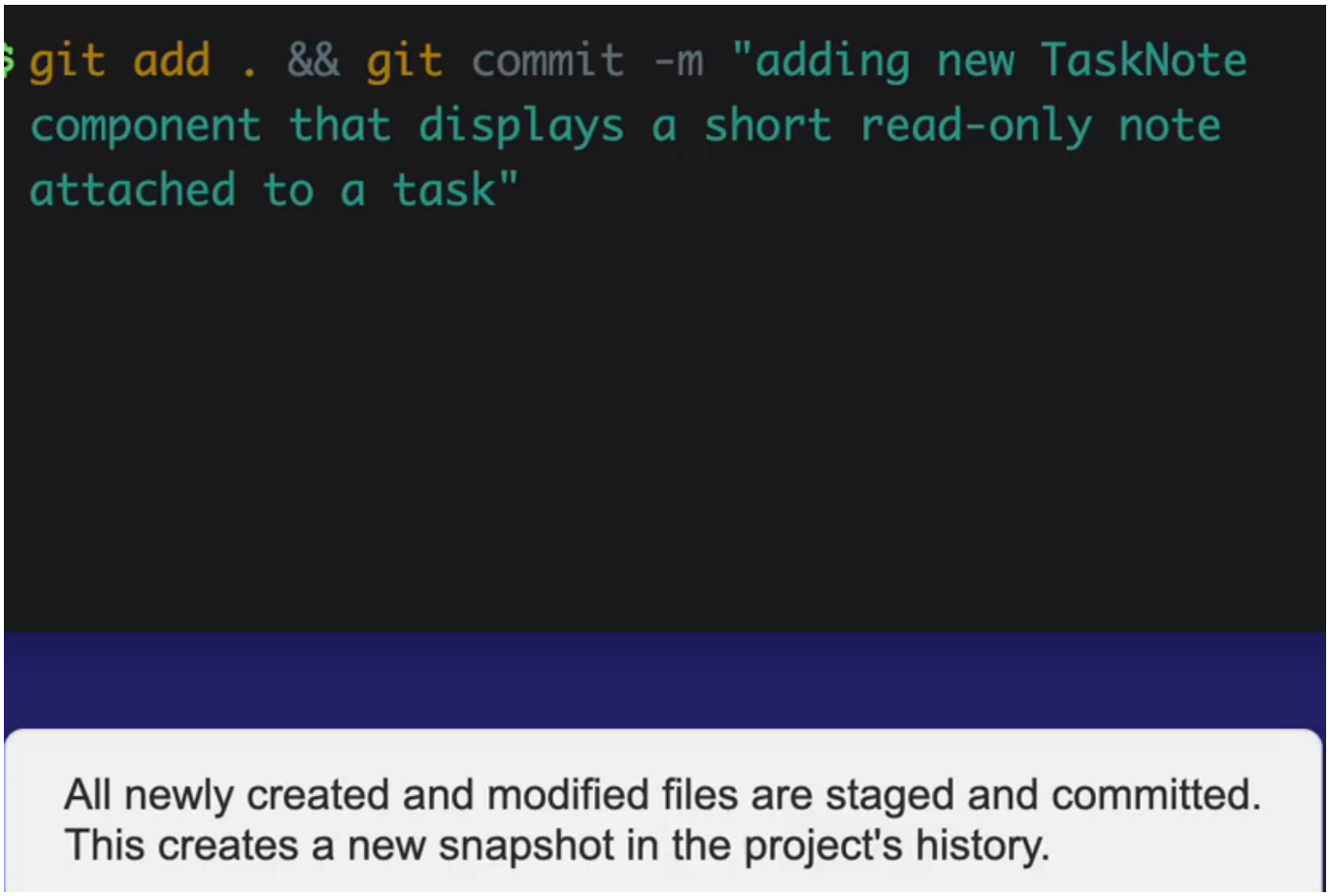
© 2026 Zenva Pty Ltd. All rights reserved.

```
• Lint passed. I'm checking the final diff so I can summarize exactly what was added.

• Ran git status --short
  └ ?? src/components/TaskNote.jsx
    ?? src/components/TaskNote.module.css

• Ran git diff -- src/components/TaskNote.jsx src/components/TaskNote.module.css
  └ (no output)
```

Once the component has been reviewed and verified, we can stage the new and modified files and commit them. The commit creates a clear snapshot of the completed change in the project history.



Adding negative constraints

Positive instructions tell Codex what to do, while negative constraints define what it must not do. Codex knows many common implementation patterns, but the most common approach may not match our project. Without explicit limits, it might add an unwanted dependency, modify a protected file, or change data that must remain compatible.

Useful negative constraints include:

- Do not install external libraries.
- Do not modify App.jsx.
- Do not change the task data structure stored in localStorage.

We should make negative constraints a regular prompting habit. Before sending a prompt, ask:

© 2026 Zenva Pty Ltd. All rights reserved.

“What is the most likely way Codex could solve this that I do not want?” We can then prohibit that approach explicitly. This small step can prevent bugs and unnecessary refactoring.

Challenging Codex’s first solution

Our interaction with Codex does not need to end after its first response. We can challenge the solution in three useful ways:

1. **Verification challenge:** Ask Codex to prove the feature works by tracing what happens when a user uses it. Following the logic from input to result can reveal flaws early.
2. **Quality challenge:** Ask, “With everything you know now, is this the best solution?” Codex may recognize that its first answer was merely the safest or most obvious option and propose cleaner alternatives.
3. **Reset challenge:** If the approach is fundamentally flawed, ask Codex to discard it, explain why it failed, and begin again with a different solution. Restarting is often more effective than continuing to patch a weak foundation.

A practical prompt checklist

Before submitting a prompt, we can use this checklist:

- **Have we provided context?** Include the current situation, relevant files, and existing patterns.
- **Have we defined the task outcome?** Describe what should exist or work when Codex is finished.
- **Have we added constraints?** Identify forbidden tools or approaches, protected files, and rules that must remain unchanged.
- **Have we specified the output format?** Request a plan, selected files, an explanation, or another useful structure.

Strong prompts combine clear context, a result-focused task, explicit constraints, and a precise output format. Existing project files make those instructions more concrete, negative constraints protect the project, and thoughtful challenges help us move beyond the first acceptable answer toward a stronger solution.

© 2026 Zenva Pty Ltd. All rights reserved.

Plan Mode gives us a think-first workflow for complex development tasks. Instead of asking Codex to begin changing files immediately, we can separate the work into research, planning, and implementation. This helps us understand the problem, clarify decisions, and agree on a direction before any code changes occur.

The research, plan, and implement loop

A reliable workflow for substantial tasks has three phases:

1. **Research:** Understand the problem, the existing codebase, and the user requirements. This prevents us from spending time on a solution that does not fit the project.
2. **Plan:** Define what we will build and how we will build it. This is where we can resolve important details, identify affected files, and anticipate challenges.
3. **Implement:** Write and verify the code after the plan is clear. Because the earlier decisions are already settled, implementation can be more focused and efficient.

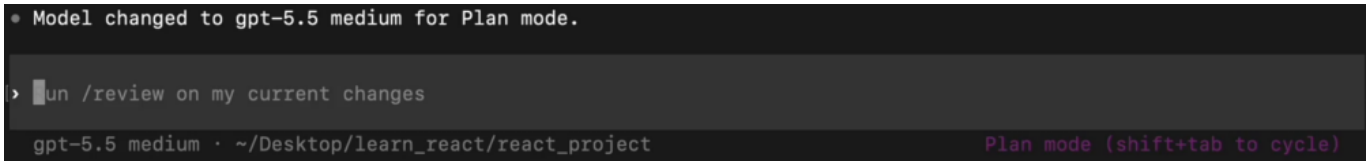
Plan Mode supports the first two phases by allowing Codex to read files, analyze a project, ask questions, and produce a detailed plan without modifying the codebase. Changes begin only after we explicitly approve the plan.

Activating Plan Mode

We can activate Plan Mode in an active Codex session in two ways:

- Enter the /plan slash command.
- Press **Shift+Tab** until the Plan Mode indicator appears.

The keyboard shortcut cycles through Codex's operational modes, so it is a quick option when we switch modes often. Once Plan Mode is active, Codex will not write files, regardless of the task described in the prompt.



We can also request a plan without changing the session's mode. The prompt must clearly state the constraint, for example by asking Codex to provide a detailed implementation plan before writing code and to wait for approval before modifying any files. Codex can then acknowledge the restriction and keep us in control of when implementation begins.

Preparing a clean checkpoint

Before planning a new feature, it is good practice to establish a clean checkpoint. We can review the project's current state, stage any existing changes, and commit them with a message such as pre-plan session checkpoint. This separates earlier work from the feature we are about to plan and makes later changes easier to inspect.

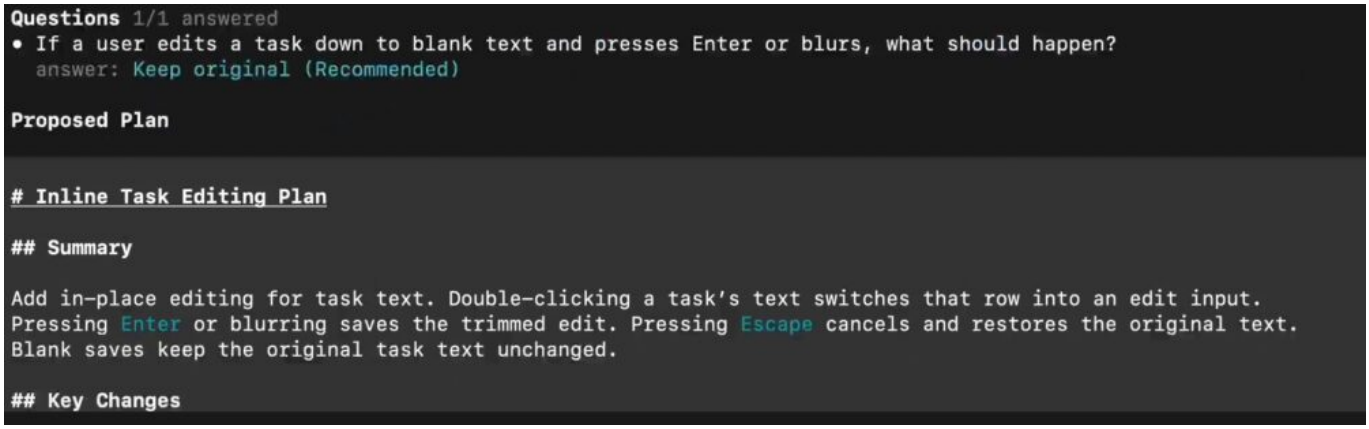
Planning an inline editing feature

To see the workflow in practice, we can plan an inline editing feature for a Todo application. The desired behavior includes double-clicking a task to edit it, using Enter or clicking away to save, and using Escape to cancel. We also ask Codex to identify relevant files, state changes, edge cases, and unknowns before writing any code.

© 2026 Zenva Pty Ltd. All rights reserved.



Codex first inspects the project structure and identifies an unresolved behavior: what should happen if a user saves an empty edit. We can decide that an empty edit should keep the original task text. Codex incorporates that answer into the plan instead of making an assumption during implementation.

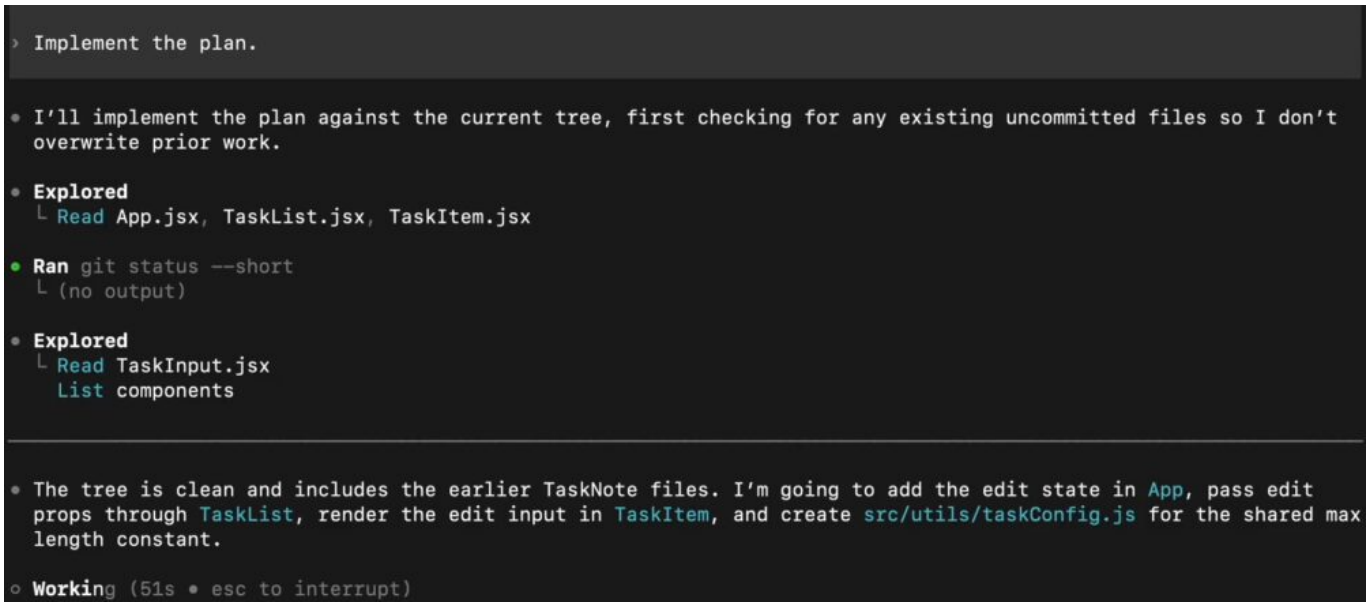


The completed plan covers changes across App.jsx, TaskList.jsx, and TaskItem.jsx. It also describes the UI interactions and the tests needed for double-clicking, saving, canceling, and handling blank edits. After reviewing the plan, we can explicitly approve it by choosing to implement the plan.

Implementing and verifying the approved plan

Once approval is given, Codex switches from planning to implementation. It checks for uncommitted files, reviews the relevant components, and applies the planned changes. In this example, it adds editing state, passes the required properties through the task list, renders the editable input, and introduces a shared task configuration file.

© 2026 Zenva Pty Ltd. All rights reserved.



Codex then adds focused tests for the editing behavior, including saving, canceling, double-clicking, and blank saves. It runs linting and the relevant tests, reports that the new editing tests pass, and identifies a known starter-test failure separately. Finally, it runs a production build and reviews the final changes. This verification demonstrates how a precise plan can guide both implementation and testing.

Using an interview to discover requirements

For a larger feature, we can ask Codex to interview us before it produces a plan. This turns a vague request into a structured requirements discussion and helps reveal decisions that might otherwise be overlooked.

For example, a request to add tags to Todo items leads to several important questions:

- **Are tags free-form or selected from a predefined list?** Choosing free-form tags affects both the interface and the data model.
- **Can a task have one tag or multiple tags?** Supporting multiple tags means each task needs an array rather than a single tag value.
- **How is the complete tag list stored?** For a simpler first version, we can derive the list from the tags already attached to tasks.
- **Do tags need persistent colors?** Omitting colors from the first version keeps the initial scope focused on the essential behavior.

By answering one question at a time, we give Codex a precise understanding of the requirements. It can then summarize our choices and prepare an implementation plan that matches the intended user experience.

Breaking a large feature into slices

Even after the requirements are clear, a feature can still be too large to implement comfortably in one step. We can improve the workflow by dividing it into small, testable slices, with each slice addressing one concern:

1. **Data structure:** Add tag data to task objects stored in local storage. This establishes the foundation without changing the interface.
2. **Input and creation:** Allow tags to be assigned when a new task is created.

© 2026 Zenva Pty Ltd. All rights reserved.

3. **Display tags:** Render the stored tags on each task item.
4. **Filter bar:** Extend the existing filter bar, or add a new component, so tasks can be filtered by tag.

This sequence lets Codex complete and verify one part at a time. Smaller slices are easier to test, review, and correct than a single broad implementation request.

Committing the completed work

After the inline editing feature is built and verified, we should inspect the final changes before committing them. Running git diff lets us review modified files, while the project status reveals any new untracked files. We can then stage the intended changes and create a commit with a concise message, preserving a clean and understandable project history.

Protecting the context window

Plan Mode also helps preserve the context window. Direct implementation consumes context quickly because Codex must repeatedly read files, write changes, and run commands. Planning is mainly a text-based conversation, so we can use Codex’s analytical capabilities with less context overhead.

By front-loading the important thinking, we spend a smaller portion of the context window resolving requirements and architecture. The later implementation phase becomes faster and more focused because Codex already knows what to build and which decisions have been approved.

Key takeaways

- Use a research, plan, and implement loop for complex development work.
- Activate Plan Mode with /plan or Shift+Tab, or state planning constraints clearly in a normal prompt.
- Create a clean checkpoint before planning a new feature.
- Let Codex ask clarification questions instead of allowing hidden assumptions to reach implementation.
- Use an interview to uncover requirements for larger features.
- Divide complex features into small, testable slices.
- Approve the plan explicitly, then verify the implementation with tests, linting, and a production build.
- Use planning to preserve context and keep implementation focused.

© 2026 Zenva Pty Ltd. All rights reserved.

Intro to Agentic Coding with Codex - Prompt and Plan Before Coding

1. You want a new component to match `TaskItem`. What is the most useful context?

a. Link `TaskItem` and its CSS module as examples
b. Describe a generic React component
c. Provide only the new component name
d. Describe only the visual result

2. A feature has unclear edge cases and touches several files. What is the safest next step?

a. Use Plan Mode to research and resolve questions
b. Implement immediately and revise later
c. Increase permissions so planning is unnecessary
d. Combine it with another feature request

© 2026 Zenva Pty Ltd. All rights reserved.

Using the Daily Reader App Starter Project

Before asking Codex to change the Daily Reader app, make sure the practice project is on your computer and running locally.

- **Use the starter project for the lessons.** The complete project is there as a reference if you want to compare your work later.

Step 1: Download the project files

Open the **Course Files** tab for this course and download:

- **Starter Projects – Codex Intro** Use it to follow the course exercises.
- **Complete Projects – Codex Intro** Use it only as a reference of the complete work.

After downloading the starter ZIP, unzip it on your computer.

Step 2: Open the App Reader project

For the first real task, open this folder in Visual Studio Code:

```
? Starter Projects - Codex Intro
  ? Daily Reader App
    ? react_project ? open this folder
```

Make sure you open the react_project folder itself. This is the folder that contains package.json.

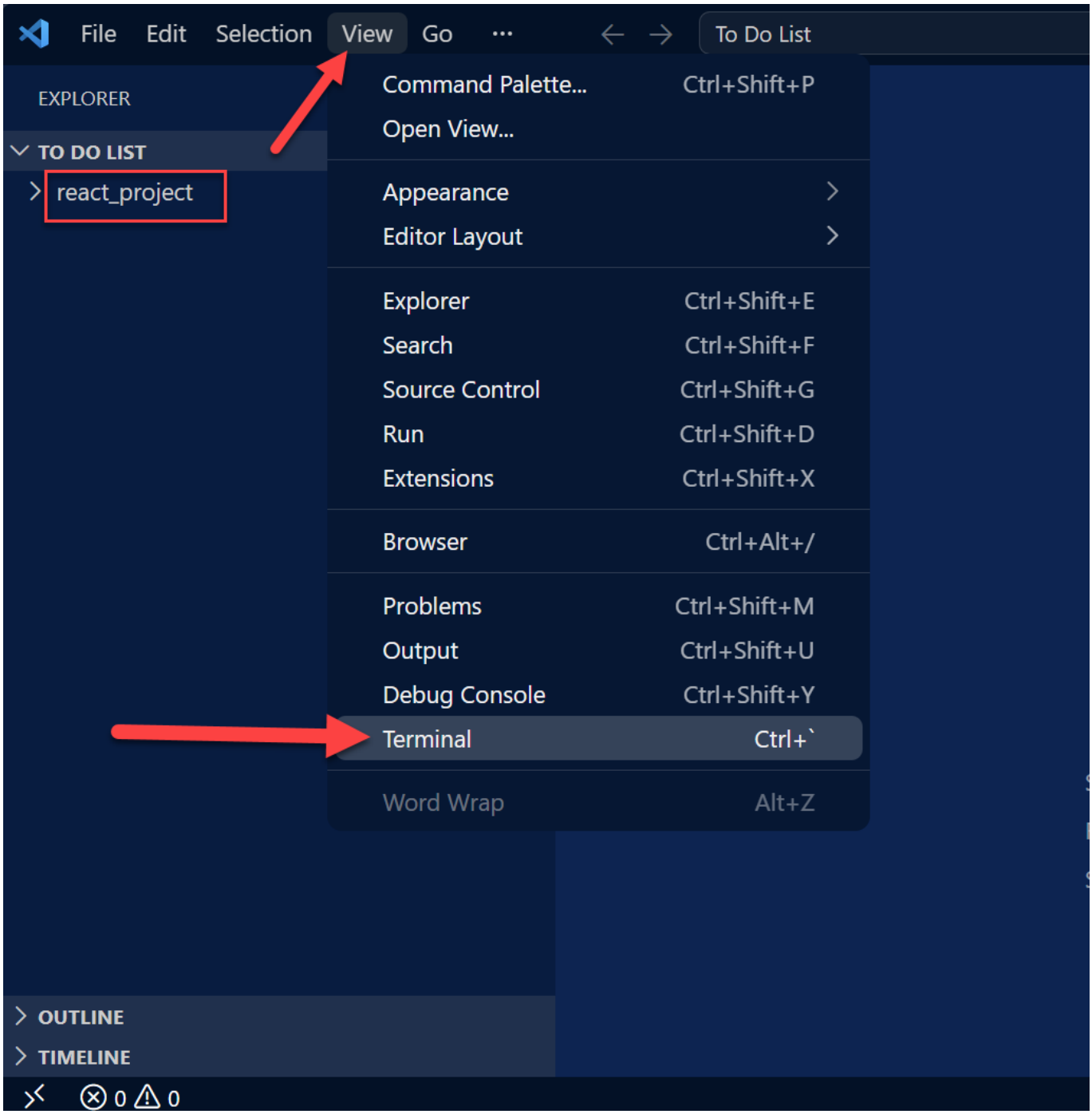
- **What is package.json?** It is a small settings file for the project. It tells Node.js which tools the app needs and which commands you can run, such as npm run dev.

Step 3: Run the app locally

- **Node.js note.** Node.js is optional for installing Codex with the native installer, but it is needed to run this React project locally.

Open the VS Code terminal while you are inside the react_project folder:

© 2026 Zenva Pty Ltd. All rights reserved.



Then, in the **Terminal**, we will run npm like so:

```
npm install
npm run dev
```

- **What is npm?** npm is the tool that downloads the project tools listed in package.json and runs commands for the app. npm is included in the official Node.js installer.

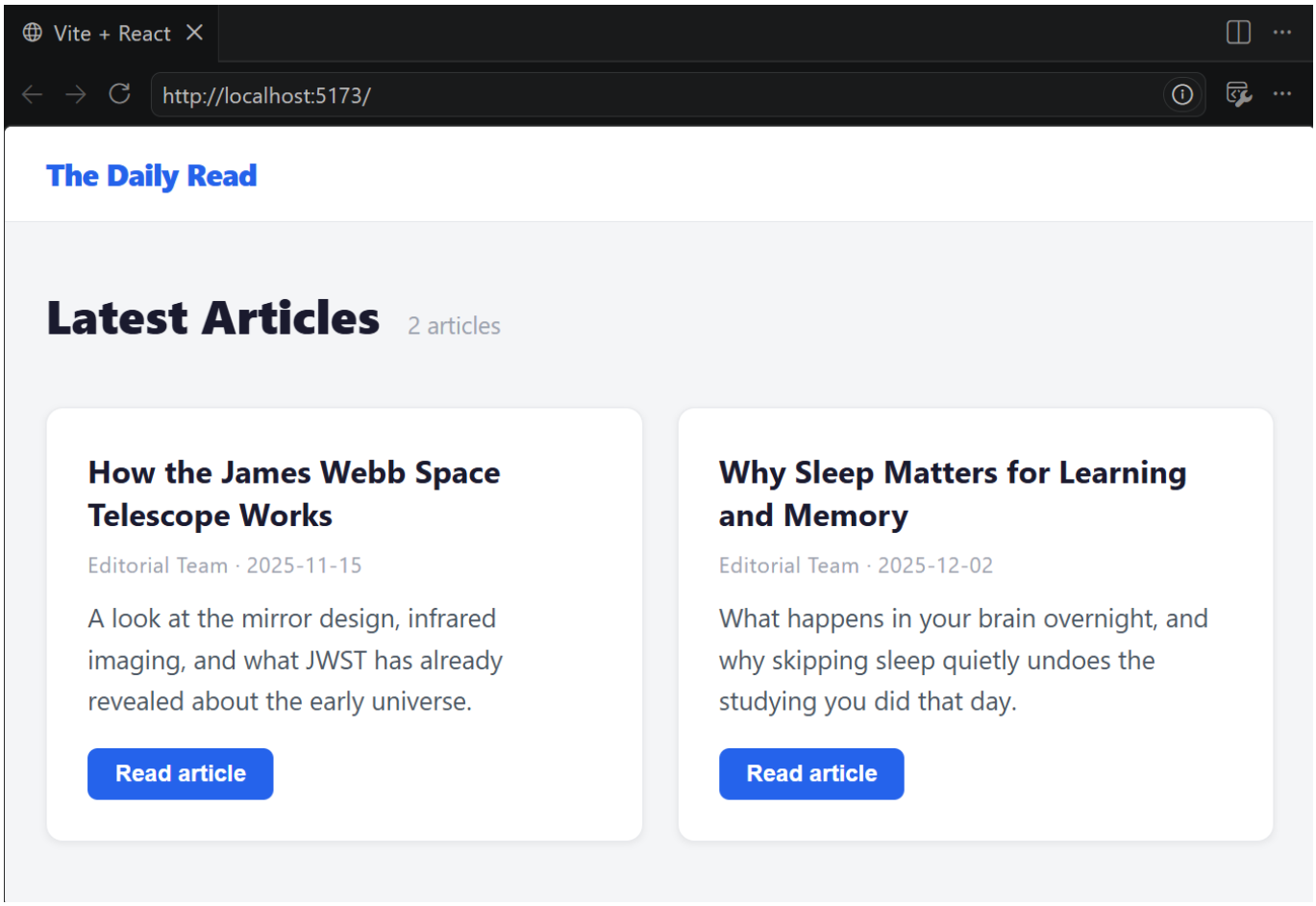
The terminal will show a local URL, usually:

```
http://localhost:5173
```

© 2026 Zenva Pty Ltd. All rights reserved.

Open that URL in your browser. Keep the terminal running while you work on the app.

You should see the App Reader app in your browser.



Step 4: Start Codex in the same folder

In the project terminal, start Codex:

```
codex
```

Codex should start inside the same react_project folder.

Step 5: Save a safe starting point

Before Codex changes the project, create a Git checkpoint:

```
git init
git add .
git commit -m "Initial starter project"
```

After Codex makes changes, you can review them with:

© 2026 Zenva Pty Ltd. All rights reserved.

```
git diff
```

If something does not work

- If npm is not found, install Node.js from https://nodejs.org/download and try again.
- If npm install fails, check that your terminal is inside the react_project folder.
- If the browser does not show the app, use the exact local URL printed in your terminal.
- If Codex cannot find the files, start Codex from the same react_project folder.

Once the app is running and Codex Code is open in the project folder, continue to the next lesson.

© 2026 Zenva Pty Ltd. All rights reserved.

Codex skills turn successful workflows into reusable capabilities. In this lesson, we will define what a skill is, distinguish skills from AGENTS.md rules, create a project-local skill from a completed workflow, and invoke that skill to add another article to a sample application.

What a Codex skill is

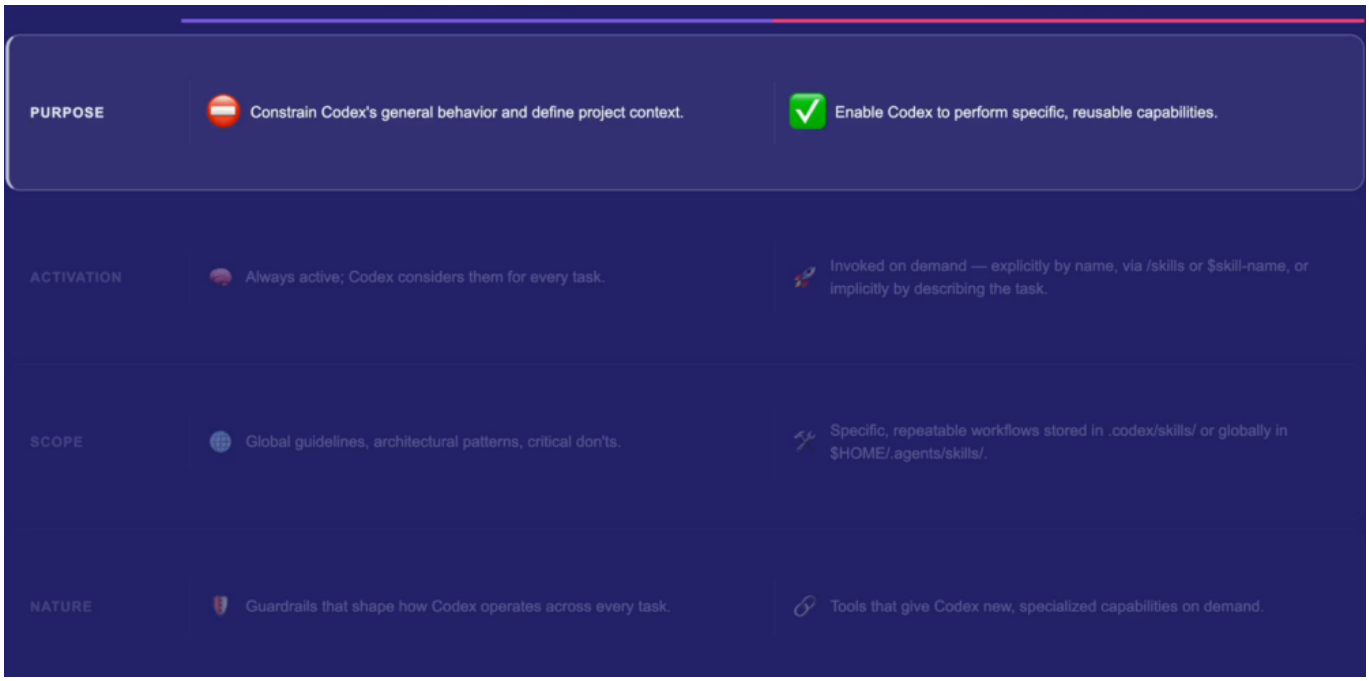
A skill is a saved, named workflow that Codex can execute on demand. It is typically stored as a Markdown file in a dedicated skills directory. The file gives Codex a clear description of the task and the exact sequence of steps to follow, which makes repeated results more consistent.

We can think of a skill as a reusable prompt template. Instead of restating every requirement whenever we perform a familiar task, we preserve the process once and invoke it when needed.

Skills and AGENTS.md rules

Skills and AGENTS.md rules influence Codex in different ways. An AGENTS.md file provides project context and constrains Codex's general behavior. Its rules are always active and serve as guardrails across every task in their scope.

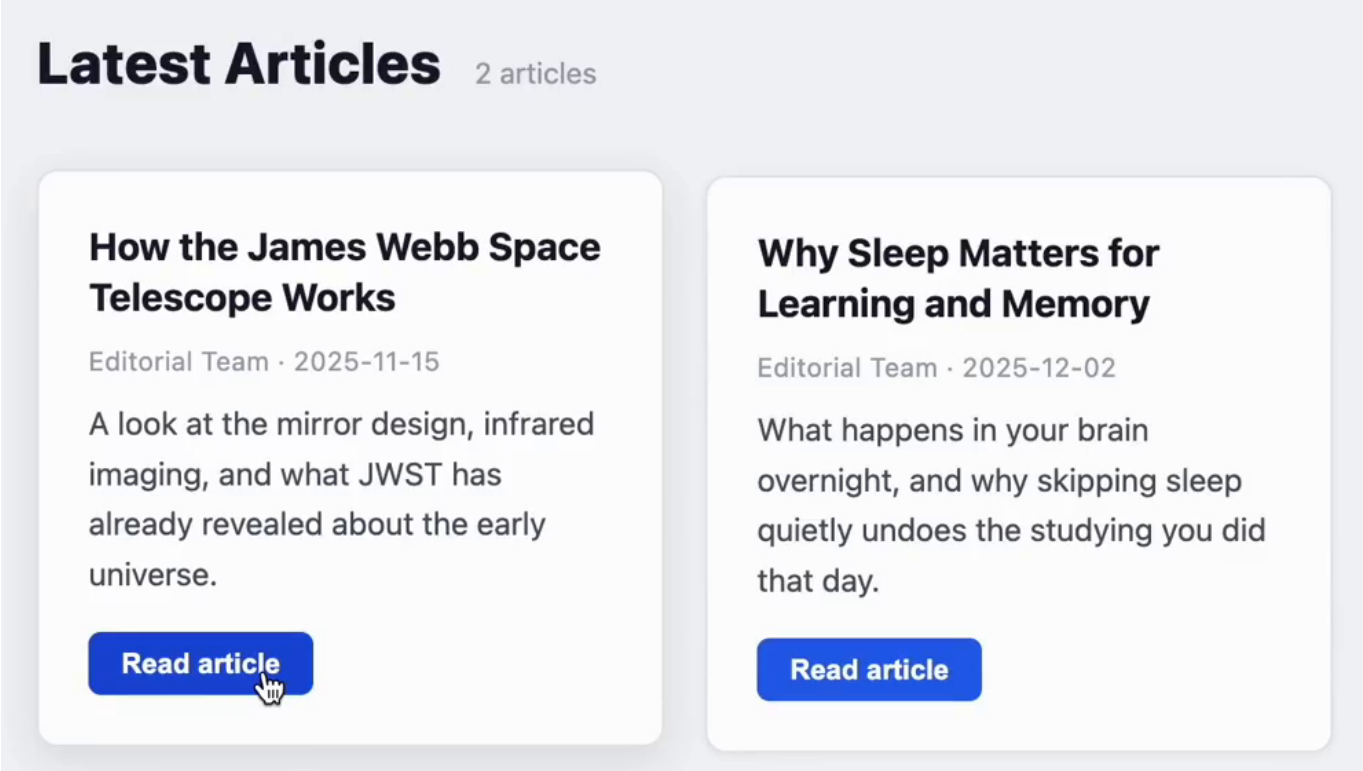
A skill provides a specific reusable capability. It is activated only when a matching task calls for it, either explicitly by name or implicitly through the task description. Rules define how Codex should operate in general, while skills define a repeatable workflow that Codex can perform when requested.



Preparing the starter project

The example uses an article showcase application called The Daily Read. Its starting page contains two article cards, and each card opens a full article before returning to the overview. We first download and inspect the starter project so we understand its current behavior and data structure.

© 2026 Zenva Pty Ltd. All rights reserved.



Our intended workflow adds a new article to this application. Codex must research the requested topic, write a concise article for a general audience, understand the fields used by articles.js, and append a new entry that follows the existing pattern.

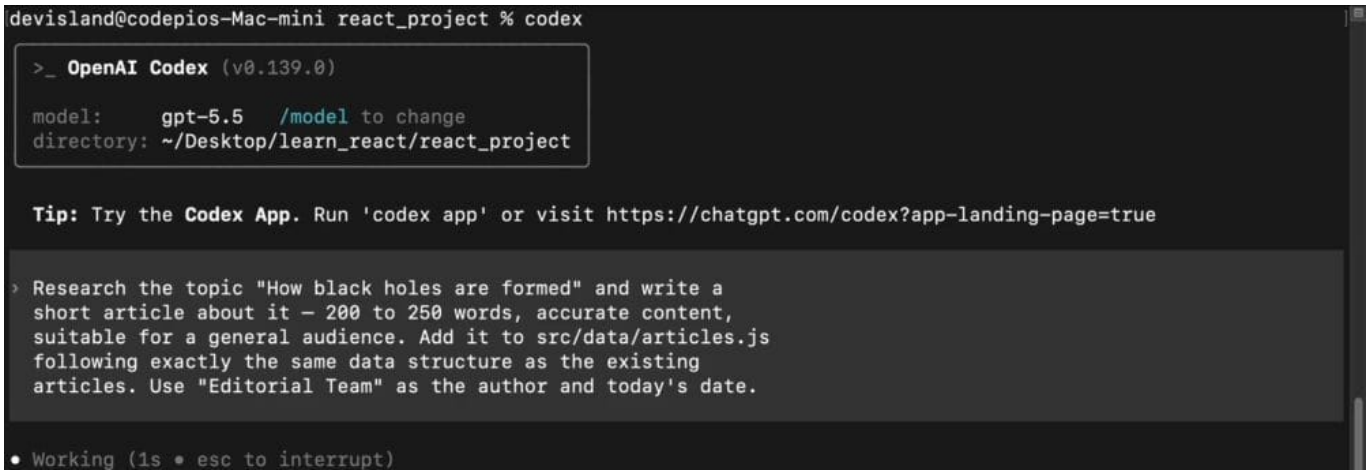
The article requirements include a clear title, well-structured content of approximately 200 to 250 words, and the existing data fields for the ID, title, summary, content, author, and date. The completed change must also pass the project’s validation checks.

Run the workflow manually

Before saving a workflow as a skill, it helps to complete it manually once. We initialize version control, stage the starter project, and create an initial commit. This gives us a known baseline and makes later changes easy to inspect.

We then ask Codex to add an article about how black holes form. Codex inspects the project and reads the existing articles to learn the required structure. It researches current, reliable information, prepares the new content, writes the new entry into articles.js, and checks the result for data cleanliness, word count, and linting.

© 2026 Zenva Pty Ltd. All rights reserved.

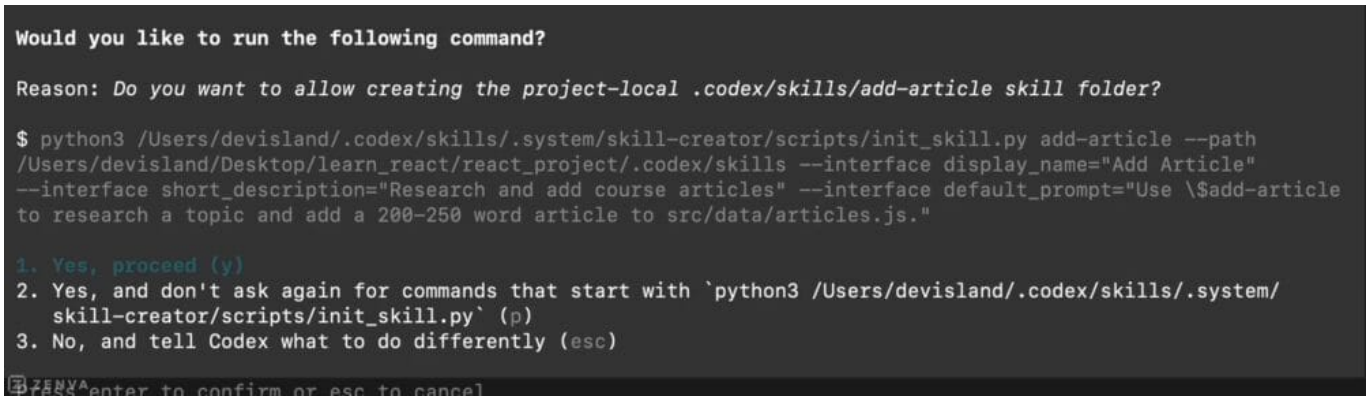


After validation passes, Codex reports the research sources it used. The application now displays a third article, How Black Holes Are Formed, which opens and returns to the main list in the same way as the original articles. This completed process gives us a concrete workflow to preserve.

Turn the workflow into a project-local skill

We ask Codex to convert the completed article workflow into a skill stored locally inside the project, rather than as a global skill. Codex evaluates the previous work, uses its skill-creation guidance, and inspects the relevant project structure before generating the required files.

Because the project does not yet contain a .codex directory, Codex prepares a project-local path for the skill. The initializer command needs to create that directory, so the sandbox requests approval. We should read the proposed command and its explanation before allowing it to continue.



Once approved, the initializer creates the add-article skill. Codex replaces the generic template with a focused workflow that covers choosing the next article ID, researching the topic, drafting the article, updating the data file, and validating word count and lint results. We review the proposed edits before confirming their application.

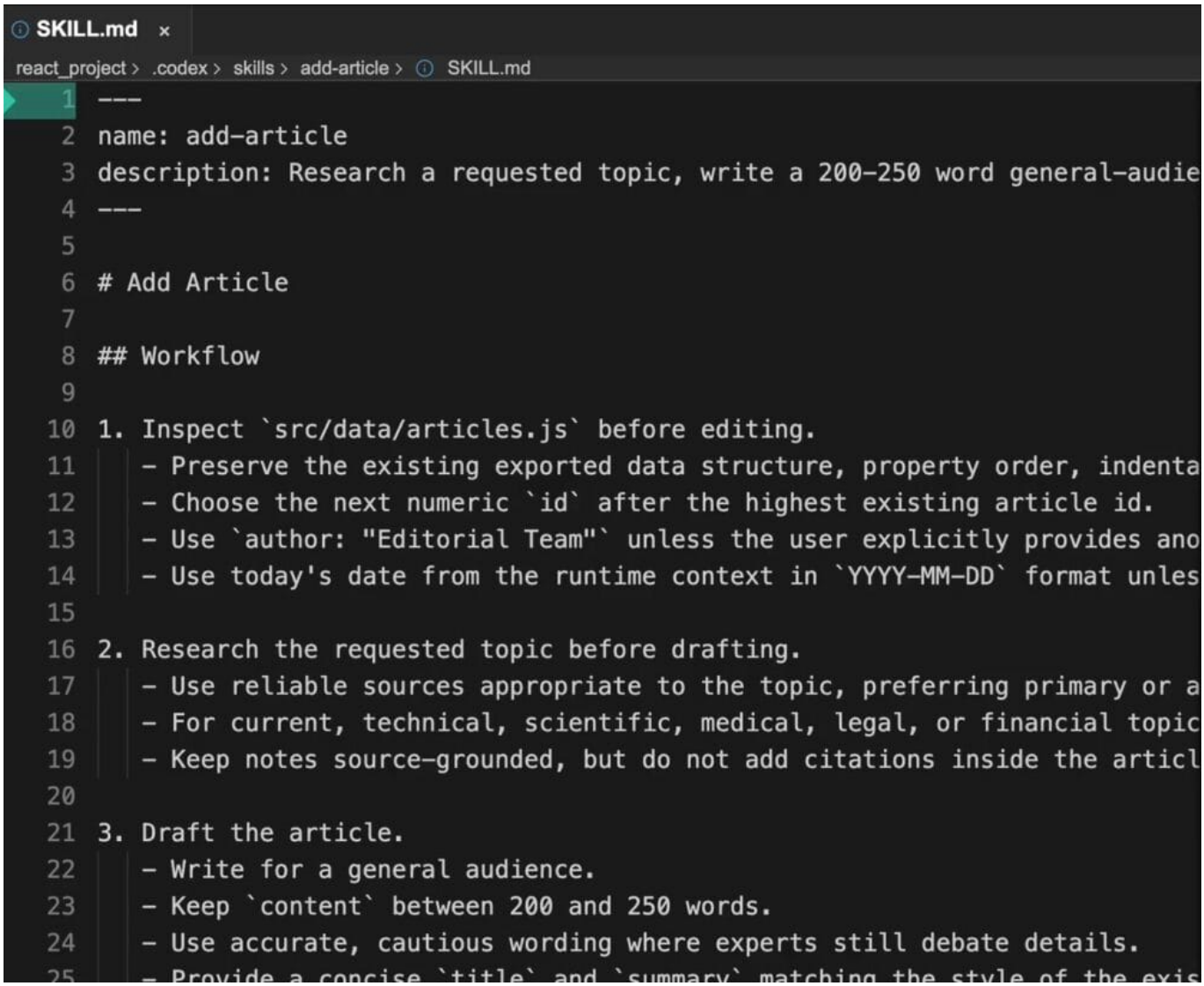
Codex then validates the new skill directory and its files. A final diff and status check confirm that the new project-local skill files are ready to commit.

Inspect the generated skill files

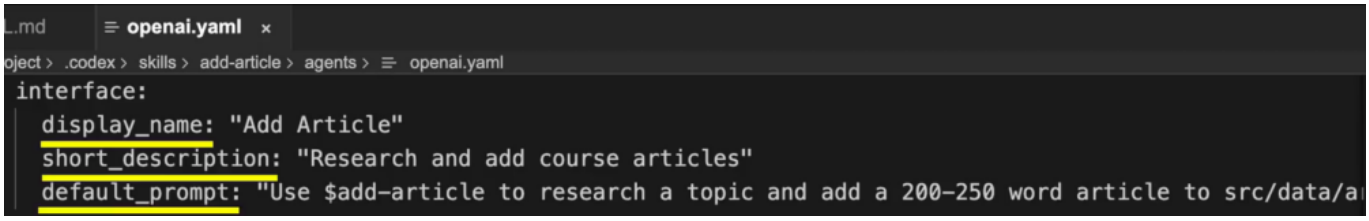
The main file is SKILL.md inside the add-article folder. Its metadata identifies the skill and describes when it should be used. Its numbered workflow tells Codex to inspect the existing article data, research the requested topic, draft for a general audience, append a matching entry, and run the

© 2026 Zenva Pty Ltd. All rights reserved.

required checks.



The skill also includes agents/openai.yaml. This machine-readable interface file defines how Codex surfaces the skill, including its display name, short description, and default prompt. It is rarely edited, but it is important for the skill's presentation and invocation.



With both files reviewed, we stage and commit the new skill so the project records the reusable workflow alongside its other source files.

Invoke a skill

Codex supports three ways to invoke a skill:

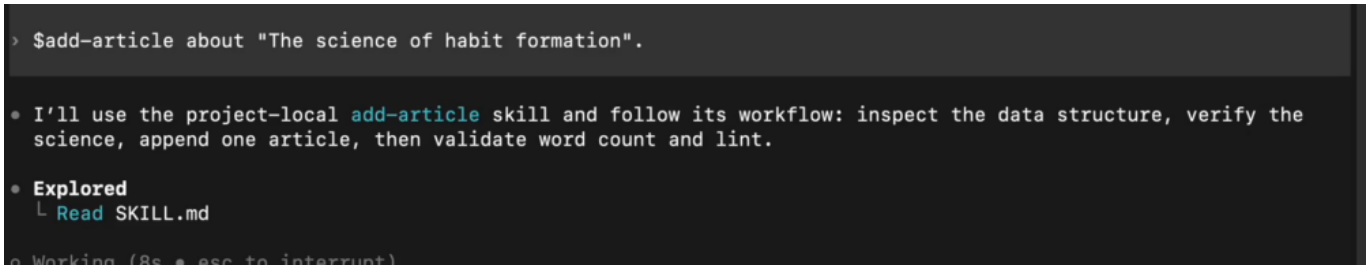
© 2026 Zenva Pty Ltd. All rights reserved.

- Use the /skills slash command followed by the skill name, such as /skills add-article.
- Use the shorter dollar-sign form, such as $add-article.
- Describe the task naturally and allow Codex to recognize that a matching skill applies.

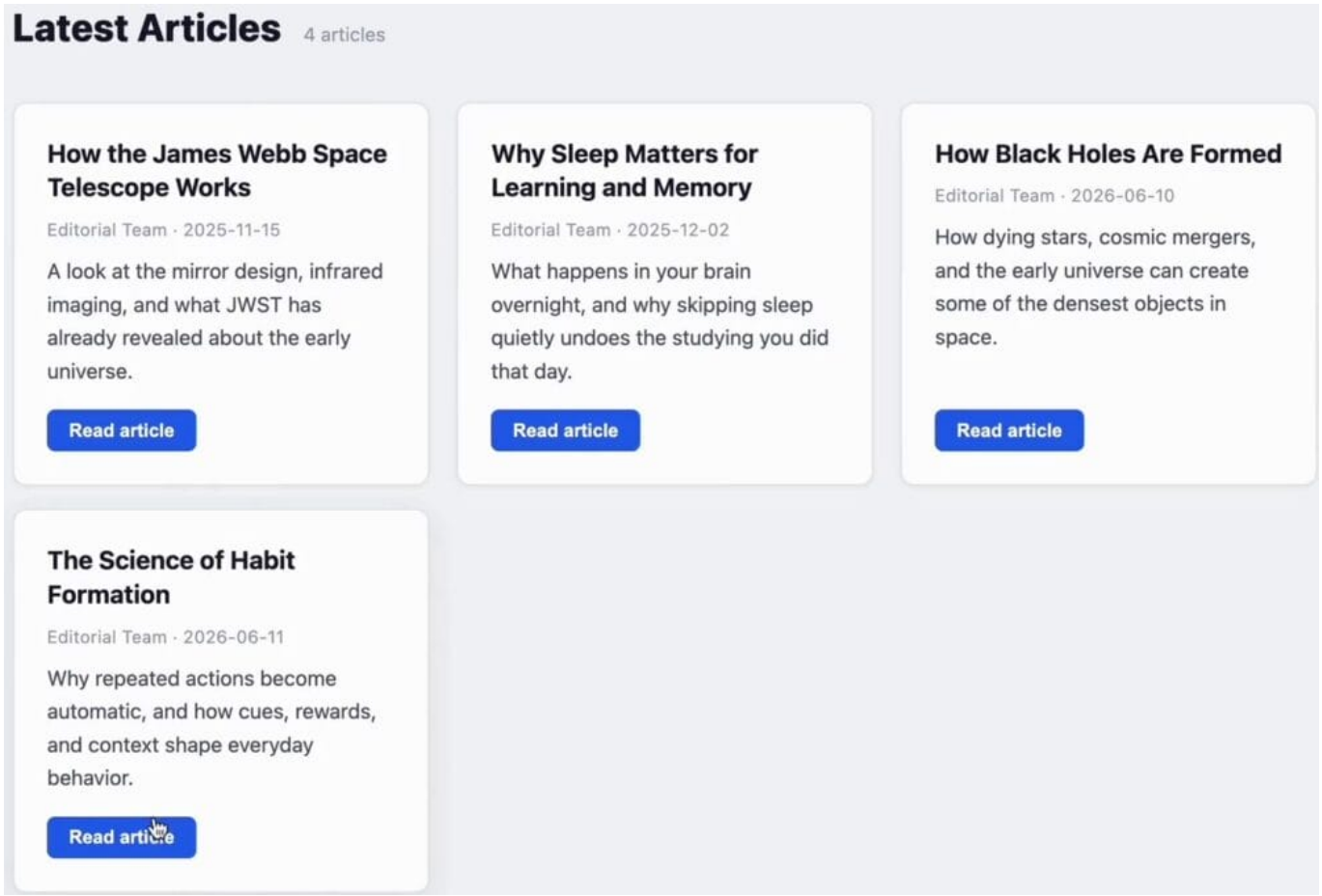
Both named forms explicitly select the skill. Implicit invocation is convenient, but an explicit command is more reliable when a particular workflow must run.

Run the reusable article skill

We invoke $add-article and request an article about the science of habit formation. Codex reads the skill definition, inspects articles.js, verifies relevant scientific facts, determines the next ID and current date, and drafts content that matches the existing data structure.



Codex appends the new article and checks its word count and linting. After the checks pass, it confirms the addition and lists the scholarly sources used for the research. The application then displays The Science of Habit Formation as the fourth article.



This second run follows the same quality checks as the manual workflow, but the saved skill supplies

© 2026 Zenva Pty Ltd. All rights reserved.

the repeatable instructions. We commit the new article change after verifying the result.

When to create a skill

Skills work best for tasks that repeat with a predictable structure. Good candidates include adding pages or components, following a consistent release checklist, and creating structured content such as the articles in this project.

One-off tasks and workflows that change substantially each time are poor candidates. We also do not need a skill for a simple operation already covered by a project-wide AGENTS.md rule. Rules should remain the always-active guardrails, while skills provide specialized capabilities on demand.

Summary

A Codex skill is a named workflow stored in Markdown and executed when needed. Unlike AGENTS.md rules, which continuously shape Codex’s behavior, skills package specific repeatable processes. By completing a workflow once, saving it as a project-local skill, reviewing its files, and invoking it explicitly or implicitly, we can produce consistent results while reducing repeated effort in future sessions.

© 2026 Zenva Pty Ltd. All rights reserved.

Codex uses config.toml files to control its model, approval behavior, and filesystem access. By combining configuration scopes, approval policies, and sandbox modes, we can give Codex enough freedom to work efficiently while preserving appropriate safeguards.

Understand the configuration hierarchy

Codex configuration is layered. Each layer can provide defaults or override settings from a broader scope:

- **Global defaults:** ~/.codex/config.toml stores personal preferences that apply across projects, such as the default model and general security settings.
- **Project-scoped settings:** .codex/config.toml lives in a project root. These settings override global defaults and can be committed to Git so everyone working on the project receives the same behavior.
- **Session profiles:** a profile provides temporary settings for one Codex session. It can override the other layers without permanently changing either configuration file.

This hierarchy lets us keep convenient personal defaults while applying stricter or more specialized rules to individual projects. A temporary profile then gives us a final layer for one-off work.

Choose an approval policy

The approval policy determines when Codex pauses and asks us to authorize an action. The three policies covered in this lesson offer different balances between automation and oversight:

- untrusted lets Codex read and edit files, but asks for approval before running commands considered untrusted. This is a cautious starting point because routine file work remains convenient while riskier commands receive review.
- on-request gives Codex more freedom inside the current workspace. It can read files, modify files, and run normal commands without prompting, then request approval when it needs elevated access, such as network access or interaction outside the workspace.
- never prevents approval prompts and allows Codex to proceed without explicit permission. We should reserve this policy for highly controlled environments with strong external safeguards.

Approval policy does not define which files Codex can access. That boundary is controlled separately by the sandbox.

Choose a sandbox mode

The sandbox mode limits what Codex can read or modify on the filesystem:

- read-only allows Codex to inspect project files without changing them. It is useful for analysis, reviews, and safely exploring an unfamiliar repository.
- workspace-write allows Codex to read and write within the current workspace. This is the most practical choice for everyday development because Codex can modify the project without gaining unrestricted filesystem access.
- danger-full-access grants unrestricted filesystem access, including locations outside the workspace. We should only use it when the environment is tightly controlled and we fully understand the risks.

The sandbox and approval policy work together. The sandbox establishes a boundary, while the approval policy determines whether Codex can ask to cross a restricted boundary.

Create a project configuration

© 2026 Zenva Pty Ltd. All rights reserved.

To add project-specific settings, create a hidden .codex directory in the project root and place an empty config.toml file inside it:

```
mkdir -p .codex && touch .codex/config.toml
```

A sensible starter configuration combines workspace-limited file access with approval prompts for untrusted commands:

```
# Sandbox mode: allow changes only inside the current workspace
sandbox_mode = "workspace-write"

# Approval policy: ask before running untrusted commands
approval_policy = "untrusted"
```

This setup lets Codex perform normal file edits inside the project while retaining oversight for commands that require additional trust.

Version the shared policy

Because .codex/config.toml defines project behavior, we should commit it to Git. This makes the security and permission settings visible, reviewable, and consistent for the whole team:

```
git add .codex/config.toml && git commit -m "add codex config - sandbox and approval policy"
```

Configuration changes deserve the same review as other project changes because they affect what Codex can do in the repository.

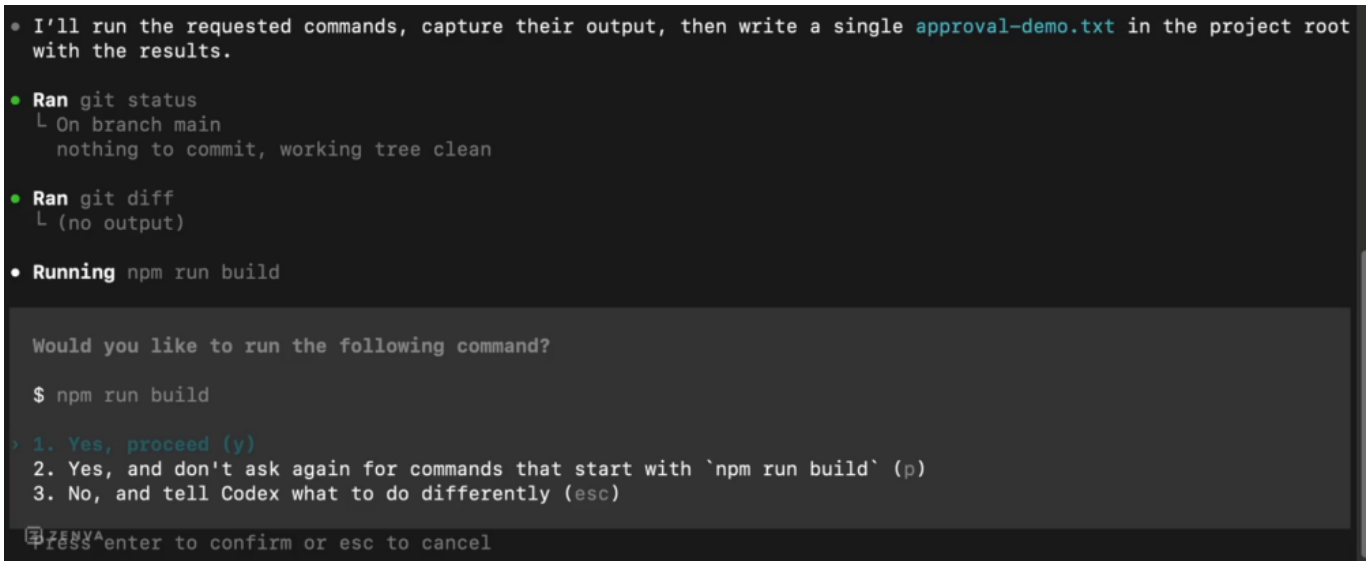
Test the untrusted policy

We can test the starter configuration by asking Codex to create approval-demo.txt from the outputs of three commands:

```
git status
git diff
npm run build
```

Under the untrusted policy, Codex can run the safe inspection commands without interruption. When it reaches npm run build, it pauses and presents approval choices. We can approve the single command, approve it and remember the command prefix, or reject it and provide different instructions.

© 2026 Zenva Pty Ltd. All rights reserved.



After we approve the command and the file edit, Codex completes the task and can read the generated file back to verify that its contents match the captured command outputs.

Switch to on-request approval

For a less interrupted workflow inside the workspace, change the project approval policy while keeping the same sandbox:

```
sandbox_mode = "workspace-write"
approval_policy = "on-request"
```

The on-request policy allows normal work within the workspace without initial approval. Codex only pauses when an action requires elevated permissions, such as network access or interaction with files outside the permitted workspace. Commit this configuration change so the updated behavior is shared and tracked.

To verify the difference, delete the earlier demo file, start a new Codex session, and repeat the same request:

```
Create a file called approval-demo.txt.

Run these commands:
git status
git diff
npm run build

Write the results into approval-demo.txt.
```

Codex now plans the work, runs all three commands, writes their outputs to the file, and performs a readback without asking for approval. The sandbox still limits file access to the workspace, but the approval policy no longer interrupts normal in-workspace commands.

Set global defaults

© 2026 Zenva Pty Ltd. All rights reserved.

Personal defaults belong in ~/.codex/config.toml. For example, we can select a preferred model and provide default security settings for every new project:

```
# Global defaults that apply to every project
# unless a project-specific configuration overrides them.
model = "gpt-5.5"
sandbox_mode = "workspace-write"
approval_policy = "untrusted"
```

If a project contains its own .codex/config.toml, the project-specific values take precedence over these global defaults. This gives us a consistent personal baseline without preventing a repository from enforcing its own requirements.

Use stricter settings for sensitive projects

Projects that involve databases, production deployments, or other high-impact operations benefit from tighter restrictions. A read-only sandbox paired with on-request approval creates a review-focused configuration:

```
# Stricter config for projects with database access
sandbox_mode = "read-only"
approval_policy = "on-request"
```

Codex can freely read and analyze the codebase, but writing a file or running a command requires leaving the read-only boundary. The on-request policy allows Codex to stop and ask for permission at that point. Nothing proceeds until we approve it, producing a slower but fully reviewed workflow where mistakes would carry a high cost.

© 2026 Zenva Pty Ltd. All rights reserved.

Intro to Agentic Coding with Codex - Reuse and Configure Codex

1. When is a workflow a good skill candidate?

a. When it repeats with predictable steps
b. When it is one-off and still unclear
c. When it should govern every task
d. When it has no validation step

2. Your team wants Codex to edit only inside the repository and ask before untrusted commands. Which project settings fit?

a. `workspace-write` with `untrusted`
b. `danger-full-access` with `never`
c. `read-only` with `never`
d. `danger-full-access` with `on-request`

© 2026 Zenva Pty Ltd. All rights reserved.

We have successfully completed Codex from Zero to Productive. Let’s review the practical skills we can now use to work with Codex confidently and effectively.

Codex setup and security

We can install and configure Codex to suit our needs. We can also read the signals shown in the terminal interface and understand Codex’s data privacy model, helping us make informed decisions for each project.

Strategic sessions and context

We can manage and resume Codex sessions strategically. By using AGENTS.md, we can give future sessions the context and guidance they need to build efficiently on our previous progress.

Precise prompting

We can construct effective prompts using context, a clear task, constraints, and a defined output format. We can also provide real file examples and use negative constraints to guide Codex toward consistent and precise results.

Planning and reusable workflows

Plan mode lets us think through an entire feature before writing code. We can scope work into clean, committable slices and reduce unnecessary rework.

We can also save reusable workflows as skills, turning a process we have completed once into something we can invoke explicitly or implicitly in future Codex sessions.

Approval policy and sandboxing

Finally, we can control Codex’s approval policy and sandbox behavior through config.toml. This allows us to add a safe starter configuration to a project and give Codex the appropriate level of access and security.

With these skills, we are ready to configure Codex, guide its work, and reuse effective processes across future projects.

© 2026 Zenva Pty Ltd. All rights reserved.

In this lesson, you can find the full source code for the project used within the course. Feel free to use this lesson as needed – whether to help you debug your code, use it as a reference later, or expand the project to practice your skills!

You can also download all project files from the **Course Files** tab via the project files or downloadable PDF summary.

To Do List App

AGENTS.md

Found in folder: **/**

This file provides project context and guidelines for developers working on the Todo app. It documents the technology stack, available commands, and coding conventions. It also establishes critical rules like extracting magic numbers into utility files.

```
# AGENTS.md

This file gives Codex context about the project.

## Project

A beginner-friendly Todo app built with React and Vite, used as a starter
project for a Codex course. Students extend the app by adding missing
features — guided by Codex.

## Stack

| Layer       | Choice                        |
|-------------|-------------------------------|
| UI          | React 19                      |
| Bundler     | Vite 7                        |
| Styling     | CSS Modules (plain CSS only)  |
| Tests       | Vitest + Testing Library      |
| Persistence | localStorage                  |

No backend. No routing. No auth. No external UI libraries.

## Commands

| Command           | Purpose                          |
|-------------------|----------------------------------|
| `npm run dev`     | Start dev server (localhost:5173)|
| `npm run build`   | Production build                 |
| `npm test`        | Run tests in watch mode          |
| `npm run test:ui` | Open the Vitest browser UI       |

## Conventions

- All state lives in `App.jsx`; components receive data and callbacks as props
- One component per file; its CSS module lives in the same directory
- Task shape: `{ id: string, text: string, completed: boolean }`
- See `/docs/ui.md` for styling rules
- See `/docs/data-fetching.md` for state and persistence rules
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
## Critical Rules

- Always extract magic numbers and configuration values into a dedicated
  `src/utils/` file. Never hardcode them inline inside a component.
```

README.md

Found in folder: **/**

This file is a project README that documents a Todo app starter for a Codex course. It lists the app's existing features, missing features to be implemented as exercises, and provides setup instructions. The document also outlines the project structure and technology stack used.

```
# Todo App — Codex Course Starter

A deliberately incomplete Todo app. You will use **Codex** to add the
missing features listed below.

## Getting started

```bash
npm install
npm run dev        # http://localhost:5173
```

Running tests

```bash
npm test           # watch mode
npm run test:ui    # browser UI
```

One test passes. One test fails — that is intentional. Fixing it is your
first exercise.

What already works

Feature	File
Add a task	`TaskInput.jsx`
Mark complete (checkbox)	`TaskItem.jsx`
Delete a task	`TaskItem.jsx`
Character counter (max 100, turns red)	`TaskInput.jsx`
Filter bar — All / Active / Completed	`FilterBar.jsx`
localStorage persistence	`App.jsx`

What is missing (your exercises)

- **Empty-input validation** — clicking Add Task with a blank field still
  creates an empty task. Add a guard in `TaskInput.jsx` to prevent this.
  The failing test in `src/__tests__/tasks.test.jsx` will go green once you do.
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
- **Task editing** — double-clicking a task text should let you edit it
  in place and save on Enter / blur.

- **Due dates** — tasks have no deadline field. Add a date picker to
  `TaskInput` and display the date in `TaskItem`.

- **Drag to reorder** — tasks are fixed in insertion order.

## Project structure

```
src/
  components/
    TaskInput.jsx + TaskInput.module.css
    TaskList.jsx  + TaskList.module.css
    TaskItem.jsx  + TaskItem.module.css
    FilterBar.jsx + FilterBar.module.css
  App.jsx + App.module.css
  main.jsx
  index.css
docs/
  ui.md            — styling conventions
  data-fetching.md — state and persistence conventions
vitest.config.js
```

Stack

- React 19 + Vite 7
- CSS Modules only (no Tailwind, no UI libraries)
- Vitest + Testing Library
- localStorage (no backend)
```

data-fetching.md

Found in folder: **/docs**

This documentation describes the state structure, persistence mechanism, and data flow for a todo application. It defines how task objects are shaped, stored in localStorage, and manipulated through callbacks. It also explains how tasks are filtered for display without mutating the original data.

```
# Data & State Conventions

## State Shape

All app state lives in `App.jsx`. There is no global state manager.

```js
// Task object — the single source of truth
{
  id: string,        // crypto.randomUUID(), set at creation time
  text: string,      // 1–100 characters (MAX_LENGTH enforced in TaskInput)
  completed: boolean // toggled by the checkbox
```

© 2026 Zenva Pty Ltd. All rights reserved.
```

````
}
```

The active filter (`'all' | 'active' | 'completed'`) is also stored in `App`
but is never persisted — it resets to `'all'` on page load.

## Persistence

Tasks are persisted to `localStorage` under the key `"todo-tasks"`.

**Read** — lazy initialiser so it runs exactly once on mount:

```js
const [tasks, setTasks] = useState(() => {
  const stored = localStorage.getItem('todo-tasks')
  return stored ? JSON.parse(stored) : []
})
```

**Write** — effect that syncs whenever `tasks` changes:

```js
useEffect(() => {
  localStorage.setItem('todo-tasks', JSON.stringify(tasks))
}, [tasks])
```

## Task Operations

All mutations live in `App.jsx` and are passed to children as callbacks:

| Prop | Signature | Behaviour |
|---|---|---|
| `onAdd` | `(text: string) => void` | Prepends a new task to the array |
| `onToggle` | `(id: string) => void` | Flips `completed` on one task |
| `onDelete` | `(id: string) => void` | Removes the task from the array |

Mutations always use the functional updater form (`setTasks(prev => ...)`) to
avoid stale-closure bugs.

## Filtering

Filtering is derived — it does **not** mutate `tasks`:

```js
const filteredTasks = tasks.filter(task => {
  if (filter === 'active') return !task.completed
  if (filter === 'completed') return task.completed
  return true
})
```

Only `filteredTasks` is passed to ``. The full `tasks` array is
used for the count in ``.
````

© 2026 Zenva Pty Ltd. All rights reserved.
````

```
## What Is Not Here (yet)

- No API calls — everything is local; no `fetch`, no async state
- No loading or error states
- No optimistic updates
- No task editing, due dates, or ordering — those are student exercises in the Codex
course


File extension: md
```

ui.md

Found in folder: **/docs**

This document defines UI conventions including layout structure, color palette, border radii, and interaction guidelines for consistent component styling. It also outlines the component hierarchy and provides steps for adding new components.

```
# UI Conventions

## Layout

- All components live in `src/components/`
- Each component has a matching `.module.css` file in the same directory
- Max content width: `600px`, centered via `margin: auto` in `App.module.css`
- Page background: `#f3f4f6` (set in `src/index.css`)

## Colour Palette

| Token        | Value     | Used for                          |
|--------------|-----------|-----------------------------------|
| Primary      | `#6366f1` | Buttons, focus rings, checkboxes  |
| Primary dark | `#4f46e5` | Button hover                      |
| Danger       | `#ef4444` | Delete hover, character limit     |
| Danger light | `#fee2e2` | Delete button hover background    |
| Border       | `#d1d5db` | Default input / pill borders      |
| Border inner | `#e5e7eb` | Task item border                  |
| Text muted   | `#9ca3af` | Empty state, count, delete icon   |
| Text base    | `#1a1a1a` | Body copy                         |
| Surface      | `#fff`    | Cards, inputs                     |

## Border Radii

| Element              | Radius |
|----------------------|--------|
| Inputs, buttons      | `6px`  |
| Task list items      | `8px`  |
| Filter pills         | `20px` |
| Delete button        | `4px`  |

## Interaction
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
- Every interactive element has a `transition` on whichever property changes
- Default transition duration: `0.15s`
- Checkboxes use `accent-color: #6366f1`

## Component Hierarchy

```
App
??? TaskInput      — input field, Add Task button, character counter
??? FilterBar      — All / Active / Completed pills + task count
??? TaskList
    ??? TaskItem   — checkbox, task text, delete button
```

Adding a New Component

1. Create `src/components/ComponentName.jsx`
2. Create `src/components/ComponentName.module.css` alongside it
3. Import CSS with `import styles from './ComponentName.module.css'`
4. Apply class names with `className={styles.className}`
5. Wire it into the appropriate parent component; lift any new state to `App.jsx`
```

App.jsx

Found in folder: **/src**

This code defines the main App component for a todo application. It manages tasks with state for the list, a filter, and editing, and persists tasks to local storage. It provides functions to add, toggle, edit, and delete tasks, and renders input, filter, and list components.

```
import { useState, useEffect } from 'react'
import TaskInput from './components/TaskInput'
import TaskList from './components/TaskList'
import FilterBar from './components/FilterBar'
import styles from './App.module.css'

function App() {
  // Initialize tasks from localStorage, or start with an empty array
  const [tasks, setTasks] = useState(() => {
    const stored = localStorage.getItem('todo-tasks')
    return stored ? JSON.parse(stored) : []
  })
  const [filter, setFilter] = useState('all') // 'all', 'active', or 'completed'
  const [editingTaskId, setEditingTaskId] = useState(null) // ID of task being edited
  const [editingText, setEditingText] = useState('') // Text for the task being edite
d

  // Persist tasks to localStorage whenever the tasks state changes
  useEffect(() => {
    localStorage.setItem('todo-tasks', JSON.stringify(tasks))
  }, [tasks])

  const addTask = (text) => {
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
  setTasks(prev => [
    { id: crypto.randomUUID(), text, completed: false },
    ...prev, // Add new task to the beginning of the list
  ])
}

const toggleTask = (id) => {
  setTasks(prev =>
    prev.map(task =>
      task.id === id ? { ...task, completed: !task.completed } : task
    )
  )
}

const saveTaskEdit = () => {
  if (!editingTaskId) return

  const trimmedText = editingText.trim()

  if (trimmedText) {
    setTasks(prev =>
      prev.map(task =>
        task.id === editingTaskId ? { ...task, text: trimmedText } : task
      )
    )
  }

  setEditingTaskId(null)
  setEditingText('')
}

const startTaskEdit = (task) => {
  // If editing a different task, save it first
  if (editingTaskId && editingTaskId !== task.id) {
    saveTaskEdit()
  }

  setEditingTaskId(task.id)
  setEditingText(task.text)
}

const cancelTaskEdit = () => {
  setEditingTaskId(null)
  setEditingText('')
}

const deleteTask = (id) => {
  // If deleting the task currently being edited, cancel edit first
  if (editingTaskId === id) {
    cancelTaskEdit()
  }

  setTasks(prev => prev.filter(task => task.id !== id))
}
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
  // Filter tasks based on the current filter state
  const filteredTasks = tasks.filter(task => {
    if (filter === 'active') return !task.completed
    if (filter === 'completed') return task.completed
    return true // 'all' filter
  })

  return (
```

Todo

```
  )
}

export default App
```

main.jsx

Found in folder: **/src**

This code initializes a React application by rendering the main App component inside a React StrictMode wrapper into the HTML element with the ID ‘root’. It imports necessary React modules, global styles, and the root App component. The application is mounted to the DOM element identified as ‘root’.

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css' // Global styles
import App from './App.jsx'

// Mount the React app to the DOM element with ID 'root'
createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>
)
```

FilterBar.jsx

Found in folder: **/src/components**

This code renders a set of filter buttons and a task counter. It highlights the currently active filter and updates the filter when a button is clicked. The task counter displays the number of tasks with proper pluralization.

© 2026 Zenva Pty Ltd. All rights reserved.

```
import styles from './FilterBar.module.css'

// Available filter options
const FILTERS = ['all', 'active', 'completed']

export default function FilterBar({ filter, onFilterChange, taskCount }) {
  return (

      {FILTERS.map(f => (
         onFilterChange(f)}
        >
          {f.charAt(0).toUpperCase() + f.slice(1)} {/* Capitalize filter name */}

      ))}

        {taskCount} {taskCount === 1 ? 'task' : 'tasks'} {/* Handle pluralization */}



  )
}
```

TaskInput.jsx

Found in folder: **/src/components**

This component provides a text input and button for adding new tasks. It tracks the input text, trims whitespace when adding, and enforces a maximum length. It also displays a character counter that changes style when the limit is reached.

```
import { useState } from 'react'
import { TASK_TEXT_MAX_LENGTH } from '../utils/taskConfig'
import styles from './TaskInput.module.css'

export default function TaskInput({ onAdd }) {
  const [text, setText] = useState('')

  const handleAdd = () => {
    // TODO: add empty-input validation here — currently submits blank tasks
    onAdd(text.trim())  // Pass trimmed text to parent handler
    setText('')         // Clear input after adding
  }

  return (


                 className={styles.input}
          type="text"
          value={text}
          onChange={e => setText(e.target.value)}
          onKeyDown={e => e.key === 'Enter' && handleAdd()}  // Submit on Enter key
          placeholder="What needs to be done?"
          maxLength={TASK_TEXT_MAX_LENGTH}
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
          aria-label="New task"
        />

          Add Task



      {/* Character counter with visual limit indicator */}


= TASK_TEXT_MAX_LENGTH ? styles.atLimit : ''}`}>
        {text.length} / {TASK_TEXT_MAX_LENGTH}




  )
}
```

TaskItem.jsx

Found in folder: **/src/components**

This component renders a single task item in a list, displaying its text and allowing it to be toggled, edited, or deleted. It manages an editing state where the task text becomes a focused input field, handling save and cancel actions via keyboard and blur events. It also plays a sound when a task is marked as complete.

```
import { useEffect, useRef } from 'react'
import { playCompletionSound } from '../utils/audio'
import { TASK_TEXT_MAX_LENGTH } from '../utils/taskConfig'
import styles from './TaskItem.module.css'

export default function TaskItem({
  task,
  isEditing,
  editingText,
  onEditStart,
  onEditTextChange,
  onEditSave,
  onEditCancel,
  onToggle,
  onDelete,
}) {
  const editInputRef = useRef(null)
  const isCancellingEditRef = useRef(false) // Tracks if edit was cancelled via Escap
e key to prevent onBlur from saving

  // Focus and select text in the input field when editing mode starts
  useEffect(() => {
    if (isEditing) {
      editInputRef.current?.focus()
      editInputRef.current?.select()
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
    }
  }, [isEditing])

  const handleToggle = () => {
    if (!task.completed) {
      playCompletionSound() // Play sound only when marking as complete, not when unc
hecking
    }

    onToggle(task.id)
  }

  const handleEditKeyDown = (event) => {
    if (event.key === 'Enter') {
      onEditSave() // Save on Enter key
    }

    if (event.key === 'Escape') {
      isCancellingEditRef.current = true // Set flag to indicate cancellation
      onEditCancel()
    }
  }

  const handleEditBlur = () => {
    // If edit was cancelled via Escape, don't save on blur
    if (isCancellingEditRef.current) {
      isCancellingEditRef.current = false
      return
    }

    onEditSave() // Save on blur (when input loses focus)
  }

  return (

              type="checkbox"
        className={styles.checkbox}
        checked={task.completed}
        onChange={handleToggle}
        aria-
label={`Mark "${task.text}" as ${task.completed ? 'active' : 'complete'}`}
      />
      {isEditing ? (
                  ref={editInputRef}
          className={styles.editInput}
          type="text"
          value={editingText}
          onChange={event => onEditTextChange(event.target.value)}
          onKeyDown={handleEditKeyDown}
          onBlur={handleEditBlur}
          maxLength={TASK_TEXT_MAX_LENGTH}
          aria-label={`Edit "${task.text}"`}
        />
      ) : (
         onEditStart(task)}>
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
          {task.text}

      )}
       onDelete(task.id)}
        aria-label={`Delete "${task.text}"`}
      >
        ?


  )
}
```

TaskList.jsx

Found in folder: **/src/components**

This component renders a list of tasks. If the list is empty, it displays a message. Otherwise, it maps each task to a TaskItem component, passing down the task data and all relevant event handlers.

```
import TaskItem from './TaskItem'
import styles from './TaskList.module.css'

export default function TaskList({
  tasks,
  editingTaskId,
  editingText,
  onEditStart,
  onEditTextChange,
  onEditSave,
  onEditCancel,
  onToggle,
  onDelete,
}) {
  // Show a placeholder message if there are no tasks
  if (tasks.length === 0) {
    return

No tasks here.

  }

  return (


            {tasks.map(task => (

            ))}


  )
}
```

© 2026 Zenva Pty Ltd. All rights reserved.

TaskNote.jsx

Found in folder: **/src/components**

This component conditionally renders a note for a task if one exists, providing an accessible label. It uses CSS modules for styling and returns a paragraph element containing the note text.

```
import styles from './TaskNote.module.css'

export default function TaskNote({ task }) {
  // Do not render anything if the task has no note
  if (!task.note) return null

  return (
    // Provide an accessible description for the note


      {task.note}


  )
}
```

tasks.test.jsx

Found in folder: **/src/__tests__**

This code tests a Todo app's UI behavior, including adding, editing, and completing tasks. It verifies that tasks are properly managed and that specific actions like double-clicking to edit or pressing Escape to cancel work correctly. It also includes a test for a sound effect when marking a task complete, using a mocked audio context.

```
import { render, screen, fireEvent } from '@testing-library/react'
import { describe, it, expect, beforeEach, vi } from 'vitest'
import App from '../App'

// Helper function to add a task via UI interactions
const addTask = (text) => {
  fireEvent.change(screen.getByLabelText('New task'), {
    target: { value: text },
  })
  fireEvent.click(screen.getByRole('button', { name: 'Add Task' }))
}

// Reset state before each test to ensure isolation
beforeEach(() => {
  localStorage.clear()
  vi.restoreAllMocks()
  delete window.AudioContext
  delete window.webkitAudioContext
})

describe('Todo App', () => {
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
// PASSING: the app should render with a visible heading.
it('renders the app title', () => {
  render()
  expect(screen.getByRole('heading', { name: 'Todo' })).toBeInTheDocument()
})

// FAILING: this test documents a missing feature.
//
// The app currently adds a task even when the input is empty.
// Your job: open TaskInput.jsx and add a guard in handleAdd() so that
// clicking "Add Task" with a blank field does nothing.
// Once you do, this test will go green.
it('does not add a task when the input is empty', () => {
  render()

  fireEvent.click(screen.getByRole('button', { name: 'Add Task' }))

  const items = screen.queryAllByRole('listitem')
  expect(items).toHaveLength(0)
})

// Tests that a sound is played when a task is marked complete
it('plays a short sound when a task is marked complete', () => {
  const start = vi.fn()
  const stop = vi.fn()
  const connect = vi.fn()
  const setValueAtTime = vi.fn()
  const exponentialRampToValueAtTime = vi.fn()

  // Mock AudioContext to avoid real audio and track calls
  class FakeAudioContext {
    constructor() {
      this.currentTime = 0
      this.destination = {}
      this.state = 'running'
    }

    createOscillator() {
      return {
        connect,
        frequency: { setValueAtTime, exponentialRampToValueAtTime },
        start,
        stop,
        type: '',
      }
    }

    createGain() {
      return {
        connect,
        gain: { setValueAtTime, exponentialRampToValueAtTime },
      }
    }
  }
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
    window.AudioContext = FakeAudioContext

    render()

    fireEvent.change(screen.getByLabelText('New task'), {
      target: { value: 'Write sound test' },
    })
    fireEvent.click(screen.getByRole('button', { name: 'Add Task' }))

    const checkbox = screen.getByRole('checkbox', {
      name: 'Mark "Write sound test" as complete',
    })

    fireEvent.click(checkbox)
    expect(start).toHaveBeenCalledTimes(1)
    expect(stop).toHaveBeenCalledTimes(1)

    // Sound should only play when marking complete, not when unchecking
    fireEvent.click(checkbox)
    expect(start).toHaveBeenCalledTimes(1)
  })

  it('shows an edit input when a task is double-clicked', () => {
    render()
    addTask('Edit this task')

    fireEvent.doubleClick(screen.getByText('Edit this task'))

    expect(screen.getByRole('textbox', { name: 'Edit "Edit this task"' })).toHaveValu
e(
      'Edit this task',
    )
  })

  it('saves edited task text when Enter is pressed', () => {
    render()
    addTask('Old task text')

    fireEvent.doubleClick(screen.getByText('Old task text'))
    const editInput = screen.getByRole('textbox', { name: 'Edit "Old task text"' })
    fireEvent.change(editInput, { target: { value: 'New task text' } })
    fireEvent.keyDown(editInput, { key: 'Enter' })

    expect(screen.getByText('New task text')).toBeInTheDocument()
    expect(screen.queryByText('Old task text')).not.toBeInTheDocument()
  })

  it('saves edited task text when the edit input loses focus', () => {
    render()
    addTask('Blur save task')

    fireEvent.doubleClick(screen.getByText('Blur save task'))
    const editInput = screen.getByRole('textbox', { name: 'Edit "Blur save task"' })
    fireEvent.change(editInput, { target: { value: 'Saved on blur' } })
    fireEvent.blur(editInput)
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
    expect(screen.getByText('Saved on blur')).toBeInTheDocument()
    expect(screen.queryByText('Blur save task')).not.toBeInTheDocument()
  })

  it('cancels edited task text when Escape is pressed', () => {
    render()
    addTask('Keep original text')

    fireEvent.doubleClick(screen.getByText('Keep original text'))
    const editInput = screen.getByRole('textbox', { name: 'Edit "Keep original text"'
 })
    fireEvent.change(editInput, { target: { value: 'Discard this text' } })
    fireEvent.keyDown(editInput, { key: 'Escape' })

    expect(screen.getByText('Keep original text')).toBeInTheDocument()
    expect(screen.queryByText('Discard this text')).not.toBeInTheDocument()
  })

  // Ensures empty or whitespace-only edits are ignored
  it('keeps the original task text when a blank edit is saved', () => {
    render()
    addTask('Do not blank me')

    fireEvent.doubleClick(screen.getByText('Do not blank me'))
    const editInput = screen.getByRole('textbox', { name: 'Edit "Do not blank me"' })
    fireEvent.change(editInput, { target: { value: '   ' } })
    fireEvent.keyDown(editInput, { key: 'Enter' })

    expect(screen.getByText('Do not blank me')).toBeInTheDocument()
  })
})
```

index.html

Found in folder: **/**

This code defines the basic structure of an HTML page, setting up metadata, linking resources, and providing a container for a React application to render into.

© 2026 Zenva Pty Ltd. All rights reserved.

App.module.css

Found in folder: **/src**

This CSS code defines styles for two classes: `.app` centers a container with a maximum width and sets a system font, while `.title` styles a large, bold, centered heading with specific spacing and color.

```
.app {
  max-width: 600px;          /* Limits container width */
  margin: 48px auto;         /* Centers container with top/bottom margin */
  padding: 0 16px;           /* Adds horizontal padding */
  font-family: system-ui, -apple-system, sans-serif; /* Uses system fonts */
}

.title {
  font-size: 2rem;           /* Sets large font size */
  font-weight: 700;          /* Makes text bold */
  text-align: center;        /* Centers text horizontally */
  margin: 0 0 32px;          /* Adds bottom margin only */
  color: #1a1a1a;            /* Sets dark gray color */
  letter-spacing: -0.5px;    /* Tightens letter spacing slightly */
}
```

index.css

Found in folder: **/src**

This CSS code sets global styling rules for all elements and the body. It applies the border-box box-sizing model to every element and its pseudo-elements. It also removes default body margins, sets a light gray background with dark text, and enables font smoothing for better readability on WebKit browsers.

```
*,
*::before,
*::after {
  box-sizing: border-
box; /* Includes padding and border in element's total width/height */
}

body {
  margin: 0; /* Remove default browser margin */
  background: #f3f4f6; /* Light gray background */
  color: #1a1a1a; /* Dark text color for contrast */
  -webkit-font-
smoothing: antialiased; /* Improves font rendering on WebKit browsers */
}
```

FilterBar.module.css

Found in folder: **/src/components**

© 2026 Zenva Pty Ltd. All rights reserved.

This CSS defines styles for a filter bar component, including layout for the bar itself and interactive buttons for filtering. The filter buttons change appearance on hover and when active to provide visual feedback. A count element is styled to align to the right with subdued coloring.

```
.bar {
  display: flex;
  align-items: center;
  gap: 8px;
  margin-bottom: 16px;
  flex-wrap: wrap;
}

.filterButton {
  padding: 6px 14px;
  font-size: 0.875rem;
  border: 2px solid #d1d5db;
  border-radius: 20px;
  background: #fff;
  color: #374151;
  cursor: pointer;
  transition: all 0.15s; /* Smooth transition for hover/active states */
}

.filterButton:hover {
  border-color: #6366f1;
  color: #6366f1;
}

.filterButton.active {
  background: #6366f1;
  border-color: #6366f1;
  color: #fff;
}

.count {
  margin-left: auto; /* Pushes count to the right side of the flex container */
  font-size: 0.875rem;
  color: #9ca3af;
}
```

TaskInput.module.css

Found in folder: **/src/components**

This CSS file defines styles for a form component with an input field, a button, and a character counter. It styles the layout, appearance, and interactive states like focus and hover. The counter text changes color when a limit is reached.

```
.wrapper {
  margin-bottom: 16px;
}

.inputRow {
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
  display: flex;
  gap: 8px; /* Space between input and button */
}

.input {
  flex: 1; /* Input expands to fill available space */
  padding: 10px 14px;
  font-size: 1rem;
  border: 2px solid #d1d5db;
  border-radius: 6px;
  outline: none;
  background: #fff;
  transition: border-color 0.15s;
}

.input:focus {
  border-color: #6366f1; /* Highlight border on focus */
}

.button {
  padding: 10px 18px;
  font-size: 1rem;
  font-weight: 600;
  background: #6366f1;
  color: #fff;
  border: none;
  border-radius: 6px;
  cursor: pointer;
  white-space: nowrap; /* Prevent button text from wrapping */
  transition: background 0.15s;
}

.button:hover {
  background: #4f46e5; /* Darker shade on hover */
}

.counter {
  margin: 4px 0 0;
  font-size: 0.8rem;
  color: #6b7280;
  text-align: right;
}

.atLimit {
  color: #ef4444; /* Red text to indicate limit reached */
  font-weight: 600;
}
```

TaskItem.module.css

Found in folder: **/src/components**

This CSS styles a list item component with a checkbox, text, and delete button. It defines visual

© 2026 Zenva Pty Ltd. All rights reserved.

states for hover, completion, and editing. The layout uses flexbox for alignment and spacing.

```
.item {
  display: flex;
  align-items: center;
  gap: 12px;
  padding: 12px 14px;
  background: #fff;
  border: 1px solid #e5e7eb;
  border-radius: 8px;
  transition: background 0.1s;
}

.item:hover {
  background: #f9fafb; /* Subtle hover feedback */
}

.checkbox {
  width: 18px;
  height: 18px;
  flex-shrink: 0; /* Prevent checkbox from shrinking */
  cursor: pointer;
  accent-color: #6366f1; /* Custom color for the checkbox */
}

.text {
  flex: 1; /* Take up remaining space */
  font-size: 0.95rem;
  color: #1a1a1a;
  word-break: break-word; /* Prevent text overflow */
}

.completed .text {
  text-decoration: line-through;
  color: #9ca3af; /* Muted color for completed items */
}

.editInput {
  flex: 1;
  min-width: 0; /* Allows input to shrink below its default minimum */
  padding: 6px 8px;
  font-size: 0.95rem;
  color: #1a1a1a;
  background: #fff;
  border: 2px solid #6366f1; /* Highlighted border for edit state */
  border-radius: 6px;
  outline: none;
}

.deleteButton {
  flex-shrink: 0;
  padding: 4px 8px;
  font-size: 0.75rem;
  background: transparent;
  border: 1px solid transparent;
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
  border-radius: 4px;
  cursor: pointer;
  color: #9ca3af;
  line-height: 1;
  transition: all 0.15s;
}

.deleteButton:hover {
  background: #fee2e2;
  border-color: #fca5a5;
  color: #ef4444; /* Red color scheme on hover */
}
```

TaskList.module.css

Found in folder: **/src/components**

This CSS defines two classes: `.list` styles an unordered list as a vertical flex container with no default styling and spacing between items, while `.empty` styles an element for displaying empty states with centered, muted text and vertical padding.

```
.list {
  list-style: none;   /* Remove default bullet points */
  margin: 0;          /* Remove default margin */
  padding: 0;         /* Remove default padding */
  display: flex;      /* Use flexbox layout */
  flex-direction: column; /* Stack children vertically */
  gap: 8px;           /* Space between child elements */
}

.empty {
  text-align: center; /* Center text horizontally */
  color: #9ca3af;     /* Set a muted gray color */
  font-size: 0.9rem;  /* Use a slightly smaller font size */
  padding: 40px 0;    /* Add vertical padding */
  margin: 0;          /* Remove default margin */
}
```

TaskNote.module.css

Found in folder: **/src/components**

This CSS code defines two classes: `.item` creates a flex container with padding, border, and rounded corners for list items, while `.text` styles text elements to take up available space with specific font size and color.

```
.item {
  display: flex;                /* Use flexbox layout */
  align-items: center;          /* Vertically center child elements */
  gap: 12px;                    /* Space between child elements */
  padding: 8px 14px;           /* Internal spacing */
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
  margin: 0;                    /* Remove default margins */
  background: #fff;             /* White background */
  border: 1px solid #e5e7eb;    /* Light gray border */
  border-radius: 8px;           /* Rounded corners */
}

.text {
  flex: 1;                      /* Take up remaining space in flex container */
  font-size: 0.85rem;           /* Smaller font size */
  color: #6b7280;               /* Medium gray text color */
  word-break: break-word;       /* Break long words to prevent overflow */
}
```

audio.js

Found in folder: **/src/utils**

This code defines a function that plays a short rising sine wave sound when called. It creates an audio oscillator that sweeps from a lower to a higher frequency while fading in and out in volume. The function handles browser audio context initialization and gracefully fails if audio playback is blocked.

```
const COMPLETION_START_FREQUENCY = 660
const COMPLETION_END_FREQUENCY = 880
const COMPLETION_RAMP_DURATION = 0.08
const COMPLETION_DURATION = 0.18
const COMPLETION_START_VOLUME = 0.001
const COMPLETION_PEAK_VOLUME = 0.08
const COMPLETION_ATTACK_DURATION = 0.01

let completionAudioContext

export function playCompletionSound() {
  // Exit if not in browser environment
  if (typeof window === 'undefined') return

  const AudioContext = window.AudioContext || window.webkitAudioContext
  if (!AudioContext) return

  try {
    // Reuse audio context if available, otherwise create new one
    const audioContext = completionAudioContext || new AudioContext()
    completionAudioContext = audioContext

    if (audioContext.state === 'suspended') {
      audioContext.resume()
    }

    const oscillator = audioContext.createOscillator()
    const gain = audioContext.createGain()
    const startTime = audioContext.currentTime

    oscillator.type = 'sine'
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
    oscillator.frequency.setValueAtTime(COMPLETION_START_FREQUENCY, startTime)
    oscillator.frequency.exponentialRampToValueAtTime(
      COMPLETION_END_FREQUENCY,
      startTime + COMPLETION_RAMP_DURATION,
    )

    // Create volume envelope: fade in quickly, then fade out
    gain.gain.setValueAtTime(COMPLETION_START_VOLUME, startTime)
    gain.gain.exponentialRampToValueAtTime(
      COMPLETION_PEAK_VOLUME,
      startTime + COMPLETION_ATTACK_DURATION,
    )
    gain.gain.exponentialRampToValueAtTime(
      COMPLETION_START_VOLUME,
      startTime + COMPLETION_DURATION,
    )

    oscillator.connect(gain)
    gain.connect(audioContext.destination)
    oscillator.start(startTime)
    oscillator.stop(startTime + COMPLETION_DURATION)
  } catch {
    // Audio playback can be blocked by browser policy or unavailable in tests.
  }
}
```

taskConfig.js

Found in folder: **/src/utils**

This code defines a constant for the maximum allowed length of task text. It exports this constant so it can be imported and used in other files. This ensures a consistent text length limit is enforced across the application.

```
// Maximum allowed character length for task text
export const TASK_TEXT_MAX_LENGTH = 100
```

Daily Reader App

AGENTS.md

Found in folder: **/**

This file provides documentation for a React + Vite article showcase project used as a coding-course starter. It outlines the project's technology stack, available commands, and development conventions. It also lists critical rules that must be followed, such as avoiding routing and external UI libraries.

```
# AGENTS.md

## Project
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
A React + Vite article showcase site built as a coding-course starter project. Studen
ts work through exercises by completing missing features in this codebase.

## Stack

- **React 19** — functional components, hooks
- **Vite 7** — dev server and bundler
- **CSS Modules** — scoped component styles, no external UI libraries

## Commands

```bash
npm run dev       # start dev server at http://localhost:5173
npm run build     # production build
npm run preview   # preview production build
npm run lint      # run ESLint
```

Conventions

- One component per file; component file and its CSS Module share the same base name.
- Props flow down only — no context, no global state store.
- State lives in `App.jsx`; components receive data and callbacks via props.
- CSS class names use camelCase in JS (`styles.backButton`).
- Article data lives in `src/data/articles.js` as a plain JS array; no API calls.

Critical Rules

- **Do not add routing.** Navigation between list and detail views is managed by a si
ngle `selectedArticle` state in `App.jsx`. Introducing React Router or any router bre
aks the intended teaching structure.
- **Do not add external UI libraries.** No Tailwind, no MUI, no Chakra. Only CSS Modu
les and plain CSS.
- **Do not add a backend or localStorage.** All data is static and lives in `src/data
/articles.js`.
- **Missing features may be intentional.** This is a course starter — gaps in the cod
e are often student exercises. Do not silently add features without being explicitly
asked.
- **Keep dependencies minimal.** Only `react`, `react-dom`, and Vite-
related devDependencies are permitted.
```

README.md

Found in folder: **/**

This file contains documentation explaining the setup and features of a React project using Vite, including available plugins, React Compiler details, and ESLint expansion recommendations.

```
# React + Vite

This template provides a minimal setup to get React working in Vite with HMR and some
 ESLint rules.
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
Currently, two official plugins are available:

- [@vitejs/plugin-react](https://github.com/vitejs/vite-plugin-react/blob/main/packag
es/plugin-react) uses [Babel](https://babeljs.io/) (or [oxc](https://oxc.rs) when use
d in [rolldown-vite](https://vite.dev/guide/rolldown)) for Fast Refresh
- [@vitejs/plugin-react-swc](https://github.com/vitejs/vite-plugin-react/blob/main/pa
ckages/plugin-react-swc) uses [SWC](https://swc.rs/) for Fast Refresh

## React Compiler

The React Compiler is enabled on this template. See [this documentation](https://reac
t.dev/learn/react-compiler) for more information.

Note: This will impact Vite dev & build performances.

## Expanding the ESLint configuration

If you are developing a production application, we recommend using TypeScript with ty
pe-aware lint rules enabled. Check out the [TS template](https://github.com/vitejs/vi
te/tree/main/packages/create-vite/template-react-ts) for information on how to integr
ate TypeScript and [`typescript-eslint`](https://typescript-
eslint.io) in your project.
```

SKILL.md

Found in folder: **/.codex/skills/add-article**

This code defines a GitHub Actions workflow that automates the process of researching and adding a new article to a React/Vite project’s data file. It provides step-by-step instructions for inspecting the existing data structure, researching a topic, drafting content, and validating the changes. The workflow ensures the new article matches the project’s formatting and style guidelines before appending it to the file.

```
---
name: add-article
description: Research a requested topic, write a 200-250 word general-audience articl
e, and add it to this React/Vite course project's src/data/articles.js file using the
 existing article data structure. Use when the user asks to add, create, research, or
 write a new article entry for the static article showcase data file.
---

# Add Article

## Workflow

1. Inspect `src/data/articles.js` before editing.
   - Preserve the existing exported data structure, property order, indentation, quot
e style, and template-literal content style.
   - Choose the next numeric `id` after the highest existing article id.
   - Use `author: "Editorial Team"` unless the user explicitly provides another autho
r.
   - Use today's date from the runtime context in `YYYY-MM-
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
DD` format unless the user provides a different date.

2. Research the requested topic before drafting.
   - Use reliable sources appropriate to the topic, preferring primary or authoritati
ve sources.
   - For current, technical, scientific, medical, legal, or financial topics, verify
facts with up-to-date sources.
   - Keep notes source-grounded, but do not add citations inside the article body unl
ess the existing article style already does.

3. Draft the article.
   - Write for a general audience.
   - Keep `content` between 200 and 250 words.
   - Use accurate, cautious wording where experts still debate details.
   - Provide a concise `title` and `summary` matching the style of the existing entri
es.
   - Avoid unsupported claims, jargon-heavy explanations, and promotional tone.

4. Edit `src/data/articles.js`.
   - Append one new object to the existing array.
   - Do not add routing, API calls, localStorage, new dependencies, or unrelated feat
ures.
   - Do not rewrite existing article content unless the user explicitly asks.

5. Validate the change.
   - Count only the words in the new `content` field and confirm it is 200-250 words.
   - Run `npm run lint` when available.
   - In the final response, mention the added file, article word count, lint result,
and sources used for research.

## Implementation Notes

- Use `apply_patch` for manual edits.
- If shell quoting makes word counting against template literals brittle, rerun with
safer quoting or inspect the object another way.
- Keep the final response brief and focused on what changed and what was verified.
```

content.md

Found in folder: **/docs**

This file provides documentation for managing articles in the data file. It explains the structure of article objects and gives step-by-step instructions for adding or removing articles. It also includes content guidelines for writing summaries and full articles.

```
# Content Guide

## Article data structure

All articles live in `src/data/articles.js` as elements of a plain JavaScript array e
xported as the default export.
```

© 2026 Zenva Pty Ltd. All rights reserved.

````
Each article object has the following shape:

```js
{
  id: 1,                    // number — unique identifier; increment for each new art
icle
  title: "...",             // string — displayed on the card and as the article head
ing
  summary: "...",           // string — one sentence; shown on the card only
  content: `...`,           // string — full article text; shown in the detail view
  author: "Editorial Team", // string — displayed on card and detail view
  date: "2025-11-15",       // string — ISO 8601 date (YYYY-MM-DD)
}
```

## How to add a new article

1. Open `src/data/articles.js`.
2. Copy the shape above and fill in all fields.
3. Give it the next available `id` (increment the highest existing id by 1).
4. Add the object to the `articles` array.
5. Save the file — the dev server will hot-
reload and the new card will appear on the home page.

No imports, no API calls, no database changes needed.

## Content guidelines

- **summary**: One sentence, ? 160 characters. Should make a reader want to click.
- **content**: Plain prose paragraphs separated by a blank line (`\n\n`). The detail
view splits on `\n\n` to render each paragraph as its own `

` tag. Aim for 200–300 words.
- **date**: Use a realistic date in `YYYY-MM-DD` format. The date is displayed as-
is; no formatting is applied.
- **author**: Currently always `"Editorial Team"`. If per-
author names are added later, update the card and detail templates accordingly.

## Removing an article

Delete the object from the array. The list re-
renders automatically. Ids only need to be unique, not contiguous.
````

ui.md

Found in folder: **/docs**

This markdown file documents UI conventions and styling guidelines for a project using CSS Modules. It defines color tokens, typography rules, layout specifications, and interactive states. The document also establishes naming conventions and explains the styling approach.

```
# UI Conventions
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
## Styling approach

All component styles use **CSS Modules** (`.module.css` files). Class names are impor
ted as a `styles` object and applied via `className={styles.myClass}`. There are no g
lobal utility classes and no external UI frameworks.

A minimal global reset lives in `src/index.css` and is imported once in `src/main.jsx
`. It sets `box-sizing: border-box`, resets `body` margin, and defines the base font
stack. Do not add component-specific rules here.

## Colour palette

| Token | Value | Usage |
|---|---|---|
| Background | `#f4f5f7` | Page background (`body`) |
| Surface | `#ffffff` | Cards, article containers |
| Border | `#e5e7eb` | Card borders, dividers |
| Text primary | `#1a1a2e` | Headings |
| Text body | `#374151` | Article body copy |
| Text muted | `#4b5563` – `#9ca3af` | Summaries, meta lines |
| Accent | `#2563eb` | Buttons, site name |
| Accent hover | `#1d4ed8` | Button hover state |

## Typography

- **Font stack**: `system-ui, -apple-system, BlinkMacSystemFont, "Segoe UI", sans-
serif`
- **Line height**: `1.6` for body text; `1.75` inside article content paragraphs
- **Heading weights**: `800` for page/article headings; `700` for card titles

## Layout

- Page content is centred with a `max-
width` on the container: `1000px` for the list view, `720px` for the detail view.
- Cards are laid out in a responsive CSS Grid: `repeat(auto-
fill, minmax(300px, 1fr))` — no media queries required.
- Cards use `flex-direction: column` internally so the "Read article" button always s
its at the bottom.

## Interactive states

- Cards: `transform: translateY(-3px)` + increased box-shadow on `:hover`.
- Buttons: background colour darkens on `:hover` via a CSS `transition`.
- All transitions use `ease` timing at `0.15s`–`0.18s`.

## Naming

CSS Module class names use camelCase (e.g., `.backButton`, `.siteHeader`) so they map
 cleanly to JS property access.
```

App.jsx

© 2026 Zenva Pty Ltd. All rights reserved.

Found in folder: **/src**

This code renders a news reading app with a header and main content area. It displays either a list of articles or a detailed view of a selected article. The user can click an article from the list to see its details and then return to the list.

```
import { useState } from 'react';
import ArticleList from './components/ArticleList';
import ArticleDetail from './components/ArticleDetail';
import articles from './data/articles';
import styles from './App.module.css';

const App = () => {
  // Tracks which article, if any, is currently selected for detailed view
  const [selectedArticle, setSelectedArticle] = useState(null);

  return (
```

App.module.css

Found in folder: **/src**

This CSS code defines styles for a web application’s layout and header. It ensures the main container fills at least the full viewport height and creates a fixed-height header with a bottom border. The header contains a site name styled with specific typography and color.

```
.app {
  min-height: 100vh; /* Ensures the app container takes at least the full viewport he
ight */
}

.siteHeader {
  background: #ffffff;
  border-
bottom: 1px solid #e5e7eb; /* Adds a subtle separator line below the header */
  padding: 0 1.5rem;
  height: 56px; /* Sets a fixed height for the header */
  display: flex;
  align-items: center; /* Vertically centers content inside the header */
}

.siteName {
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
  font-size: 1.1rem;
  font-weight: 800;
  color: #2563eb; /* Primary brand color for the site name */
  letter-spacing: -0.02em; /* Slightly tightens letter spacing for a modern look */
}
```

index.css

Found in folder: **/src**

This CSS code sets global styling rules for all elements and the body. It applies border-box sizing to every element and its pseudo-elements. It also defines default styles for the body, including removing margins, setting a font stack, background color, text color, and line height.

```
*,
*::before,
*::after {
  /* Use border-
box model for all elements, making width/height include padding and border */
  box-sizing: border-box;
}

body {
  margin: 0; /* Remove default browser margin */
  font-family: system-ui, -apple-system, BlinkMacSystemFont, "Segoe UI", sans-
serif; /* Modern system font stack */
  background-color: #f4f5f7; /* Light gray background */
  color: #1a1a2e; /* Dark blue-gray text color */
  line-height: 1.6; /* Improved readability with increased line spacing */
}
```

ArticleCard.module.css

Found in folder: **/src/components**

This CSS defines styles for a card component and its child elements. It sets up a visually distinct card with a hover effect that lifts and adds shadow. The card contains a title, metadata, summary text, and a button, each styled for hierarchy and interaction.

```
.card {
  background: #ffffff;
  border: 1px solid #e5e7eb;
  border-radius: 10px;
  padding: 1.5rem;
  cursor: pointer; /* Indicates the card is clickable */
  display: flex;
  flex-direction: column;
  gap: 0.5rem; /* Spacing between child elements */
  transition: transform 0.18s ease, box-shadow 0.18s ease;
  box-shadow: 0 1px 4px rgba(0, 0, 0, 0.06);
}
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
.card:hover {
  transform: translateY(-3px); /* Lifts card on hover */
  box-shadow: 0 6px 20px rgba(0, 0, 0, 0.1);
}

.title {
  margin: 0;
  font-size: 1.15rem;
  font-weight: 700;
  color: #1a1a2e;
  line-height: 1.35;
}

.meta {
  margin: 0;
  font-size: 0.8rem;
  color: #9ca3af;
  letter-spacing: 0.02em;
}

.summary {
  margin: 0;
  font-size: 0.95rem;
  color: #4b5563;
  flex: 1; /* Allows summary to grow and fill available space */
}

.button {
  align-self: flex-start;
  margin-top: 0.5rem;
  padding: 0.45rem 1rem;
  background: #2563eb;
  color: #ffffff;
  border: none;
  border-radius: 6px;
  font-size: 0.85rem;
  font-weight: 600;
  cursor: pointer;
  transition: background 0.15s ease;
}

.button:hover {
  background: #1d4ed8;
}
```

ArticleDetail.module.css

Found in folder: **/src/components**

This CSS file defines styles for a container, a back button, and an article layout including its title, metadata, and content paragraphs. It sets visual properties like colors, spacing, borders, and hover effects to create a clean, responsive design. The styles ensure consistent typography and interactive

© 2026 Zenva Pty Ltd. All rights reserved.

feedback for the user interface.

```
.container {
  max-width: 720px;          /* Center content with a maximum width */
  margin: 0 auto;            /* Horizontal centering */
  padding: 2rem 1.5rem;      /* Internal spacing */
}

.backButton {
  background: none;
  border: 1px solid #d1d5db;
  border-radius: 6px;
  padding: 0.4rem 0.85rem;
  font-size: 0.875rem;
  color: #4b5563;
  cursor: pointer;           /* Indicates clickable element */
  transition: background 0.15s ease, color 0.15s ease; /* Smooth hover effects */
  margin-bottom: 2rem;       /* Space below button */
}

.backButton:hover {
  background: #f3f4f6;       /* Light background on hover */
  color: #1a1a2e;            /* Darker text color on hover */
}

.article {
  background: #ffffff;
  border: 1px solid #e5e7eb;
  border-radius: 10px;
  padding: 2rem 2.5rem;
  box-shadow: 0 1px 4px rgba(0, 0, 0, 0.06); /* Subtle shadow for depth */
}

.title {
  margin: 0 0 0.5rem 0;
  font-size: 1.75rem;
  font-weight: 800;
  line-height: 1.3;
  color: #1a1a2e;            /* Dark color for prominence */
}

.meta {
  margin: 0 0 1.75rem 0;
  font-size: 0.8rem;
  color: #9ca3af;            /* Muted color for secondary information */
  letter-spacing: 0.02em;
}

.content p {
  margin: 0 0 1.2rem 0;      /* Spacing between paragraphs */
  font-size: 1rem;
  color: #374151;
  line-height: 1.75;         /* Improved readability */
}
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
.content p:last-child {
  margin-bottom: 0;          /* Remove bottom margin from last paragraph */
}
```

ArticleList.module.css

Found in folder: **/src/components**

This CSS defines styles for a layout container, header, and responsive grid. It sets up a centered content section with a header containing aligned title elements. The grid creates flexible columns that automatically adjust based on available space.

```
.section {
  max-width: 1000px;        /* Constrain width for large screens */
  margin: 0 auto;           /* Center horizontally */
  padding: 2rem 1.5rem;     /* Internal spacing */
}

.header {
  margin-bottom: 2rem;      /* Space below header */
  display: flex;
  align-items: baseline;    /* Align text baselines */
  gap: 1rem;                /* Spacing between child elements */
}

.heading {
  margin: 0;
  font-size: 1.75rem;
  font-weight: 800;
  color: #1a1a2e;           /* Dark primary color */
}

.subheading {
  margin: 0;
  font-size: 0.875rem;
  color: #9ca3af;           /* Muted secondary color */
}

.grid {
  display: grid;
  /* Responsive columns: fill available space, minimum 300px each */
  grid-template-columns: repeat(auto-fill, minmax(300px, 1fr));
  gap: 1.25rem;             /* Spacing between grid items */
}
```

articles.js

Found in folder: **/src/data**

This code defines an array of article objects, each containing metadata and content for a blog post. It exports the array as the default export for use in other parts of the application.

© 2026 Zenva Pty Ltd. All rights reserved.

```
const articles = [
  {
    id: 1,
    title: "How the James Webb Space Telescope Works",
    summary:
      "A look at the mirror design, infrared imaging, and what JWST has already revea
led about the early universe.",
    content: `The James Webb Space Telescope, launched December 25, 2021, is the most
 powerful space observatory ever built. At its heart is a 6.5-meter primary mirror co
mposed of 18 hexagonal gold-plated beryllium segments — more than six times the light
-collecting area of Hubble. That gold coating is deliberate: it maximizes reflectivit
y at infrared wavelengths, where Webb does most of its work.

Unlike Hubble, which observes primarily in visible and ultraviolet light, Webb operat
es across near- and mid-infrared wavelengths (0.6 to 28 micrometers). Infrared light
passes through dust clouds that block visible light, letting Webb peer inside stellar
 nurseries and the cores of galaxies. It also captures the heavily redshifted light o
f the most distant galaxies — objects so far away that their light has been traveling
 for over 13 billion years, stretched from ultraviolet into the infrared by the expan
sion of the universe.

To detect this faint heat radiation, Webb's instruments must be kept extraordinarily
cold. A five-layer sunshield the size of a tennis court blocks heat from the Sun, Ear
th, and Moon, holding the telescope at around ?233°C (?387°F).

Since beginning science operations in 2022, Webb has delivered remarkable results: th
e deepest infrared view of the universe, the first direct detection of carbon dioxide
 in an exoplanet's atmosphere, and images of galaxies that formed within 300 million
years of the Big Bang — reshaping our understanding of early cosmic history.`,
    author: "Editorial Team",
    date: "2025-11-15",
  },
  {
    id: 2,
    title: "Why Sleep Matters for Learning and Memory",
    summary:
      "What happens in your brain overnight, and why skipping sleep quietly undoes th
e studying you did that day.",
    content: `Sleep is not passive downtime — it is when the brain does much of its m
ost important work. During the hours you sleep, neural circuits that fired during the
 day are replayed and reorganized, converting fragile short-
term impressions into durable long-term memories.

The process depends on sleep's distinct stages. During non-REM slow-wave sleep — the
deep, dreamless stages — the hippocampus, the brain's short-term memory hub, systemat
ically reactivates the day's experiences and transfers them to the neocortex for perm
anent storage. Characteristic bursts of neural activity called sleep spindles help bi
nd new information into existing knowledge networks.

REM sleep, which intensifies during the later cycles of the night, serves a different
 but complementary role. Brain imaging shows the prefrontal cortex goes quiet while e
motional and associative regions remain highly active — the conditions for creative i
nsight, flexible thinking, and the processing of emotionally charged memories. Cuttin
g a night short preferentially reduces REM, which is why even moderate sleep restrict
ion blunts creative problem-solving the next day.
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
The practical implications are clear. Students who sleep after studying recall signif
icantly more than those who stay awake for the same period. A 90-minute nap taken bet
ween a learning session and a test can nearly double retention. Chronic sleep depriva
tion gradually shrinks the hippocampus and impairs the formation of new memories. The
 research is unambiguous: for learning to stick, sleep is not optional — it is part o
f the process.`,
    author: "Editorial Team",
    date: "2025-12-02",
  },
  {
    id: 3,
    title: "How Black Holes Are Formed",
    summary:
      "How dying stars, cosmic mergers, and the early universe can create some of the
 densest objects in space.",
    content: `Black holes form when gravity compresses enough matter into a region so
 dense that not even light can escape. The best understood route begins with a very m
assive star. For most of its life, the star holds itself up by fusing lighter element
s into heavier ones, releasing energy that pushes outward against gravity. Eventually
, the core runs out of useful fuel. If the remaining core is massive enough, gravity
wins: the core collapses while the outer layers may blast away in a supernova.

What remains depends on the mass of the core. Less extreme collapses can leave behind
 a neutron star, an object only about the size of a city. But if the core is heavier,
 it keeps shrinking until it forms a stellar-mass black hole. NASA estimates that sta
rs born with about 20 times the Sun's mass or more can end this way.

Black holes can also grow after they form. They may pull in gas from nearby stars, sw
allow other compact objects, or merge with other black holes. These mergers release g
ravitational waves, ripples in space-time that observatories on Earth can detect.

The largest black holes, found in the centers of galaxies, are harder to explain. The
y may have grown from smaller black holes, formed through repeated mergers, or begun
as massive "seeds" from collapsing gas clouds in the young universe. Scientists are s
till testing these ideas.`,
    author: "Editorial Team",
    date: "2026-06-10",
  },
  {
    id: 4,
    title: "The Science of Habit Formation",
    summary:
      "Why repeated actions become automatic, and how cues, rewards, and context shap
e everyday behavior.",
    content: `Habits are the brain's way of saving effort. Instead of deciding from s
cratch every time you brush your teeth, check your phone, or make coffee, your brain
learns to link a situation with an action. Psychologists call this a context-response
 association: a cue such as a time, place, emotion, or preceding action starts a beha
vior with little conscious planning.

At first, a new habit usually needs intention. You decide to stretch after waking up
or read before bed. Repetition in the same context gradually makes the action easier
to trigger. Rewards help because they tell the brain the behavior was worth repeating
. The reward does not have to be dramatic; a sense of relief, pleasure, progress, or
```

© 2026 Zenva Pty Ltd. All rights reserved.

```
reduced friction can strengthen the pattern.

Neuroscience points to the basal ganglia, a group of deep brain structures involved i
n learning routines, selecting actions, and processing reward. As a habit strengthens
, control can shift away from deliberate, goal-focused thinking and toward faster aut
omatic responses. That is why habits can be helpful when they support goals, but stub
born when they do not.

There is no universal "21-day" rule. In one real-world study, automaticity often took
 weeks to months and varied widely by person and behavior. The practical lesson is si
mple: make the desired action easy, repeat it in a stable context, and attach it to a
 reward you actually feel.`,
    author: "Editorial Team",
    date: "2026-06-11",
  },
];

// Exports the array of article objects as the default module export
export default articles;
```

© 2026 Zenva Pty Ltd. All rights reserved.